Berkely soft manual

Implementation of a complete Berkeley Snobol4 system for console-based applications

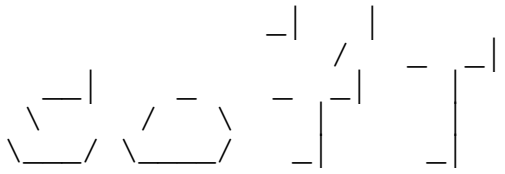
Antonio Maschio

Design, implementation and Study
for a quite complete integration with the Bell Snobol4

Independent Design and Realization

23 June 2026

ABSTRACT



Introducing soft, my Snobol4 interpreter. soft, which is to be written always in lower letters, means '**S**nobol **OF T**ony'. This means that soft is not an official version of Snobol4, but my personal version, basically obeying to the rules of the so called *Berkeley version*, described in the book by Robert Gaskins and Laura Gould, dating Spring 1972, which is a wonderful manual about this language, defined by their authors as the "Computer Programming Language for the Humanities". Some sections of the original manual weren't completed and probably will never be, but I hope this manual is.

In any case, I hope you like it.

Read also the man page for other more general info, all options description and the installing instructions.

I tried to project this software avoiding all causes of segmentation faults and memory errors, but I fear there could be some left. You are encouraged to report all errors and inconsistencies found (both in the program and in the manual) to:

ing dot antonio dot maschio at gmail dot com

I don't ask money for this program. I only ask you for a voluntary contribution, to be made through PayPal (any amount), just to support my work and let me know you like it. Donations can be directed to

tbin at libero dot it

Thanks in advance to anyone of you who will contribute. And thanks anyway to anyone of you who will download and use soft!

Thanks.

This work is dedicated to the memory
of Ralph E. Griswold *(1934-2006)*
American computer scientist

and of Robert Gaskins Jr *and* Laura Gould
authors of "A Computer Programming Language for the Humanities"
University of California, Spring 1972

1. INTRODUCTION

Extracts from the original preface:

"*Edmund Fuller*[1] *has described hearing an interview in which Edward R. Murrow*[2] *asked Mickey Spillane*[3] *how he could bring himself to pander to the public taste by writing the kind of books he did: Spillane's luminous reply, according to Fuller, was: "*I write the kind of books I want to read and can't find*."*

We[4]*, with much the same motivation, have written this description of Snobol4, a computer programming language for the humanities. Our own training and interest is in the study of language and literature, and so the examples and exercises are directed particularly toward the machine manipulation of linguistic data and literary texts. Even so, the description should be useful to students of many disciplines, since the first part of each chapter presents features of the language in a generalized way, and the particular examples in the second part of each chapter have been chosen to exhibit principles and techniques which can easily be applied to verbal or symbolic data in a wide range of humanistic and social science applications.*"

This preface, beyond the great prefixed scopes, states one much important thing: Snobol is for everyone, but not without study and application. Snobol is not an easy language (easy in the sense of *readable*), if Gaskins and Gould felt the urge to write a manual. It's a beautiful and powerful language, yes, and it offers many different solutions to solve the same problems, but it's not *easy*. Yet, the Snobol engine works on few concepts and the language has a small amount of commands, at least for the basic core, and I believe that learning this language has made me a better programmer.

1.1. What is this manual?

This manual illustrates soft, my Snobol4 interpreter. soft must be written always in lower letters (so no Soft, SOFT or the like) and it is principally based on the Berkeley version described in the book *Snobol4 - A Computer Programming Language for the Humanities*, by Robert Gaskins and Laura Gould, published by the University of California in Spring 1972, in which the Berkeley version of the software is exposed. You may ask: "Why did you choose to design a Berkeley version and not a Bell version?"

I decided to design a Berkeley version for the following reasons:

- It is the first version I studied;
- It requires less statements than the Bell version;
- The manual is shorter and most manageable than the famous Green Book;
- I couldn't find any other interpreter understanding the Berkeley version, so mine could have some chance to please someone;
- It has a very beautiful motivation in its principles (a language for the humanities! How could this motivation not attract me?)[5];

---

[1] Edmund Maybank Fuller (March 3, 1914 - January 29, 2001) was an American educator, editor, novelist, historian, and literary critic. Source: Wikipedia at https://en.wikipedia.org/wiki/Edmund_Fuller

[2] Edward Roscoe Murrow (April 25, 1908 - April 27, 1965), born Egbert Roscoe Murrow, was an American broadcast journalist and war correspondent. Source: Wikipedia at https://en.wikipedia.org/wiki/Edward_R._Murrow

[3] Frank Morrison Spillane (March 9, 1918 - July 17, 2006), better known as Mickey Spillane, was an American crime novelist. Source: Wikipedia at https://en.wikipedia.org/wiki/Mickey_Spillane

[4] Supposedly Robert Gaskins Jr., an American computer science expert in applications to research in humanities (literature, art, music) and linguistics, at the University of California, Berkeley (who later will invent Powerpoint, a software program that Microsoft would buy and that would became the *de facto* standard for presentations), and Laura Gould, an American expert in linguistic, who also wrote with Robert W. Hsu, a member of the Department of Linguistics, University of Hawaii, the book *A Linguist's Introduction to SNOBOL*, a book that I'd read with pleasure, but I couldn't find it anywhere.

- The Gaskins and Gould manual is written on a typewriter, something that has always fascinated me. I can't help with it.

The original introduction to the language (by Gaskins and Gould) offers a first glance about programming languages; here are the authors speaking:

"*Snobol is a programming language, one of many such artificial languages which may be used to convey instructions to a computer. Most computers may be instructed in a wide variety of programming languages; these languages differ from one another, as do natural languages, by having different vocabularies and syntactic structures. More importantly, however, they differ in the range of concepts which they are capable of expressing.*

Different programming languages have been developed for different kinds of problems or problem areas. Some have been devised primarily for describing general numeric or algebraic problems, others for describing the structure of business records and files, still others for highly specific purposes such as controlling machine tools, simulating economic systems, or making computer-generated movies.

Snobol is distinguished by very powerful and general capabilities for manipulating strings of characters, making it particularly convenient for working with data elements such as linguistics, literature, verbal behavior, and the humanities in general, it's also very useful for expressing sophisticated non-numeric problems in the field of computer science."

1.2. A bit of history

Snobol was developed by Ralph E. Griswold, David J. Farber and Ivan P. Polonsky at the Bell Labs in 1962, far earlier than C or Pascal. Originally, it bore the name of String Extractor Interpreter, with the easy acronym of SEXI. (Sometimes, fantasy of men goes in one direction only...)

According to David Farber, the name Snobol came out later, after the publication in the JACM review (Journal of the Association for Computing Machinery) of an article related to the language; the story deserves to be told[6]. The Acronym SEXI went well until one day David was submitting a batch job to assemble the system and, as normal on job cards, (he punctuates, the first card in the deck) he wrote the job and his own name: SEXI Farber. One of the girls at the Center looked at it and said "*That's what* you *think*", in a humorous way. David: "*That made it clear that we needed another name! We sat and talked and drank coffee and shot rubber bands and after much to much time someone said, most likely Ralph,* «We don't have a snowball chance in hell of finding a name*». All of us yelled at once «*WE GOT IT! SNOBOL!*» in the spirit of all the BOL languages. We then stretched our mind to find what it stood for."*

The first version of Snobol was compiled in assembly language for the IBM 7090 (circa 1963), with the basic pattern evaluation engine, but still lacking the built-in procedures.

The second version (called Snobol2, 1964) had some built-in procedures, but it was with Snobol3 (circa 1965) that a modern and sophisticated language was born! Snobol3 could now offer programmer-defined procedures!

In 1966, the advent of third-generation computers with larger memories and faster CPU's eased the development of a new version, called Snobol4, with the addition of numeric data types, arrays, structures and finally tables, with an improved patterns evaluation engine. The initial design and implementation of Snobol4, according to Gaskins and Gould, was done at the Bell Telephone Laboratories on an IBM System 360 machine. (This version of Snobol4 is known since then as the *Bell version*.)

Since then, several compilers and interpreters have been developed, mainly in the Universities circuit. Later language implementations were SPITBOL360 and SPITBOL370, Macro SPITBOL (a portable implementation, including MaxSPITBOL and SPITBOL-386), SITBOL, FASBOL, ELFBOL and others, collectively known as the Spitbol

[5] After many *technical* programming languages, as Algol60 (taxi), BASIC (decb and tbas), forth (tetra), more or less all related to math, science or computer science, a code devoted to literature and human languages analysis is like a breath of fresh air! And I confess here that, not knowing the exact meaning of "humanities" as *the branch of learning that includes arts, classics, literature, philosophy, history, etc.*, and thinking erroneously it meant *mankind*, it was the reason that made me decide to write an interpreter for Snobol4: how could I remain indifferent to a language conceived for the whole mankind? Later, the real meaning came to the surface, but the program was on its way and the new true scope, though not attractive as the original mistaken one, remained, nonetheless, a very good scope.

[6] Read the full story here: https://ip.topicbox.com/groups/ip/Ta3a72b87e1ed436c-M3254f266ea84487ee265da5c

family, introducing many new procedures, features and increasing the execution speed.

The original Snobol4 continued to live in the universities and among students and is still a tool for many projects about music analysis, language decomposition, symbolic mathematics, and many others. Nonetheless, the great variety of compilers/interpreters in use in those times did not follow common-rules sets, thus featuring many different procedures and characteristics that were hardly portable, making programs-sharing quite impracticable.

The definitive manual, known today as the *Green Book*, the basis for the Bell version (from now on GB), was written by Ralph E. Griswold, James F. Poage, and Ivan P. Polonsky and published by Prentice-Hall in 1971. This manual is freely available today as a pdf file here:

http://www.math.bas.bg/bantchev/place/snobol/gpp-2ed.pdf

In 1972, Robert Gaskins and Laura Gould felt the urge to write a draft manual, simpler and faster, about this language, fixing in some general concepts and describing the basic features of the language developed at the Computer Center of the University of California, known today as the Berkeley version. This version of Snobol4 was implemented by Paul McJones and Charles Simony for the CDC 6000 series machines. The Berkeley version was slightly different from the Bell version: the terminology was in part different, some features changed, some were missing, some added. But the pattern procedures and the pattern recognition mechanism is the same. The Gaskins and Gould manual describes the Berkeley version of Snobol4, and soft is an implementation of the language described by Gaskins and Gould.
The Gaskins and Gould manual (from now on GGM) was targeted for students and remains an unfinished opera since many chapters (mainly those bearing some examples) lacked. But it's written in a clear and concise way (sometimes even too much concise). The pdf scanned version can be downloaded from the site of Robert Gaskins:

https://www.robertgaskins.com/files/gaskins-gould-cal-snobol4-1972.pdf

Another important text is "The Macro Implementation os Snobol4 - A Case Study of Machine-Independent Software Development", that can be found here:

https://www.b-ok.cc/book/746532/64385f

available in djvu format, which can eventually be converted to pdf in many sites that offer free conversion tools. This book is useful along with its companion book "Implementing SNOBOL4 in SIL; Version 3.11", available here:

https://www.snobol4.org/doc/arizona/s4d58.pdf

I cannot end this documentary exposition without quoting the Vanilla Snobol project, by Mark B. Emmer, which was a great inspiration for me. It is a modern Bell rendition, with an interactive feature. You can find it in

https:/rosettacode.org/wiki/Vanilla_SNOBOL4

The Vanilla pdf manual can be downloaded here:

http:/www.math.bas.bg/bantchev/place/snobol/vtrm.pdf

For what about the language in general, see also the following sites:

http://www.snobol4.org/

rich of notes about the various implementations and with a complete documentation. Other sites are:

http:/cs.ecs.baylor.edu/~maurer/SieveE/snobol4.htm

http:/www.sachsdavis.clara.net/Snobol4Pdoc.html

Snobol was discussed also in the area of linguistic: see for instance, the essay "Programming Languages for Computational Linguistics", by Arnold C. Satterthwait, Washington State University, Pullman, Washington, available here:

https://coek.info/pdf-programming-languages-for-computational-linguistics-.html

Another interesting point of view is the older essay "Tutorial on generalized programming languages and systems", by Paul J. Fasana, Columbia University Libraries, New York, N. Y., and Russell Shank, Smithsonian Institution, Washington D. C., available here:

https://files.eric.ed.gov/fulltext/ED027743.pdf

In the latter two texts, Snobol is the discussed languages, but what is interesting in these essays is how Snobol is regarded, that is as a major language for treating and manipulating any type of language.

****

This manual, while it explains some Snobol4 features in detail, was not designed as a primer, so the descriptions of some Snobol4 procedures are brief and maybe they are not exhaustive. The GGM, or even the GB (for what applies to the Berkeley features also) should be consulted for a deeper knowledge of the required features.

2. INTRODUCTION TO SNOBOL

I begin this introduction with the same chapter in the GGM. I maintain it here, because it may prove useful for the beginner, introducing some common Snobol terms that are valid for soft as well.

2.1. Devising a program

by
Robert Gaskins, Jr.
Laura Gould

A description of how a computer is to go about solving a problem consist of a list of tasks or actions to be performed. A specification in some programming language which describes such a series of tasks completely is called a *program text*. Before a program text can be written, the task which it is to describe must be clearly understood. If, for example, a task has been expressed in English as *find all vowels in a word*, the following questions must be resolved before the programming of the task in some programming language can be undertaken:

(1) What is vowel?
(2) What is a word?
(3) What should be done with the vowels which are found?

The answers might be as follows:

(1) one of the characters A, E, I, O or U
(2) a string of characters to be provided as data to the program
(3) count them and then print the total

Given these clarifications, one can then translate the imprecise English sentence *find all vowels in a word* into a rigorous step-by-step description of what must be done. This step-by-step description can then be translated again into a series of statements in an appropriate programming language. The intermediate translation may exist only in the mind of the programmer, as is often the case if the task is a simple one, or may be recorded in some fashion so that it may be considered for correctness.

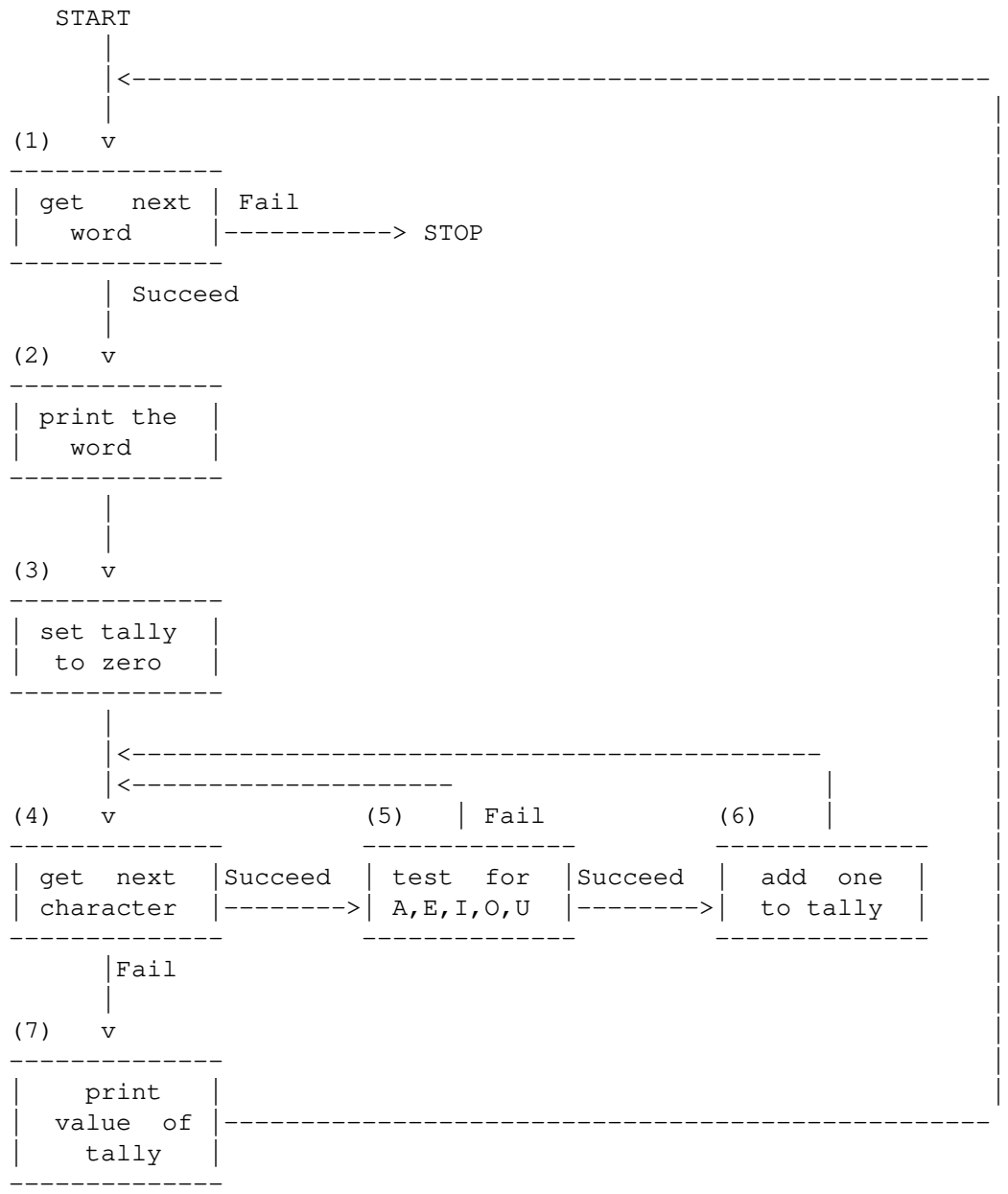
One of the best ways of recording a step-by-step description is to write down a series of numbered statements specifying exactly what is to be done. These statements are still in English, but a much more detailed and careful English than that of the original problem. The statements differ from the sentences of a natural language paragraph in that they are not intended to be processed only once or in the order in which they are presented; hence, the statements are numbered so that the order in which they are to be processed, often repeatedly, may be specified. A set of numbered statements describing how to count all the vowels in a series of words and to print the counts might look as follows:

```
START
(1) Get the next word; if no more words, STOP.
(2) Print that word.
(3) Set the tally to zero.
(4) Get the next character of this word; if no more characters remain,
    go to (7); otherwise go to the next statement.
(5) Determine whether or not this character is an A, E, I, 0 or U;
    if it's not, go back to (4); otherwise, go to the next statement.
(6) Add one to the tally which is keeping track of the number of vowels
    in this word; go back to (4).
(7) Print the value of the tally, which now represents the total number
    of vowels in the word.
(8) Go back to (1) and attempt to get another word.
```

Note that this program description has been augmented to count the vowels in any number of words, one after another, and to print the counts separately. It would not be useful to write a program to count the vowels in a single word only, as the counting could be accomplished by hand much faster than the program could be written. (However, for more complicated tasks, a program can often be written much more easily than the task can be performed even once by hand; that such a program could then be used again might well be of secondary importance.)

Another method of recording a step-by-step description is to use what is called a *flow chart*. In a flow chart the

specification of what is to be done next, or the *flow of control*, is indicated by means of lines and arrows rather than by phrases of the form *go back to (1).* A flow chart equivalent to the numbered statements just provided might look as follows:



[...] Now that a detailed method for solving the problem is clearly understood, it may be translated into a set of statements in the Snobol language. Seven Snobol statements are provided below, one for each of the numbered English sentences, or, equivalently, one for each box of the flow chart. These statements are provided here to illustrate the close correspondence between the Snobol statements and the step-by-step description, to give some indication of the appearance of a programming language, and to point out some features of the Snobol language in particular; a complete discussion of the meaning of these statements must be deferred to later chapters of the text. (comments, beginning with asterisks, have been inserted for spacing and to explain the purpose of the statements.)

(You can find the text of this program in the samples/ *directory of the installation package in the file* test.sno*; to execute it, step into this directory from the console and type*

```
$ soft --input=from test.sno
```

The from *textual file contains the input values and is contained in the same directory. The output will be directed to the screen.)*

```
* STEP 1:  READ IN THE NEXT WORD - IF NO MORE WORDS, STOP
*
READ     WORD  =  TRIM(INPUT)          :  F(END)
*
* STEP 2:  PRINT THE WORD JUST READ IN
*
         OUTPUT  = WORD
*
* STEP 3:  SET THE TALLY TO ZERO
*
         TALLY  =  0
*
* STEP 4:  GET THE NEXT CHARACTER OF THIS WORD - IF NO MORE
*          CHARACTERS, PRINT THE VOWEL COUNT FOR THIS WORD
*
GETCHAR  WORD  LEN(1) . CHAR = NULL    :   F(PRINT)
*
* STEP 5:  SEE IF THIS CHARACTER IS A VOWEL - IF NOT,
*          GO BACK AND GET NEXT CHARACTER
*
         CHAR  ANY('AEIOU')            :   F(GETCHAR)
*
* STEP 6:  CHARACTER IS A VOWEL - ADD ONE TO THE TALLY
*
         TALLY  =  TALLY + 1           :   (GETCHAR)
*
* STEP 7:  PRINT NUMBER OF VOWELS AND RETURN TO
*          READ IN THE NEXT WORD
*
PRINT    OUTPUT  =  TALLY       :   (READ)
*
END
```

If the file of data to be used as input for the program text above were the following list of words:

```
HIPPOPOTAMUS
HIPPOS
HIPPOSIDEROS
HIPPOSPONGIA
HIPPOTIGRINE
HIPPOTOMY
HIPPOTRAGINE
HIPPOTRAGUS
```

then the output produced by the program would be the list:

```
HIPPOPOTAMUS
5
HIPPOS
2
HIPPOSIDERO5
5
HIPPOSPONGIA
5
HIPPOTIGRINE
5
HIPPOTOMY
3
HIPPOTRAGINE
```

```
5
HIPPOTRAGUS
4
```

Results from executing a program may be printed on screen for personal perusal or written on file. Since the last is machine-readable as well as machine-writeable, the output may be used again, without modification, as input data to be further processed by still another program.

Here ends the introductory section by Gaskins and Gould. From now on, this manual - with its errors and inconsistencies - is due to me and me only. Address any error you may find to

```
ing dot antonio dot maschio at gmail dot com
```

and I will try to correct them ASAP.

By the way: the program above may be run from console as follows:

```
$ soft test.sno < from
```

The files `test.sno` and `from` are in the package.

2.2. Elements of the language

Programming in Snobol4 consists in writing program lines that obey to some rules, exposed in the following sections.

2.2.1. The first character

The first character of the line (the character at position 0 in the program text line) may influence the *meaning* of the line. Here is the list of recognized characters.

Asterisk

A line beginning with an asterisk, as already said, is interpreted as a comment and will be ignored during the execution phase. A comment extends to the end of line, and there is no technique for multi-line comments.

The content of a comment can be anything, because during the loading phase, a comment is simply ignored and won't be loaded at all. So even lines of code can be safely commented to disable them temporarily.

Grid

A line beginning with the grid # symbol is also interpreted as a comment, and extends to the end of line; it will also be ignored during the execution phase. It was introduced because soft is a Unix/Linux program, and thus it must be able to host the famous *she-bang* comment in the first line of a Snobol4 program:

```
#!/path/to/soft
...
LABEL   OUTPUT = ...
...
END
```

If the execution bits of this program, say `prog`, is changed as in the following:

```
$ chmod +x prog
```

the program will be treated as an executable; calling

```
$ prog
```

will execute the program (provided the current directory is in the `$PATH`), because the *she-bang* token calls the executable in the first line and gives it the file `prog` as argument. This is a bash feature, not a soft feature; soft's duty is only to ignore a line beginning with the character #.

Blanks

A line beginning with a blank (space or tab) is evaluated as a label-less Snobol4 instructions set, as explained further.

Plus or dot

A line beginning with a plus character or a dot is concatenated to the previous line, setting the current line as empty. For instance, the lines:

```
 [1]       LABEL1  ...something 1...
 [2]+ something 2...
 [3]       LABEL2  ...something 3...
```

(the numbers in square brackets are only indicative) will be loaded as if written as:

```
 [1]       LABEL1  ...something 1... something 2...
 [2]
 [3]       LABEL2  ...something 3...
```

(notice the empty middle line numbered 2). For the sake of clearness, since all lines are trimmed, before storing (trailing blanks are removed), when a line is joined to the previous, the "+" is discarded but all blanks following it are retained.

soft accepts also the dot as the first character of the line in place of the + to follow other implementations' behaviour. This is possible because a label must begin with an alphanumeric character, in the Berkeley version, and so the dot is correctly interpreted.

Caveat! At all effects, the joining of two lines alters the program code line numbering, so if using this feature is necessary, for instance to append one go-to section (see ahead) to the current line where there is no space left in the editor, you can have soft do the job by using the `-l/--list` option to list the whole program with all the reconstructed numbered lines before execution, or by using the `-L/--listing` option to list the whole program at the termination, with all the reconstructed numbered lines[7].

Cautions must be adopted in case one string is broken in two lines; see the following example:

```
    VAR = "FIRST PART"
+" SECOND PART"
    OUTPUT = VAR
```

Without any space between the + and the quote, with Berkeley the string would be recomposed as

```
"FIRST PART"" SECOND PART"
```

resulting in the output

```
FIRST PART" SECOND PART
```

which is probably what you didn't mean (the Bell version, instead, considered correctly the double instance, because it could see *two* strings there).

soft is conscious of this problem, and in this specific case (if a quote follows the +), it adds automatically a blank, to force the recognition of a single string instance. If you instead write something like:

```
    VAR = "FIRST PART
+ SECOND PART"
    OUTPUT = VAR
```

(that is *the string is broken*), soft joins the two strings, and you will see

[7] This is even more useful in case you program in the typical limit of 80 characters of many monitors.

```
STRING 1 STRING 2
```

but remember that other Snobol4 and Spitbol interpreters will see an error there, because they don't automatically close a string at the end of the line, as soft does.

Minus

A line beginning with a minus character has the scope of performing special procedures, collectively called *Directives*; blanks may appear between the minus sign and the directive name. One or more blanks must separate the specifier (e.g. `LIST` from `LEFT` or `RIGHT`). An erroneous or unrecognized directive line is simply ignored. See the dedicated chapter.

2.2.2. The programming elements

As can be seen in the previous chapter, a Snobol4 program is composed by a series of lines, each carrying its amount of information. Apart from the comments, a feature that any language has (and in this case, the lines starting with an asterisk), a command line is composed by three sections, none mandatory:

label *rule* *go-to*

Here follows something about them.

2.2.2.1. The label section

Whenever the first character of the line is an alphabetical letter, it is interpreted as the start of a label; it's the *label* section, that starts from the first character of the line, and extends indefinitely. Labels are used to identify a particular line that can be the destination of a jump.

A label extends to 2047 characters (plus the terminator) but, of course, this length is totally unmanageable... Generally, labels are made up of single words, or few particular letters combinations (see the previous program). Labels must begin with an alphabetic character, followed by alphanumeric characters or the dot.

At least one blank must follow the label, if a rule section follows.

2.2.2.2. The rule section

The *rule* section starts from the second character on (provided the first is a blank), or it follows a label plus at least one blank, and extends indefinitely. This section is the core of a Snobol engine. In this section, all commands, assignments and pattern evaluations are processed. It is regarded as the procedural section, where things are effectively accomplished.

This rule section is so important that a whole chapter is devoted to it. See ahead.

2.2.2.3. The go-to section

The *go-to* section is the third and last section of a Snobol4 program line. As the rule section, it starts from the second character on, (provided the first is a blank), or it follows a label plus at least one blank, or it follows a rule section, with or without a blank following, and extends indefinitely.

This section serves as the decisional process for jumps. Normally, being the last evaluated section, it should not be followed by anything else (apart from semicolons introducing next lines).

It is introduced by the colon `:`, followed by one or more of the following token:

```
S(Label)
F(Label)
(Label)
```

The case *S(Label)* is performed only if the evaluation was a success. The case *F(Label)* is performed only if the evaluation was a failure. The case *(Label)* is performed in any case (unconditional jump).

It's not mandatory that all labels are used throughout the program; you are free to *name* one particular line with a

proper label, and this may prove useful to identify some program section, even if you'll never use a go-to pointing to that label.

In facts, any label in the go-to Section must exist, but not the contrary: the Snobol culture, since its beginnings, has introduced the possibility to give names to specific sections of the listing, using the label space; not all these labels are destination of a jump, but the listing is more clear.

Label is any legal label that appears in the program text, otherwise an execution-time error is raised; it can be evaluated *on the fly* by using the `$` operator. In case *Label* is a composition of strings, this composition must be put into one more couple of parenthesis, as in the following example:

```
:F((NAME SIZE(NAME)))
```

with the supposition that `NAME` is not an empty String, and that the label identified by `NAME SIZE(NAME)` does exist[8]. The form:

```
:F(NAME SIZE(NAME))
```

is invalid because soft would recognize the sole `:F(NAME)`, but with a syntax error (the closing parentheses was not found), and with the risk that the label `NAME` may not exist.

Multiple go-tos may be specified, but only if mutually exclusive, like:

```
: S(LAB1) F(LAB2)
```

because no ambiguity exists: either the clause succeeds, or it fails, *tertium non datur*. Instead, the case:

```
: S(LAB1) (LAB2)
```

is forbidden because, in case of success, the system cannot decide between the two destinations, which are both valid.

There is another go-to jump to be considered: the jump to Code lines (lines introduced by the *CODE()* procedure). This jump is performed with:

```
:[VarName]
```

where `VarName` is any variable created with the *CODE()* command. In this case, the jump will happen to the start of the coded section. (See the chapter devoted to the *CODE()* command.)

2.2.2.4. Grouping program lines

As introduced earlier, on a single line of text, code sections can be grouped using the semicolon to separate them, as in the following example:

```
X = 2; Y = 3  ;LAB3  Z = 10
```

You may have already noticed that the semicolon is a placeholder for the *beginning of line*; this means that if you want to use a label, like `LAB3` in the example, **no space is added after the semicolon**, in order to let the label be recognized as such by starting from the first character of the line, and conversely **there must be always at least one blank after the semicolon** in order to recognize rules and go-tos that don't start from the first character (see the previous paragraphs).

Anyway, the grouped lines, during the loading phase, are separated and stored in consecutive lines, all counted, and this alters the program code line numbering (which has the original grouped lines in one single line); so, if this feature is necessary, for instance to setup all variables in a few lines all together, use the listing options, that will show the reconstructed numbered lines.

---

[8] This is a feature of soft, that is not available in Spitbol or in the Bell version.

2.2.2.5. Summing up...

As a further exemplification, the third line of the previous program is here repeated and commented:

```
READ    WORD   =   TRIM(INPUT)            :   F(END)
```

The label section identifies `READ` as a label (elsewhere in the program, a jump to `READ` is performed).
The rule section is constituted by the assignment `WORD = TRIM(INPUT)`.
The go-to section is built for jumping to `END` in case the assignment cannot be done, returning a failure (this happens when `INPUT` cannot read any more characters from the input program text).

2.2.3. The rule section in depth

As said, the rule section is the central and main part of a Snobol engine.

It is composed by two kinds of possible operations:

- Assignments
- Pattern evaluations

Let's see them in detail.

2.2.3.1. Assignments

Assignments are the first primary operations in Snobol. They are necessary to set values and to solve input and output.

An assignment is performed with the following syntax:

```
<var> = <value>
```

`<var>` is a name of an identifier. It must start with a letter, and may contain letters, digits or dots[9]. If this identifier was not previously created, it's created anew.

The symbol = is the assignment operator. In Snobol, the equal sign must be surrounded by blanks (though this is not strictly necessary neither in the Berkeley version nor in soft).

`<value>` is any combination of numbers, strings, pattern procedures, or anything belonging to the language, with the following general notions:

- If `<value>` is a math expression returning an integer number, this number is stored and the type is set to Integer;
- If `<value>` is a math expression returning a real number, this number is stored and the type is set to Real;
- If `<value>` is a string or a collection of strings, the concatenated series of strings is stored and the type is set to String.
- If `<value>` is a pattern or a collection of patterns or strings, the concatenated series of items is stored and the type is set to Pattern.
- If `<value>` is in form of a Name object, the variable is instantiated with the name of the object, and the type is set to Name.
- If the assignment refers to the procedures *ARRAY()*, *DATA()* or *CODE()*, objects of type Array, Data and Code are respectively created, respecting the proper prototype.
- If the assign procedure cannot recognize one expression item in `<value>`, it is stored as-is and the final type is set to Pattern, with all math expressions calculated and string concatenations executed, leaving the check of the assignment to a further pattern evaluation.

[9] Variable names can be extended to have *any* character combination thorough the `$` operator, that is the operator that uses the content of a string variable as the name of the variable; this process is called *indirect referencing* and it's a funding and very interesting feature of the Snobol environment. See the relative chapter.

For instance, see the following example:

```
        VAL = 45
        VAL1 = VAL + 1
        VAL2 = 1 + VAL
        VAL3 = VAL + "HELP"
        VAL4 = "HELP" + VAL
        OUTPUT = "VAL1 = " VAL1
        OUTPUT = "VAL2 = " VAL2
        OUTPUT = "VAL3 = " VAL3
        OUTPUT = "VAL4 = " VAL4
END
```

The addition for `VAL3` in line 4 and for `VAL4` in line 5 are not proper: the operator + cannot be used outside mathematical expressions. The current Snobol and Spitbol interpreters end with an error message, here. The behaviour of soft is different; the output is:

```
VAL1 = 46
VAL2 = 46
[PATTERN]
[PATTERN]
```

because any combination of characters not reducible to numbers or strings, if nothing else applies, is considered a Pattern, and left to a pattern evaluation find the error.

The type of the variable to the left of the equal sign is reset at every re-assignment, and this means there are no typed variables in Snobol; rather, a variable is simply a name that can, at any time, identify an integer, a real number, a string, a pattern, an array, a data item, a code element or a name, and it changes its type accordingly to Integer, Real, String, Pattern, Array, Data, Code or Name.

If an uninstantiated variable is used in any string combination or math expression, it's implicitly declared as an empty string (acting as zero in math expressions).

The variable named *OUTPUT* is instantiated like any other variable, but it also has a side effect: printing the content of the assignment to the output channel, which is the screen by default, but it can be changed to a write-file.

The variable named *SCREEN* is instantiated like any other variable, but it also has a side effect: printing the content of the assignment always to screen.

The variable named *INPUT* is instantiated like any other variable with the text content of the next line from the current input channel, which is the keyboard, but it can be changed to a read-file.

The variable named *TERMINAL* is instantiated like any other variable with the content of the input always from the keyboard.

Some Snobol4 commands can operate in a very special way, acting as a sort of *direct* commands; e.g., to open a file, the `INPUT()` procedure - with parameters - can be used; the basic and formal way is in the form of an assignment:

```
        VAR = INPUT(...)
```

Since this procedure, as all direct commands, returns the void string, the assignment sets `VAR` to the null string. To avoid the proliferations of null variables, these commands can be used alone:

```
        INPUT(...)
```

This command fails if the reading from the source fails for some reasons (for instance, the file is not found). Of course, since *INPUT()* is at all effects a string procedure, it can be combined in a string collection:

```
        RES = "THIS " INPUT(...) "THAT"
```

This apparently useless piece of code, assigns to `RES` the string `"THIS THAT"`, and in doing so, it sets also an input channel. If this operation should fail, the whole assignment wouldn't take place, and this could be an intended effect. That's why this is not such an absurd programming line: Snobol programmers think laterally!

2.2.3.2. Pattern evaluations

Pattern evaluations are the other vital primary operations in a Snobol environment. A pattern evaluation is composed by three parts (the second and third not mandatory):

base pattern [= *subst*]

The two elements *base* and *pattern* are separated only by blanks. The *base* is a string in any form (literal or variable), and it is also the first item of the line; if you have to use a *base* composed by more items, they must be surrounded by parenthesis, as in the example:

(base1 base2 base3) pattern

The *pattern* may be composed by multiple pattern instances, called *patches*, also separated by blanks.

The natural operation performed by an evaluation is checking if all patches in *pattern* occur in the same order in *base*. This is done through the following process:

1 Initially, the cursor position is zero, that is it is positioned before the first character of *base*.

2 The next patch in *pattern* is determined and checked against the current cursor position in *base*.

3 If the match occurs and there are no more patches, the evaluation terminates with success, advancing the cursor position to match the next character after the pattern in *base* and concatenating the result to the global pattern string. If instead there are other patches, proceed again from 2.

4 If the match does not occur, the cursor is incremented by one, the patches pointer is reset to point to the first patch and the evaluation proceeds again from 2.

5 If the match does not occur, and the cursor cannot be incremented (because beyond the *base* length or the state is set to failure), the match terminates with a failure.

6 In case of success, if the equal sign is found after the *pattern* strings, the global match string is substituted with the string *subst*; this can be accomplished only if *base* is not a literal string or a string collection, that is it must be a natural variable or an array item.

If the whole *pattern* is null, the match always succeeds, as for:

base

which is equivalent to

base `NULL`

In this case, the evaluation always succeeds[10], and I call this kind of match *implicit success*. Of course, if *base* is null and *pattern* is also null

`NULL NULL`

the match succeeds as well (null is null, after all), and I call this *implicit null success*.

The various patterns procedures let you define in great detail the patterns generation through patches sequences (using literal String items, numeric expressions or pattern procedures like *LEN()* or *SPAN()*).

2.2.3.3. More on pattern evaluation

As said before, the pattern matching process can *match* or *unmatch* the current string patch against the base string; this is not related in any way to a *success* or a *failure* of the evaluation: it's simply a *match* or an *unmatch* state. A *match* state causes the scanner to try the next patch, by finding another *match* or *unmatch* state, and appending the

[10] Every string is terminated with the null character, and thus the null character is actually a part of the string, which is regularly found.

resulting string to the global result; it then proceeds until there are no more patches. An *unmatch* state, instead, causes the scanner to advance the cursor one character ahead, pointing to the next character in*base*, and retry, reanalyzing the whole pattern from the first patch; this is called *rematch* (or backtracking). The *success* of an evaluation is the final state of consecutive *matches* for all patches. The *failure* is the impossibility to proceed with a rematch. So, basing on these definitions, an *unmatch* is not a *failure*, and a *match* alone cannot be a *success*.

An example is a thousand words:

```
VAR1 = "N"
"ANNA"   "AN" VAR1 LEN(1)
```

This is exemplified in the following line, that shows the starting phase:

```
ANNA
^
```

The patch `"AN"` is initially evaluated, and since it is followed by a String variable, this is concatenated; the result value `"ANN"` is tried with a *match*, and the cursor is advanced to point to the final `"A"` in `"ANNA"`:

```
ANNA
   ^
```

Finally, the last patch `LEN(1)` is tried with one more *match* (the letter `"A"` is one char), and the cursor is set to point to the end of `"ANNA"`:

```
ANNA
    ^
```

resulting in a *success*. The final pattern is `"ANN"` + `"A"` and thus it forms the string `"ANNA"`; the cursor ends with the value 4.

```
A N N A
0 1 2 3 4
```

The *failure* of an evaluation means there are no more possibilities, even if not all clauses were matched against the base string, finding no matches, e.g.:

```
VAR2 = "S"
"JOHN"   "JO" VAR2 LEN(1)
```

Again, this is exemplified with:

```
JOHN
^
```

The patch `"JO"` is tried, and since it is followed by a string variable, this is joined to form the string `"JOS"`, which is tried with a *unmatch*. The process starts a *rematch* re-evaluating the first patch with cursor = 1:

```
JOHN
 ^
```

but `"JOS"` doesn't match with `"OHN"`; then another *rematch* is tried with cursor = 2:

```
JOHN
  ^
```

but again `"JOS"` doesn't match with `"HN"`; the process continues until the cursor is 4^{11} and stops here because further *rematches* are not possible anymore; the final state is a *failure*. Note that the patch `LEN(1)` was not tested even

once.

Sometimes the *match* occurs after some *rematches*:

```
STR = 'U'      VAR3 = "VACUUM CLEANER"
VAR3 "CU" STR
```

In this case, the patch `"CUU"` (formed by concatenating `"CU"` with the String variable `STR`) is tried with cursor pointing to `"V"`, with an *unmatch*; a *rematch* is tried, so the cursor is advanced by one, with cursor pointing to `"A"`; again, an *unmatch* occurs, and another *rematch* is tried, so the cursor is advanced again by one. At this point a *match* occurs, and since there are no more patches, the pattern results in a *success*. (This is possible only in case of Anchor mode disabled. See *ANCHOR()*.)

So, the final states that are of interest in the pattern evaluation comprehension are: *match*, *unmatch*, *rematch*, *success* and *failure*.

There are some pattern variables that, when invoked into a pattern, influence the previous states:

- *ABORT* sets the *failure* state even if some matches were found; in case it's used, the failure is guaranteed to happen, and the Snobol engine steps to the next program line or the next alternation item.
- *FAIL* sets the *unmatch* state even if some matches were found; the Snobol engine thus tries other alternatives starting a *rematch*. The same *unmatch* state is set by *POS()* and *RPOS()* if the cursor does not obey to the position specified by their arguments.
  If tried repeatedly as the last patch, the final result of a *FAIL* pattern is of course a *failure*, but this may be simple the effect of the last operation, with all the intermediated results already successfully printed; so this is an *apparent failure* state.
- *FENCE* sets the *failure* state, much like *ABORT*, if a *rematch* is tried that goes leftward past its position. In all other cases, it's a transparent word.

2.3. Peculiarities of the language

Apart from the usual algebraic elements, and the predefined procedures (whose design is specific, like *EQ()* or *IDENT()* not surprisingly), the Snobol4 language has some peculiarities that are rarely found in other programming languages, and in the next chapters I will expose these characteristics, warning that the short essays in this manual cannot replace reading the GGM or the GB, which are unbelievably useful in any case.

2.3.1. Indirect assignments

The indirect assignment is one of the most powerful techniques of assignment, because it acts *in the middle of a pattern evaluation*, that is it can be used to store intermediate results without interfering with the pattern evaluation itself. It is called *indirect* because its form does not use the = (equal) sign.

There are two types of indirect assignments: the *immediate assignment* and the *conditional assignment*.

Immediate assignment

The immediate assignment is performed by the `$` (dollar) symbol, followed by at least one space and by a variable name. Whenever a match occurs, if after the last patch (or the last patch enclosed in parentheses) a `$` is found followed by at least one blank, the variable name that follows is instantiated with the pattern match string. In case the match does not occur, either a rematch or a failure follows, and the instantiation is not performed.

This kind of indirect assignment is useful in case you want to examine all the intermediate results, independently of the final state. For instance:

```
"CAVEAT!" LEN(3) $ OUTPUT FAIL
```

would print all the triples in the base string, with the result:

[11] The system works in full-scan mode, so it is not aware that no match is possible because of the length of the remaining base part. See the relative chapter.

```
CAV
AVE
VEA
EAT
AT!

No.
```

with a final failure (due to *FAIL*).

Conditional assignment

The conditional assignment is performed by the `.` (dot) symbol, followed by at least one space and by a variable name. Whenever a match occurs, if after the last patch (or the last patch enclosed in parentheses) a dot is found followed by at least one blank, the pattern matching string is temporarily stored and in case the evaluation should end with a success, the variable is instantiated with the temporarily saved value. If the match does not occur, either a rematch or a failure follows, and the instantiation is not performed at all and the temporary value is lost.

This kind of indirect assignment is useful in case you want to obtain only the final result of a pattern evaluation, without boring about the intermediate results and provided that the final state is a success. For instance, the clause:

```
        "CAVEAT!" ("V" LEN(2)) . OUTPUT
```

would print the match string `"VEA"`:

```
VEA

Yes.
```

with a final success.

Whenever there are chances that the clause would fail, the conditional assignment must be substituted with the immediate assignment that, in case of success, is perfectly equivalent (and, in case of failure, it keeps working):

```
        "CAVEAT!" ("V" LEN(2)) $ OUTPUT   DIFFER(VAR,NULL)
```

The previous clause, while failing (because `VAR`, at this point, *is* `NULL`) would nonetheless print the intermediate result:

```
VEA

No.
```

The examples in this section were performed in PIE, but in any program they would behave as expected, except for the final Yes/No output.

2.3.2. Loops

In the most common programming languages, loops are performed with special structures (more or less the same for all languages), structured as follows (the design is the one of C, but any language has its own design):

Definite loops

```
for (fixedcond) {
        forcommand
}
```

where *fixedcond* depends on the language (for instance it's `x=0; x<11; x++` in C or `x = 0 to 10` in BASIC, and *forcommand* is the complex of instructions to be performed.

Indefinite loops

```
initialize_logical_value
while (logicalcond) {
       whilecommand
}
```

where the programmer must initialize a logical value (the same that is evaluated in *logicalcond*, and *whilecommand* is the complex of instructions to to be performed.

These commands are very definite, and may be used practically for everything (the sources of soft are full of such structures.

In Snobol4, instead, there is not a command to build loops. If the programmer wishes to repeat instructions, a proper label/go-to addressing; let's consider an example:

C programming: Counting from 1 to 10

```
int x=1;
while (x<=10) {
       printf("%d0,x);      x++;
}
```

Snobol4 programming: Counting from 1 to 10

```
        x = 1
LOOP    OUTPUT = x
        x = x + 1
        LE(x,10)              :S(LOOP)
```

The two structures are quite similar, but the Snobol4 version is one level down (the programmer must build the `while` cycle). But if you look closer, you can see that they perform the very same task, that is the loop.

You get the idea, now.

2.3.3. The deferred evaluation operator

Normally, an assignment such as:

```
        A = "STR"
        PAT = A LEN(3)
```

would store in `PAT` the pattern `"STR" LEN(3)`, and the following lines:

```
        A = "ION"
        "ABSTRACTION" PAT . OUTPUT
```

would print the result

```
STRACT
```

because the variation in `A` is not automatically reversed in `PAT`, which retains the value that it had previously received. But what if different values for `A` were required?

In these cases, the *deferred evaluation operator* comes to help:

```
        PAT = *A LEN(3)
```

In the previous code line, `PAT` is not instantiated with the value of `A` but with `A` itself. Whenever `PAT` is executed,

the *current* value of A is retrieved, stored in PAT and used; for instance:

```
A = "STR"
PAT = *A LEN(3)
"ABSTRACTION" PAT . OUTPUT
```

would print

```
STRACT
```

as before (the target is the same) but, without changing PAT, we can ask different patterns:

```
A = "ABS"
"ABSTRACTION" PAT . OUTPUT
```

would print

```
ABSTRA
```

which is a totally different result than before. This is quite powerful, isn't it?

I must warn you that the deferred evaluation in the Berkeley version, as stated on page 174 of the GGM, is not extended as in the Bell version; Let's listen to Gaskins and Gould: "*The unary operator * is called in the Bell version the unevaluated expression operator, and expressions introduced by it are of datatype Expression. This operator is defined more narrowly in the Berkeley version. It is called the deferred evaluation operator, and may be applied to simple variables only; thus *EQ(X,Y) causes an execution-time error. The datatype Expression is not defined in the Berkeley version: expressions introduced by the deferred evaluation operator are of datatype Pattern. Hence LEN(*V) causes an execution-time error since the argument of LEN() cannot be a Pattern.*".

In soft things are a bit different. While *LEN()* can manage the phrase LEN(*V) because the expressions evaluator can calculate the deferred value V by retrieving its value even if it's a Pattern, the deferred evaluation cannot be used with variables of type Array, Data, Name and Code, nor with expressions like *EQ(X,Y); this was in the Berkeley version too.

In these cases, the program breaks and the following error appears:

```
ON PROGRAM eq.sno:
? ERROR 9 IN STATEMENT 1 AT LEVEL 0:
THE VALUE OF A VARIABLE IN A DEFERRED-EVALUATION PATTERN (UNARY *)
MUST BE A PATTERN OR STRING.

PAT = *EQ(X,Y) "ALT"
        ^
```

The error message underlines the fact that the deferred evaluation operator can be only used on String types (String, Integer or Real) and with Pattern types; in all other cases, the clause breaks when the pattern is evaluated.

The workaround for this apparent inability is using the parenthesis to enclose the items to post-evaluate (even expressions or whatever); whereas *GT(...) is not evaluable, the following is:

```
PAT = *( GT(...) )
```

The variable PAT is instantiated with the Pattern '(*(GT(...)))'; whatever is the content of PAT, it is re-evaluated when used, and this solves the apparent limits in the deferred evaluation operator. The following PIE session shows how:

```
$ PAT = *(GT(X,Y))

Yes.
```

```
$ X = 24

Yes.

$ Y = 10

Yes.

$ "CALLING RINGO" PAT "LL" . OUTPUT
LL

Yes.

$ Y = 100

Yes.

$ "CALLING RINGO" PAT "LL" . OUTPUT

No.
```

The variable `Y`, initially 10, makes the pattern evaluation succeed (the clause in `PAT` succeeds), and this is attested by the `LL` string that is printed.

When later `Y` is set to 100, the clause in `PAT` now fails, the pattern evaluation is not completed and no instantiation is performed.

This trick, that I think was also in the Berkeley version, overrides the impossibility to address the deferred evaluation operator with Expression types, procedures and created variables, available in the Bell version.

2.3.4. The cursor position operator

The cursor position operator `@` (the *at* sign) has a variable as its operand, immediately attached (no blanks between `@` and the first character of the variable's name), and is used within the pattern part of a rule. The variable is assigned, by immediate assignment, an Integer value representing the current position of the cursor. Thus the clause

```
        'ABC'  @OUTPUT 'B'
```

causes `OUTPUT` to be successively instantiated with values zero and one, representing the cursor advancing at the start of the matching process (zero) and after the first rematch, before the match occurs (one).

```
0
1

Yes.
```

The Berkeley version missed this operator. In describing the Bell features, the GGM reports on page 179 the same previous example, writing the line:

```
        'ABC'  'B' @OUTPUT
```

but this piece of code, failing the first match `"A" = "B"`, advances the cursor, matches `"B" = "B"` and advances the cursor again, so that the final result is

```
2

Yes.
```

and not the one indicated by Gaskins and Gould (this is confirmed by the Spitbol behaviour)[12].

2.3.5. Referenced variables

One of the most powerful features of Snobol4 is the variables referencing. Let's examine a simple example:

```
PLANT = "TREE"
TREE = "OAK"
OAK = "CHINKAPIN"
```

You see that the first two variables contain a string which is the name of another variable; the rule of referencing uses the *indirect referencing operator*, the dollar sign `$`, which operates a referencing, evaluating not the variable content, but the variable whose name is its variable content. The dollar sign must be attached to the variable name that follows, to distinguish it from a conditional assignment. For instance:

```
OUTPUT = PLANT
```

returns the string `"TREE"` (of course); but

```
OUTPUT = $PLANT
```

returns the string `"OAK"`, because the figure `$PLANT` is equivalent to `TREE`, so that the two calls:

```
OUTPUT = $PLANT
OUTPUT = TREE
```

return the same value `"OAK"`.

The dollar operator `$` can be reiterated, so that:

```
OUTPUT = $$PLANT
```

returns the string `"CHINKAPIN"`, because the following rule holds:

`$$PLANT = $($PLANT) = $TREE`

The referencing operator can be used also in assignment; the assignment:

```
$TREE = "OTHER TREE"
```

does not change `TREE` but the variable whose name is in `TREE`, which is `OAK`; the previous line is equivalent to:

```
OAK = "OTHER TREE"
```

This technique is widely used in Snobol, and a Snobol programmer must master it.

2.3.5.1. Advanced uses of referenced variables

Let's image that in the variable `VOW`, the count of all vowels of a line must be stored. The common way is, for each vowel met, to perform:

```
VOW = VOW + 1
```

Now, suppose that `VOW` is used as the vehicle that holds the vowel name, and the count must be done for each vowel; at first, the count of vowel A must be calculated. One typical usage is:

```
VOW = 'A'      $VOW = $VOW + 1
```

This piece of code declares a variable `VOW` holding `'A'` and another variable `A` that initially is the empty String, but the second line calculates a numerical value (`0 + 1`) and stores the value 1 in `A`. Now we want to count for, say, the vowel E:

[12] Of course, the mistaken text in the GGM must be a simple typo, present on the original draft, not removed from the final version.

```
        VOW = 'E'       $VOW = $VOW + 1
```

You have certainly noted that the second line remains unchanged. But now VOW holds the letter 'E'. The power of this technique can be seen in action in the following example:

(You can find the text of this program in the samples/ *directory of the installation package in the file* count.sno*; to execute it, step into this directory from the console and type*

```
$ soft --input=from count.sno
```

The from *textual file contains the input values and is contained in the same directory. The output will be directed to the screen.)*

```
* PROGRAM TO MAKE A CHARACTER COUNT
* SET UP CHARACTER-FINDING PATTERN
        CHAR.PAT = LEN(1) . CHAR
*
* READ IN THE DATA
READ    LINE = TRIM(INPUT)                                      : F(OUT)
*
* FIND THE NEXT CHARACTER - ASSIGN IT TO THE VARIABLE CHAR
LOOP1   LINE CHAR.PAT = NULL                                    : F(READ)
*
* ADD ONE TO THE COUNT FOR THAT
INC     $CHAR = $CHAR + 1                                       : (LOOP1)
*
* SPECIFY THE ALPHABET FOR RECOVERING COUNTS
OUT     ALPHA = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
*
* GET THE NEXT LETTER WHOSE COUNT IS TO BE RECOVERED
* ASSIGN IT TO THE VARIABLE CHAR
LOOP2   ALPHA CHAR.PAT = NULL                                   : F(END)
*
* IF LETTER DID NOT OCCUR, GIVE IT THE VALUE ZERO, NOT NULL
        $CHAR = IDENT($CHAR,NULL) 0
*
* PRINT LETTER AND ITS COUNT
        OUTPUT = CHAR '    ' $CHAR                              : (LOOP2)
END
```

If the file of data to be used as input for the program text above were the following list of words:

```
HIPPOPOTAMUS
HIPPOS
HIPPOSIDEROS
HIPPOSPONGIA
HIPPOTIGRINE
HIPPOTOMY
HIPPOTRAGINE
HIPPOTRAGUS
```

then the output produced by the program would be:

```
A    4
B    0
C    0
D    1
E    3
F    0
```

```
G     4
H     8
I     13
J     0
K     0
L     0
M     2
N     3
O     12
P     18
Q     0
R     4
S     6
T     5
U     2
V     0
W     0
X     0
Y     1
Z     0
```

Similarly, a reference to any procedure which returns a String (String, Integer or Real) as its value may be used. For instance the rule

```
$SIZE(WORD) = $SIZE(WORD) + 1
```

creates a variable that has as name an Integer, and words may be thus counted by their length.

It must be noted that the indirect referencing operator overrides the naming rule of Snobol4, which establishes that variables names must begin with a letter followed by any letters, digits or dots. The following is accepted:

```
$',' = $',' + 24
```

This statement creates a variable with the comma as name. Strange as it seems, but the following is even more strange[13]:

```
$'\t' = $'\t' + 24
```

Anyway, my advice is to write any non-standard referenced name enclosed in quotes, to avoid any possible error of interpretation; **this rule is obligatory if the non-standard name contains any among `( [ ] )` - i.e. the characters for round and square brackets: in this case, the reference **must** be written enclosed in quotes, or the name will unbalance the brackets count, and the program would break with an error message.**

2.3.6. The concatenation operator

The GGM reports on page 17 the following text:" The concatenation operator. *One of the most frequently used operators is the concatenation operator. When the operands of this binary operator are strings, it specifies that the two strings are to be concatenated together, i.e., that the second string is to be appended directly to the first. The symbol for this binary operator, since it occurs so often, is simply a single blank (Which requires, therefore, no further blanks to separate it from its operands)*".

This applies to assignments or pattern evaluations, and implies that two or more adjacent strings are glued together to form a unique string. This is straightforward and immediately comprehensible in the following examples:

```
OUTPUT = "FIRST" "SECOND"
```

In the previous example, `OUTPUT` is instantiated with the string `"FIRSTSECOND"`; note that the two strings are concatenated without any intermixed character. If the concatenation is a text phrase, the internal blanks must be

[13] I had never seen a programming language capable of using the tab character as the name for a variable...

given *inside* the strings, not outside:

```
        OUTPUT = "FIRST " "SECOND"
```

that returns `"FIRST SECOND"`. In case of two numeric strings, the result depends on the final result:

```
        OUTPUT = 3 2      OUTPUT = DATATYPE(OUTPUT)
```

The previous line what will output? Integers are strings, after all, and so the parser evaluates `3` (an Integer not followed by an operator, done) and then `2` (Integer that is concatenated to the previous element); result: `32`, which is a string, but it evaluates to an Integer, because it is formed by digits only. The result is:

```
32
INTEGER
```

And what will happen with Real numbers:

```
        OUTPUT = 3.2  2.4
        OUTPUT = DATATYPE(OUTPUT)
```

The first item is a Real, with decimal part not followed by an operator, and so it evaluates to `3.2`; the second part is another real, with decimal part; it evaluates to another Real that is concatenated to the first part, giving birth to the sequence `3.22.4`, which cannot be converted to a single number. So it remains a String item; the result is:

```
3.22.4
STRING
```

The concatenation acts also in pattern evaluations like this:

```
        STR = "C"
        "VACUUM CLEANER" ( STR "UU" LEN(1)) . OUTPUT
```

where the evaluation is initially tried not with `STR` (which contains `"C"`) but with the concatenation `"CUU"`, that forms the first part of the pattern evaluation (concatenation has a higher priority); after the match, the second patch `LEN(1)` is tried, with a further match; `OUTPUT` is instantiated with the result string `"CUUM"`, obtained by joining `"CUU"` + `"M"` from the relative patches.

Concatenations also prevails also over alternations; see for instance the following program:

```
        LINE = "I WANT THIS SOURCE AND A MAGNET"
        WORD1 = "THE"
        WORD2 = "THIS"
LOOP    LINE ( ' ' 'A' ' ' | ' ' WORD1 ' ' | ' ' WORD2 ' ')
+ $ OUTPUT = ' ' :S(LOOP)
        OUTPUT = "LINE=" LINE
END
```

This program uses concatenated strings inside the alternations limits, trying to match the result-strings `' A '`, `' THE '` or `' THIS '` to be searched in `LINE`. The final result, after the completion of the loop, is:

```
 THIS
 A
LINE=I WANT SOURCE AND MAGNET
```

The two strings in the first two lines are preceded and followed by blank spaces (which are not visible).

Perhaps, considering the blank as an operator is strange, but still this is a Snobol4 feature that must be taken into consideration.

2.3.7. The alternation operator

Suppose the following pattern evaluation is required:

```
BASE = "SAND CASTLE IN THE BEACH"
....
BASE 'THIS'                :F(LOOP)
BASE 'THE'                 :F(LOOP)
```

The search is done twice on the same base string because both `'THIS'` and `'THE'` are needed. But this leads to two conditions, two pattern evaluations and two go-tos; this is not a clear task, and what's worse, what if more than two clauses were needed?

The solution is to use the alternation operator | (the vertical bar), which means *or* as in a common logic language. The result is:

```
BASE  'THIS' | 'THE'
```

The sequence *This or The* will succeed if the value of the base string contains any of the two strings. The search for a match proceeds as follows: the first string `'THIS'` is checked against the base string; if the match should not occur, the evaluation engine tries a second alternative, from the same cursor position. As soon as anyone of the alternatives is found, the final result is a success, that can be directed to a specific label with a go-to. The pattern matching fails only when all strings in the alternation have been examined and no alternatives of the pattern have been found.

The alternation may be replicated in the pattern line, and this increases the combinations in the research; for instance[14]:

```
"READERS" ('A' | 'D' ) 'E' ('A' | 'B' | 'R')
```

The previous line has two alternations; the items are tried in turn, following this order:

1 First try with `'A' + 'E'` with an unmatch
2 Then try with `'D' + 'E' + 'A'` with an unmatch
3 Then try with `'D' + 'E' + 'B'` with an unmatch
4 Then try with `'D' + 'E' + 'R'` with a match

This is the complete search as shown in PIE:

line	scan	match	status	altern.	alt scan	next scan	pattern
L1	0	A	NO	'D') '	0	0	/
L1	0	D	NO	/	0	1	/
L1	1	A	NO	'D') '	1	1	/
L1	1	D	NO	/	0	2	/
L1	2	A	V	/	0	3	A
L1	3	E	NO	'D') '	2	2	/
L1	2	D	NO	/	0	3	/
L1	3	A	NO	'D') '	3	3	/
L1	3	D	V	/	0	4	D
L1	4	E	V	/	0	5	DE
L1	5	A	NO	'B' |	5	5	DE
L1	5	B	NO	'R')	5	5	DE
L1	5	R	V	/	0	6	DER

The green clauses show the match phases; as you can see, `'A'` is found first, but it's a dead branch; then `'D'` is tried, and `'E'` is appended with success; then `'B'` is tried with an unmatch, and finally `'R'` is tried with a success.

[14] Whenever single letters are involved, the procedure *ANY()* should be used. The purpose of this example is to show how the alternation operator works.

(You can find a more complex program, reads.sno *in the* samples/ *directory of the installation package, with four alternations; to execute it, step into this directory from the console and type*

```
$ soft -p reads.sno
```

and you will see the power of the pattern debug, and also all the alternatives tried to find the final pattern.)

Cautions must be used in using multiple alternations mixed with strings; for example, consider the following excerpt:

```
        WORD = "LA BELLA MONTEBELLUNESE LELLA BALLA SULLA BELTA'"
DO3     WORD 'B' | 'L' ( 'E' 'L' )  'L' | 'S' | 'T' ( 'A' | 'U' )
+ = "BRILLA"   :S(DO3)
```

The intended meaning is to catch the following combinations:

```
BELLA
BELLU
BELSA
BELSU
BELTA
BELTU
LELLA
LELLU
LELSA
LELSU
LELTA
LELTU
```

but the effect varies from interpreter to interpreter, because the pattern is seen as:

```
        WORD = "LA BELLA MONTEBELLUNESE LELLA BALLA SULLA BELTA'"
DO3     WORD 'B' | ['L' ( 'E' 'L' )  'L'] | 'S' | 'T' ( 'A' | 'U' )
+ = "BRILLA"   :S(DO3)
```

that is `B` *or* `LELL` *or* `S` *or* `T` *and* one between `A` *or* `U`. This can lead to an infinite cycle or to a failure, depending on the search algorithm (soft ends in a failure).

My advice is **to enclose all alternations in parentheses**, to make clear the order of precedence for all interpreters you may consult:

```
        WORD = "LA BELLA MONTEBELLUNESE LELLA BALLA SULLA BELTA'"
DO3     WORD ('B' | 'L')  'E' 'L'  ('L' | 'S' | 'T') ('A' | 'U')
+ = "BRILLA"    :S(DO3)
```

This solution is unambiguous and returns correctly:

```
LA BRILLA MONTEBRILLANESE BRILLA BALLA SULLA BRILLA'
```

If you ask me what these words mean: I have yet to find out.

2.3.7.1. Alternations and ANY()

The procedure *ANY()* works in a similar way for what about alternations, with the difference that alternations can use multi-characters strings while *ANY* works with the single characters in the argument string. Obviously, in case alternations are used with single characters, the two works in the same way.

So one may ask: what is the best method? Let's examine the same example as before:

```
"READERS" ('A' | 'D' ) 'E' ('A' | 'B' | 'R')
```

It can be rewritten as follows, using *ANY()*:

```
"READERS" ANY('AD') 'E' ANY('ABR')
```

In this case, what is the evaluation analysis pattern track? Here is it is.

(You can find this program in the file reads2.sno *in the* samples/ *directory of the installation package; to execute it, step into this directory from the console and type*

```
$ soft -p reads2.sno
```

and you will see the two evaluations in series.)

```
line    scan    match           status  altern. alt scan        next scan       pattern
-------------------------------------------------------------------------------------
L6      0       /               NO      /       0               1               /
L6      1       /               NO      /       0               2               /
L6      2       A               V       /       0               3               A
L6      3       E               NO      /       0               3               /
L6      3       D               V       /       0               4               D
L6      4       E               V       /       0               5               DE
L6      5       R               V       /       0               6               DER
```

As you can see, comparing this result to the previous, you can see that:

1 The result is the same

2 The result is achieved in 7 steps, with 3 failures, while the previous result was achieved in 13 steps with 9 failures.

This demonstrates that whenever single characters are involved, the usage of *ANY()* is preferable, because of its compactness.

3. THE BERKELEY VERSION IN DEPTH

The Berkeley Snobol4 compiler is a subset of the most complete and most general so-called Bell version, which is the *de facto* Standard in Snobol4, and the one to which all the compiler and interpreters available today adhere.

Warning: This chapters assumes you have some knowledge about the Snobol4 language of the Bell version. If you should not understand some of the terms used, please refer to the GB.

3.1. The Berkeley axiom

Letting apart for the moment the practical differences (the commands and features present in one version and not in the other, that will be treated more deeply in the next sections), there is a principal and main difference between the two philosophies: in the Berkeley Snobol4 **everything is a string procedure**. So every command returns a string, in much cases the empty string. And this means that even the setting of some flag is performed by a procedure; for instance, whereas in the Bell version the anchor mode is set as an assignment to change the content of a system variable

```
&ANCHOR = 1
```

in the Berkeley version, the same task is accomplished by a string procedure:

```
ANCHOR(1)
```

which is a procedure with one argument returning the empty string, so that the following is also possible:

```
VAR = ANCHOR(1)
```

with the side effect of setting `VAR` to the empty string. This approach has a consequence: the Berkeley version lacks many controls and features that were common in the Bell version by means of the so-called *keywords*; for instance *&DUMP*, *&TRIM*, *&TRACE*, *&CASE*, *&FULLSCAN*; in much cases the Berkeley version developed a proper substitute (e.g. the procedure *ANCHOR()* as seen earlier), but in other cases the Bell keywords (which are many) remain without any counterpart.

Where the two philosophies match is in considering the so-called *Snobol first rule*, which states that **everything is a string**. This means that strings like `"ANNA"` and `24` are two strings with different types, the former with the String type, the latter with the Integer type (and the property that it can belong to math expressions), but they both behave as (and basically are) strings.

You will find this rule very strange, at first, but you will soon realize how handy it is.

3.2. Comparing the Berkeley and Bell versions

The Berkeley and Bell versions are very similar in most cases. But sometimes the differences between the two worlds is subtle and not really easy to spot at a first glance. Even for what is in common compatibility is not guaranteed.

So, just to chat, a review of the differences between the two worlds is summarized here.

3.2.1. A case study of non-interoperability: INPUT() and OUTPUT()

As an exemplification of such differences, I present here a small description of the two procedures *INPUT()* and *OUTPUT()* which are present in both versions, but with different arguments lists.

The prototypes of the Bell version are:

INPUT(name, number, length)
OUTPUT(name, number, format)

where *name* is the name of the variable that will act as the input/output procedure, *number* is an Integer value that identifies the unit from which the input or to which the output is exercised, *length* is the Integer size of the input (no more than *length* characters are read each time) and *format* is a string containing the instruction for the output, usually a Fortran IV output sequence (for instance, '(1X,132A1)' is the *format* for *OUTPUT*, the principal and default

output command).

The prototypes of the Berkeley version are:

INPUT(name, scope, length)
OUTPUT(name, scope, attach)

where *name* is the name of the variable that will act as the input/output procedure, *scope* is an alphanumeric String identifying the unit from which the input or to which the output is exercised (DISK, TAPE, a file name, etc.), *length* is the Integer size of the input (no more than *length* characters are read each time) and *attach* is the character(s) concatenated to the output string of every record written (for instance, the New Line character is the default *attach* for *OUTPUT()*.

The two synopses are similar enough to be confused, but *they cannot be confused*! They cannot be used indifferently, because they don't use the same objects; in particular, the Bell version works with Integer values for the *units*[15], while the number of the input/output channel unit is hidden and invisible in the Berkeley version. Besides, the *format* in Bell depends on the Operating System (the printer coding), while *attach* in Berkeley is a simple string (even the empty string) appended to the output.

With such differences, the *INPUT()* and *OUTPUT()* procedures must be searched and translated when passing from a Berkeley to a Bell program and vice versa, surely an uneasy task.

This is so true that modern Snobol4 interpreters and compilers, and even Spitbol, since they no longer run on Fortran IV, use a fourth argument for *INPUT()* and *OUTPUT()* which contains the filename as String, and this means that they are pointing towards the Berkeley conception of procedures, rather than the other way around... A little satisfaction for me...

3.2.2. A case study of interpretive difference: Name operator's nature

The GGM (page 174) reports that, whenever the Name operator is used upon a natural variable, it returns a Name object. The GB, instead (page 133), that whenever the Name operator is used upon a natural variable, it returns a String object.

This is not a secondary question; consider for instance the two procedures *IDENT()* and *DIFFER()*; while they work in the same way for both versions, it is a fact that `IDENT(.VAR,'VAR')` must fail in Berkeley, because the object returned by `.VAR` is not a String but a Name object, and thus `.VAR` and `"VAR"` differ *by nature*. And that the same clause succeeds in Bell, because the object returned by `.VAR` is a string. The same can be said, with opposite results, for `DIFFER(.VAR,"VAR")`. This may cause different executions mechanisms because of the go-to architecture, that may be activated with success and or failures, and thus differently for the two versions.

This shows how disparate the point of view is with respect to the nature of the Name objects, which are more defined and sound in the Berkeley version, rather than in the Bell version.

3.2.3. A case study of mutual incomprehension: CONVERT()

The procedure *CONVERT()* in the two Snobol4 environments Berkeley and Bell was designed with the same name, but with very different scopes.

The Berkeley version had one argument, the Bell two.

The Berkeley version had a very limited conversion types, the Bell potentially all.

The Berkeley version could easily fail, the Bell had even the ability to return Array and Data pointers, and also to assign Code variables.

The differences were so many that the usage in one environment could not be ported easily to the other, and maybe in the past this caused some panic, among Snobol4 programmers.

[15] Like for instance BASIC with, say `PRINT #1, "HELLO"`, or Algol60 with `outreal(1,3.14)`, where in both examples the output channel is an integer known by the programmer.

See the dedicated chapters, that includes also the implicit conversion processes. See also Appendix 3, where the differences between the two version and also the soft version features are reported.

3.3. The structural differences

Other differences involve the *writing* of programs, and have a direct interest for programmers (see page 176 of the GGM). For instance, the final `END` label, a mantra for Bell, is not strictly required for Berkeley. Some other properties are surely welcome: for instance, the fact that a decimal number can be introduced with the leading dot; what modern language does not benefit of this property? The Bell interpreter didn't (and all the Spitbol family along).

Another major difference is in the string treatment: the double instances of single or double quotes inside a string counted as one, and this makes these strings not compatible with the Bell version, which couldn't recognized such double instances[16]:

```
        OUTPUT = "A PLACE CALLED ""HOME""!"
```

Bell rendition

```
 A PLACE CALLED""HOME""!
```

Berkeley rendition

```
 A PLACE CALLED"HOME"!
```

```
        OUTPUT = 'THAT''S THE WAY I LIKE IT!'
```

Bell rendition

```
 THAT''S THE WAY I LIKE IT!
```

Berkeley rendition

```
 THAT'S THE WAY I LIKE IT!
```

Another important difference is in the array pictures: whereas the Bell writes `LIST<2,3>` the Berkeley writes `LIST[2,3]`; this reflects a modern point of view but makes program portability a real trouble; the usage of `[..]` in place of `<..>` is used constantly: for example, the jump to a Code section is made as `:<NULP>` in Bell and as `:[NULP]` in Berkeley, and this adds another brick to the wall of incomprehension between the two worlds, even if I must observe that Spitbol, maybe in an attempt to solve this problem, can manage both formats indifferently (and so soft).

A final specific of the Berkeley version is that the assignment sign (=) does not need to be bounded by blanks, and the colon to identify a go-to does not need to be preceded by a blank. But try to omit these blanks with a Bell compiler!

In other cases, the Berkeley version proves less successful: for example, no more than 132 characters can be printed by *OUTPUT*, and the rest is lost. While this is not a serious problem, it is quite incomprehensible, as a string can be up to 131070 characters long[17]; the Bell version, on the other hand, could print all the text in as many lines as necessary[18]

My advice, anyway, is to write, if possible, compatible programs that can be executed anywhere: this makes life easier.

3.4. The Berkeley specifics

What is surely specific in the Berkeley version is the implementation of the meta-analysis tools, tools that the Bell version didn't include completely. Here is a comparative table for the meta-analysis tools[19]:

---

[16] See also the section "Plus or dot" at page 12 and following.

[17] See page 141 of the GGM where *MAXLNGTH()* is discussed.

[18] soft, being a Berkeley interpreter, has the same behaviour but widens the limit to host a whole string, whose dimension, in the current implementation, is fixed to a reassuring 2047 characters (plus the terminator).

[19] See also the GGM at Appendix M *Non Standard Features*, section III *Features not present in the Bell version*, on page 182, where the matter is discussed in detail.

Berkeley	Bell	Notes
/	*ARG()*	
ARRAY()	*ARRAY()*	the Bell version adds a second argument
CONVERT()	*CONVERT()*	the Bell version is more structured
DATATYPE()	*DATATYPE()*	substantially the same
FAMILY()	/	
/	*FIELD()*	
FIRST()	/	
ITEM()	*ITEM()*	the Berkeley version is more versatile
/	*LOCAL()*	
NEXTVAR()	/	
PROTOTYPE()	*PROTOTYPE()*	the Berkeley version treats more objects
REST()	/	
RIGHT()	/	
SELECTOR()	/	
TYPE()	/	
/	*VALUE()*	

In details:

- The Bell version could use specific commands, not implemented in Berkeley: *ARG()*, to access the name of an argument of a programmer-defined procedure, *FIELD()*, to access the name of a data-defined procedure method, *LOCAL()*, to access the name of a local variable of a programmer-defined procedure, *VALUE()*, for direct and indirect access to the content of a variable; all these tools offer a limited ability to analyze the program components.
- The Berkeley version instead introduced specific commands, not implemented in Bell, to access all components of an item: *FAMILY()*, *FIRST()*, *LEFT()*, *NEXTVAR()*, *REST()*, *RIGHT()*, *SELECTOR()* and *TYPE()*. (The descriptions of these commands can be read in this manual, or better in the GGM.) These tools analyze in a deeper way any program items.
- Some commands, implemented in both versions, worked a bit differently: *CONVERT()*, that in Bell was more versatile, with two arguments and a wider range of conversions, *PROTOTYPE()*, that was more extended in Berkeley, so that it could be applied to structures, Patterns, and Names, as well as to Arrays, and *ITEM()*, that was also more extended in Berkeley.
- Some other commands are more or less the same in both versions: *ARRAY()* (Bell adds the second argument for the default instantiation value) and *DATATYPE()*.

All considered, my opinion is that the Berkeley version could treat more in detail some meta-analysis on the text components.

3.5. The Berkeley missing pieces

The Berkeley version was missing some important pieces of the Snobol4 Standard, perhaps because its implementers could not complete their work, or perhaps because they found these features to be of little or no use in a normal average Snobol usage. They are listed in the following table, with notes about the behaviour of soft for each topic.

Bell	implemented in soft?
ARG()	YES
BACKSPACE()	YES
CLEAR()	YES
COLLECT()	NO
COPY()	YES
DUMP()	YES
DUPL()	YES
EVAL()	YES
FIELD()	YES
INTEGER()	YES
LOAD()	NO
LOCAL()	YES
OPSYN()	NO
REMDR()	YES
REPLACE()	YES
STOPTR()	YES, in part
SUCCEED	NO
TABLE()	NO
TRACE()	YES, in part
UNLOAD()	NO
VALUE()	NO
@ (cursor position)	YES
~ (negation)	NO
** (exponentiation)	YES
& (keywords)	YES, in part

In general, I suppose that their lack didn't compromise the functionality of Snobol, but this may be the reason why the Berkeley version wasn't replicated in Snobol's history, and that has some significance (above all for the Table type). Anyway, sometimes I wonder if the two versions could really coexist because I'm afraid that programs written on one side weren't really usable on the other side; probably, the two worlds could survive using their specific libraries, but any interchange was probably quite impossible.

However, in the design of soft, I tried to add some of these missing pieces, as shown in the previous table, when it seemed to me that they were relevant procedures, or in case their implementation did not compromise the basic structural coding. In any case, I want soft to be a Berkeley rendition, though adapted to the modern computer architectures.

3.6. The Ampersand keyword operator

In the Bell version, several system variables names are prepended by the ampersand symbol `&`; this symbol signals that the variables are *system variables*, called *keywords*, and that they usage is specific.

The Berkeley version made no use of these keywords; in soft, instead, I implemented quite all the Bell keywords, because I think this is a very efficient way to keep track of some system settings, while their absence may be a problem. All these keywords can be read, but only a few can be assigned. Some of these keywords have a Berkeley procedure counterpart, but the majority don't. Moreover, all these keywords are contained in program registers that don't belong to the world of the natural variables, so that they won't appear in the *DUMP()* output (unless explicitly required).

The ampersand symbol has no other peculiar usage in Berkeley Snobol4.

3.7. The difference in the used language

On pages 172-173 of the GGM there is what is explained as *Different terminology* in the definition of the program alphabet.

The following table shows the differences in terminology to identify the same object or area of interest:

Bell description	Berkeley description
primitive	predefined
defined	programmer-defined
function	procedure
predicate	test procedure
value of function name	value of result variable
formal argument	formal variable
local variable	internal variable
function procedure	procedure body
entry point	entry label
explicit name	string name
created name	Name
implicit name	Name
generated variable	indirect reference
aggregate	family
referencing argument	selector
array element	array item
array reference	item reference
field function	field selection procedure
source program	program text
statement component	statement part
subject (assignment)	left side
subject (pattern match)	string reference
object	right side
compilation error	compile-time error
program error	execution-time error
listing controls	listing directives

In soft I tried to adopt the Berkeley terminology, but for some of the new features I'm afraid I've slipped into Bell's slang, and I apologize if somewhere the text is contradictory...

3.8. Original restrictions of the Berkeley version with respect to the Bell version

The Berkeley version was built intentionally different from the Bell. It follows from the power of the meta-linguistic features, which were not in the Bell version.

All the missing features, previously exposed, were described in appendix M of the GGM, but my impression is that they were kept off by design. This would mean that the original design of the Berkeley version was built as a stand-alone version.

On the other hand, the fortune of the Berkeley version is known: it had no fortune, while the Bell version still circulates today, the branch of Snobol and the branch of Spitbol as well.

I hope soft will be considered as the interpreter that continues the Berkeley tradition, even if some characteristics were added, removed or changed, establishing a bridge from the Seventies to today.

The differences are briefly summarized in the table at Appendix 3.

3.9. Differences between soft and the Berkeley versions

Writing an interpreter isn't really an easy task, and sometimes I had to decide whether to implement some commands differently, partly because the original old machines were built differently, partly because I'm not a professional programmer, and I have to deal with my skill level, and partly because I work alone and just can't always go into the deep.

In this chapter I report all the restrictions, the enhancements, the modifications and in general all the news that you may need to know to use soft consciously.

One of the differences is linguistic: I substituted the word Structure with the word Data because what is called Structure in the GGM is built with the *DATA()* procedure, which creates a Data constructor.

3.9.1. Restrictions with respect to the Berkeley version

The following restrictions are present in soft, which are not in the Berkeley version:

- The six predefined variables cannot be changed, by default. I set this to avoid any wrong usage of the system variables names. If you need such a name, add a dot at the end, like in `ABORT.`: this will create a variable with a different name than *ABORT*; if you absolutely want to alter the behaviour of a system variable, use the `--alter` option; I warn you: you must be aware of what you're doing. To reassign the predefined value, set it as a string corresponding to the original content (`"&ABORT"` for the present instance) and soft will recognize that this is a system readdress, and will set *ABORT* to be a regular pattern variable again.
- The argument of `:[...]` must be a Code variable or a literal String, and cannot be a *CODE()* statement.
- The procedure *ALPHABET()* and the system variable *&ALPHABET* both return the soft alphabet in the range from ASCII 1 to ASCII 255, because character 0 in position 0 would null the string (it's a C feature); in soft, this is substituted with ASCII FF; remember though that characters from ASCII 128 to ASCII 255 are platform-dependent.
- The procedure *MAXLNGTH()* can be used only in read-mode, that is the string length is not changeable.
- The extended syntax in the Berkeley Snobol4 (exposed starting on page 156) is not implemented; in particular, soft does not support:
  - alternative prototype representation of arrays: `ARRAY('O/2,O/3')`
  - alternative Item references representation: `LIST(/2,3/)`
  - alternative go-to parts representation: `/F(LABEL)`
  - alternative alternation representation: `A // B`

  This alternative syntax was introduced because some keyboards in those days (those used to punch program cards) could be missing some keys, and this could mean the inability to insert a program. Modern keyboards are more or less standardized and the characters used in the standard syntax are all available, so no alternative syntax is needed.
- The evaluation of patterns is always returned enclosed in brackets, to ensure that deferred assignments are correctly interpreted. I don't know if this is a real issue or if the Berkeley compiler behaved differently. In any case, the brackets are never shown, unless you use option `--print-complete` or other debugging techniques and their presence does not influence by any means the pattern evaluation engine.

  Look for instance at the following PIE[20] session:

```
$ set
Set flag : printc
Set state: on

$ PAT = LEN(1) (SPAN("JK"))

Yes.

$ OUTPUT = PAT
[PATTERN] = (LEN(1) ( SPAN("JK") ) )

Yes.

$ "I LOVE JK FITZEGERALD" PAT $ OUTPUT
 JK

Yes.
```

  The variable `PAT` contains the pattern `LEN(1) ( SPAN("JK") )"`, but it is saved with a couple of extra brackets. When later it is used in a pattern evaluation, the added brackets help in assigning to `OUTPUT` the string `" JK"` (with a prepending blank), rather than the single token `"JK"` that would emerge using no extra

[20] Look at chapter 8 for the PIE session details.

brackets, unless you type

```
$ "I LOVE JK FITZEGERALD" (PAT) $ OUTPUT
```

which is unnatural and unsound.

3.9.2. Enhancements with respect to the Berkeley version

The following enhancements were introduced in soft, which are not in the Berkeley version:

- The lower alphabetical characters are available. Normally, all items (even the created items) can be written in lower or upper characters, because soft is case-insensitive, but all strings can be written with lower or capital, and they will be maintained both in memory and in output.
- In input, strings are not limited to 80 characters but extend to 2047 characters, for wider input; the drawback is that if the string is shorter than 2047, the rest is filled with blanks. This suggests (as in the original Berkeley version), the usage of *TRIM()* or set *&TRIM* to anything different than zero.
- The numeric argument of a system- or user-procedure may be any mathematical expression; this is an enhancement with respect to current Snobol implementations, which accept only single numeric entities whereas a number is expected. I don't know what was the behaviour of the original Berkeley version.
- Mixed-mode Real/Integer arithmetic or comparison between Real and Integer numbers is available.
- The `\` operator for the integer division is available.
- The deferred assignment operator is recognized as a procedure's argument, e.g. `LEN(*V)`.
- The `@` cursor operator is available, if directly bound to the left of the assignment variable.
- The ability to recognize some C escape characters in strings (that are performed when printed); the following characters are recognized:

Feature	Escape	ASCII value	Code/name
Alarm beep	`\a`	07	BEL
Back space	`\b`	08	BS
Horizontal tab	`\t`	09	HT
Line feed	`\n`	10	LF
Vertical tab	`\v`	11	VT
Form feed	`\f`	12	FF
Carriage return	`\r`	13	CR
Double quote	`\"`	34	quotes
Single quote	`\'`	39	apostrophe

  If you try to escape other characters, The `\` slash characters and the following character count as themselves (e.g. the string `"\n"` has length one, the string `"\Z"` has length two).
- The ability to convert implicitly the < and > symbols to the most common `[` and `]` characters respectively, in an attempt to reduce the incompatibilities with the Bell version.
- To catch single ASCII characters in any position, two procedures were provided: *CHAR()* and *ORD()*, both not belonging to the Berkeley version, that work with any integer in the range from ASCII 0 to ASCII 255. *CHAR()* is a string procedure, while *ORD()* is a numeric procedure.
- a procedure's argument that expects a number may be a math expression of any kind, whose calculated result is the argument's value.
- The procedure *ARRAY()* can manage the second argument, that is assigned to all items.
- The following math procedures are available in math expressions: *ABS()*, *ARCTAN()* (with alias *ATAN()*), *COS()*, *ARCCOS()* (with alias *ACOS()*), *EXP()*, *LOG()* (with alias *LN()*), *PI()* (with alias *&PI*), *ROOT()* (with two parameters), *SIN()*, *ARCSIN()* (with alias *ASIN()*), *SQRT()* (with alias *SQR()*) and *TAN()*. All these math procedures return a Real number, except for *ABS()*, which preserves the type of the argument, like for instance in `ABS(-3)`.
- The following bitwise/logical procedures are available: *AND()*, *OR()*, *XOR()*, *NOT()*, *EQV()*, *IMP()*. When used with Integers, they work in bitwise mode and return Integers; when used with test procedures, they work in logical mode and return a success/failure state (and work as test procedures).

- The following bitwise procedures are available: *LSHIFT()* (with alias *SHL()*), *RSHIFT()* (with alias *SHR()*), *LROLL()* (with alias *ROL()*), *RROLL()* (with alias *ROR()*). They always work in bitwise mode[21].
- The following procedures for the generation of pseudo-random numbers are available: *RANDOM()* and *RANDOMIZE()*.
- Statistics are provided by the `-s/--stats` option.
- Tracing is provided by the added *TRACE()* Snobol4 command (in a reduced syntax, with respect to the Bell version). See the relative chapter.
- Dumping is provided by the added *DUMP()* Snobol4 command. See the relative chapter.
- Quite all the Bell keywords (plus some more) are added, and for what is the correspondence with the Berkeley counterparts, using one or the other method is the same - for instance using `&ANCHOR = 1` is the same as `ANCHOR(1)`.
- The following procedures, also in Berkeley, were added in soft with some enhancements:
  - The procedure *CLOCK()* returns the current time of the day, in the format `"HH:MM:SS"`; e.g. `TIM = CLOCK()` assigns to `TIM` the string `"15:25:48"` (an example). In soft *CLOCK()* has been enhanced to return even single data. See the vocabulary reference.
  - The procedure *DATE()* returns the current date, in the format `"DD MON YY"`; e.g. `DAT = DATE()` assigns to `DAT` the string `"24 APR 20"` (an example). In soft *DATE()* has been enhanced to return even single data, and also the Spitbol format date-plus-hour format. See the vocabulary reference.

3.10. Full scan or quick scan?

The original Snobol4 (at least in the Bell version) could use the `&FULLSCAN` system variable for controlling the scan mode; if set to 1, every pattern evaluation produced all the alternatives, even those that clearly are not possible (a technique called *full scan mode*; if set to zero, only the possible results were found, using a heuristic technique to avoid the impossible results (a technique called *quick scan mode*). The Berkeley version, instead, having no ampersand system variables, was set to use the full scan mode; on page 181 it's written: "*There is no quickscan mode of pattern-matching (a mode which makes use of heuristics). This is the normal mode in the Bell version, while fullscan is the normal mode in the Berkeley version.*".

soft, being an emulator of the Berkeley version, has no quick scan mode but, to make the program compatible with other listings, the `&FULLSCAN` variable is read and can be assigned, but it is not used.

Incidentally, also Spitbol was built to use the fullscan mode, and also in Spitbol the `&FULLSCAN` variable is read and can be assigned (with a non null value), but it is not effectively used; this is what is written in the Spitbol manual (at pages 123-124):

"*Pattern matching can be time-consuming because of the number of possibilities which must be attempted. The original SNOBOL4 system applied heuristics to the process, eliminating match attempts that could not possibly succeed. This was done by carrying out side computations of the remaining subject characters, and the amount required at any point in the pattern.*

Whether the extra memory and computations required by the heuristics are worth the savings in pattern matching time has long been debated. What is known is that the introduction of the immediate assignment operator meant that heuristics were no longer invisible to the programmer. Eliminating some match attempts eliminated some immediate assignments, and programs might run differently with the heuristics turned on or off.

The author of SPITBOL has concluded that the heuristics do not result in a significant increase in speed. Furthermore, it often produces malfunctioning patterns when deferred evaluation is used within a pattern. Accordingly, pattern matching is done exhaustively and no heuristics are applied. In particular, deferred expressions are not assumed to match at least one character, and recursive patterns always work properly.

Heuristics were called the "Quickscan mode" of pattern-matching. They were controlled with the &FULLSCAN keyword-zero for Quickscan, or non-zero for "Fullscan mode," where all combinations are tried.

The &FULLSCAN keyword has been retained, but it may only be set to a non-zero value. Attempting to set it to zero will produce an error message."

[21] The normal and alias names come from different traditions (Spitbol and Fasbol).

soft does not print any error message if you assign zero to *&FULLSCAN*, but sharing this point of view with the Berkeley and Spitbol versions can be defined as being in *good company*!

3.11. Programming tips

The GB (par. 11.5, p. 194) reports some simple programming rules that are worth remembering; I report them here with the necessary adapting to the Berkeley side:

1 Use the anchored mode if possible.

2 Use *BREAK()* instead of *ARB* if possible.

3 Use *ANY()* instead of alternation if possible.

4 Avoid use of *ARBNO()*.

5 Use conditional rather than immediate value assignment if possible.

The compliance with these suggestions is not mandatory. But I, who know all the possible weaknesses of soft, feel that they must definitely be followed.

Now it's up to you.

4. THE soft's SNOBOL REFERENCE: THE VOCABULARY

Here follows the Snobol4 reference for all the procedures that are builtin in soft, divided among the following field of interest:

1 pattern procedures
2 test procedures
3 language procedures
4 string procedures
5 mathematical procedures
6 bitwise and logical procedures
7 Bell keywords

4.1. General notions

The fast reference guide of the Snobol4 language in the next chapters is written in accordance with the following convention:

- A string argument has the suffix *_str*
- An integer argument has the suffix *_int*
- A generic numeric argument has the suffix *_num* (Integer or Real)
- A pattern argument (even of multiple patches) has the suffix *_pat*
- An Array argument has the suffix *_arr*
- A Data argument has the suffix *_dat*
- A Name argument has the suffix *_nam*
- A generic argument (which may be of any type) has the suffix *_val*
- If the arguments number is of indefinite length, an ellipsis such as ... is added after the last argument
- Each argument is prefixed with a name describing the scope or purpose of the argument itself:
  - in case it's a generic argument, the prefix is *arg*;
  - in case it's a prototype, the prefix is *proto* (and you are invited to consult some Snobol4 manuals for the correct usage);
  - in case it's an ignored argument, the prefix is *dummy*;
  - otherwise it's a name that describes the function of the argument.
- If more than one argument of the same type is contained in the procedure declaration, arguments my be suffixed with a progressive index following the name, not the suffix; e.g. *arg1_str*, *arg2_int*.

In the body of an entry, the word *fail* in general indicates a failure of the rule, that may be coordinated by a proper *goto*; the word *break* indicates a formal error that is raised with a proper message, and the execution stops immediately.

Each entry reports one of the following marks at the end of the definition:

- *Standard*: it means that the procedure belongs both to the Bell and to the Berkeley versions. This can be decorated with an asterisk to underline the fact that the soft version behaves differently.
- *Berkeley Standard*: it means that the procedure belongs to the Berkeley version only. This can be decorated with:
  - \+ to indicate an enhancement of the procedure, with respect to the original Berkeley feature;
  - \- to indicate the presence of less efficient features in the procedure;
  - * to indicate a different behaviour of the procedure.
- *Bell Standard*: it means that the command belongs to the Bell version only, and is added to soft as an enhancement. This can be decorated with an asterisk to underline the fact that the soft version behaves differently.
- *Not Standard*: it means that the command belongs neither to the Bell nor to the Berkeley versions, and is added to soft as an enhancement.

4.2. Pattern procedures

The pattern procedures can be generally used inside a pattern evaluation, and have no direct scope or use in other fields (e.g. assignments). I include in this set the six default pattern variables, which have a similar field of application.

ABORT
(system variable) causes the immediate failure of the pattern during an evaluation. *Standard*.

ANY(arg_str)
matches one character in the base string that also belongs to the set of characters in its argument string *arg_str* or fails. If *arg_str* is null or not a proper character string, it breaks. *Standard*.

ARBNO(arg_pat)
matches zero or more consecutive occurrences of the string matched by the argument pattern *arg_pat*. *ARBNO()* matches the shortest string possible, starting from the null string; in case of rematch, the patterns number is increased by one, and fails when no match is possible. *Standard*.

ARB
(system variable) matches an arbitrary number of characters in the base string, returning the shortest string possible, starting from the null string; in case of rematch, the number is increased by one character, and fails when no match is possible. *Standard*.

BAL
(system variable) matches the shortest string which is composed by single characters or by a complete section enclosed in `(` and `)`; the first unbalanced `)` character found causes the failure of the match. The first return value is always a not-empty string. *Standard*.

BREAK(arg_str)
matches the longest string composed by any characters not belonging to its argument string *arg_str*. It can match the null value. See the dedicated chapter. *Standard*.

FAIL
(system variable) matches **no strings** (not even the null value), and always fails. *Standard*.

FENCE
(system variable) marks a position in the pattern; if the evaluation, on backtracking, tries to step back beyond *FENCE*, the evaluation fails. See the dedicated chapter. *Standard*.

LEN(arg_int)
matches any string of characters of the length given in its argument *arg_int*. If *arg_int* is lower than zero or greater than than *MAXLNGTH()*, it breaks. *Standard*.

NOTANY(arg_str)
matches one character in the base string that does not belong to the set of characters in its argument string *arg_str* or fails. If *arg_str* is null or not a proper character string, it breaks. *Standard*.

POS(arg_int)
returns a pattern which will match only the string position specified by its argument *arg_int*, and fails otherwise; it matches no characters at all. (String positions follow the numbering convention of *TAB()*.) If *arg_int* is lower than zero or greater than than *MAXLNGTH()*, it breaks. See the dedicated chapter. *Standard*.

REM
(system variable) matches all the remaining (none-or-more) characters from the current cursor position. Another pattern equivalent to this is `RTAB(0)`. *Standard*.

RPOS(arg_int)
returns a pattern which will match only the string position specified by its argument *arg_int*; it matches no characters at all (the numbering convention of *RTAB()* is used for the string positions). If *arg_int* is lower than zero or greater than than *MAXLNGTH()*, it breaks. See the dedicated chapter. *Standard*.

RTAB(arg_int)
specifies pattern matching not in terms of character content or of length, but in terms of position within the base string. The argument *arg_int* is a non-negative integer, and the returned pattern matches all the characters up to that position within the string reference, starting from right. If *arg_int* is lower than zero or greater than than *MAXLNGTH()*, it breaks. See the dedicated chapter. *Standard*.

SPAN(arg_str)
matches the longest string composed exclusively of the characters belonging to its argument string *arg_str*. It cannot match the null value and causes backtracking in case. See the dedicated chapter. *Standard*.

TAB(arg_int)
specifies pattern matching not in terms of character content or of length, but in terms of position within the base string. The argument *arg_int* is a non-negative integer, and the returned pattern matches all the characters up to that position within the string reference, starting from left. If *arg_int* is lower than zero or greater than *MAXLNGTH()*, it breaks. See the dedicated chapter. *Standard*.

4.3. Test procedures

The test procedures analyze their argument(s) and alter the final state, influencing thus the go-to process.

DIFFER(arg1_val[, arg2_val])
succeeds if the two arguments *arg1_val* and *arg2_val* are not equal, both in the content or the type, and fails otherwise; if the scope is finding if *arg1_val* is not null, the second argument can be omitted because it evaluates to the empty string anyway. *Standard*.

EOI(scope_str)
tests whether the fileset specified by its argument *scope_str* is positioned at the end-of-information. If so, the procedure succeeds and returns the null value. If there is more information on the file, the procedure fails. If the current standard input or output channels are selected it or, if *scope_str* is a file not already opened, it breaks. *Berkeley Standard*.

EQ(arg1_num, arg2_num)
succeeds if the two arguments *arg1_num* and *arg2_num* are numerically equal in value and in type. Thus `EQ(23,'+00023')` will succeed, since both arguments have the numeric value of the Integer 23, while `IDENT(23,'+00023')` would fail since character for character identity cannot be found between the two strings. The expression `EQ(NULL,0)` will succeed since the null value and zero are arithmetically identical. *Standard*.

FILE(filnam_str)
succeeds if the file *filnam_str* is found, and fails if the file is not found, or anyway it is not available. The name can be given with path, in absolute or relative format. The `~` and `$HOME` symbols are substituted with the user home directory (Unix/Linus only). *Not Standard*.

GE(arg1_num, arg2_num)
succeeds if the first argument *arg1_num* is numerically greater than the second argument *arg2_num* or if it is equal in value and type to the second argument, and fails otherwise. *Standard*.

GT(arg1_num, arg2_num)
succeeds if the first argument *arg1_num* is numerically greater than the second argument *arg2_num*, and fails otherwise. *Standard*.

IDENT(arg1_val[, arg2_val])
succeeds if the two arguments *arg1_val* and *arg2_val* are equal character-by-character or, in case of variables, if they are equal in the content and in the type; if the scope is finding if *arg1_val* is null, the second argument can be omitted because it evaluates to the empty string anyway. Besides, while `EQ(23,'+00023')` will succeed, since both arguments have the numeric value of the Integer 23, `IDENT(23,'+00023')` would fail since character for character identity cannot be found between the two strings. The expression `IDENT(NULL,"")` will succeed since the null value and the empty string are identical. *Standard*.

INTEGER(arg_num)
succeeds if the value of *arg_num* is an integer and fails otherwise. Thus, `INTEGER(X)` succeeds for `X = 3` or `X = '3'` or X = '' (which corresponds to the zero value) but fails for `X = 'INT'` or `X = ' ' (space) or X = 3.0 (Real)`.. *Bell Standard**.

LE(arg1_num, arg2_num)
succeeds if the first argument *arg1_num* is numerically lesser than the second argument *arg2_num*, or if it is equal in value and in type to the second argument and fails otherwise. *Standard*.

LGT(arg1_str, arg2_str)
compares the two string arguments *arg1_str* and *arg2_str* to see if they are alphabetically ordered, and succeeds if *arg1_str* is greater than *arg2_str*, that is if the first argument follows the second argument[22], using as an alphabet the computer's character set in its standard collating sequence, which is the ASCII table for the present case, and fails otherwise. *LGT* is a mnemonic for *Lexicographically Greater*. *Standard*.

LT(arg1_num, arg2_num)

[22] This may be difficult to understand in full at first.

succeeds if the first argument *arg1_num* is numerically lesser than the second argument *arg2_num*, and fails otherwise. *Standard*.

NE(arg1_num[, arg2_num])
succeeds if the two arguments *arg1_num* and *arg2_num* are numerically not equal in value or in type. Thus `NE(23,'+00023')` will fail, since both arguments are numerically equal, while `DIFFER(23,'+00023')` would succeed, since character for character identity cannot be found between the two strings. The expression `NE(NULL,0)` will fail since the null value and zero are arithmetically identical. *Standard*.

4.3.1. Positions of test procedures

A test procedure can be used in several positions. Let's review all the cases.

Assignments

If a test procedure is used in an assignment, it is used to set a condition on the assignment itself, that can or cannot take place; for instance, the line:

```
TEST = GT(A,0) "STRING"
```

sets `TEST` to `"STRING"` only if A>0. In case A is negative or null, `TEST` will be assigned the empty String, so that it is anyway created. The success or failure of the assignment can of course coordinate a proper go-to.

To avoid the creation of an empty variable, the test can precede the variable name:

```
GT(A,0) TEST = "STRING"
```

In this case, if A is negative or null, the variable `TEST` is not even created, and this avoids the proliferation of empty variables[23], which in any case is of no harm, in Snobol.

Pattern evaluations

If the test procedure is used into a pattern evaluation, it may be used to set a condition on the evaluation itself, that can make it fail or succeed independently of other pattern figures; for instance, the line:

```
BASE DIFFER(A,"VAL") A = "TERM"
```

starts the evaluation only if `A` does not contain the term `"VAL"`; if so, the pattern evaluation is tried, and it may fail or succeed. In case it succeeds, the content of `A` in `BASE` is substituted with `"TERM"`.

If the latter strategy succeeds, this is because of the combined successes of *DIFFER()* of and the pattern evaluation; if, on the other hand, it fails, this is may be due either to the failure of *DIFFER()* or to the failure of the pattern evaluation; this has the consequence that, in case of failure, it's not possible to know which one failed. Of course it's always possible to perform the two tests separately, elsewhere, if this distinction should be necessary.

As a standalone procedure

Finally, a test procedure can be used also alone on a line, and in this case its success or failure is the one that coordinates the go-to; for instance, the line

```
LT(A,0)                 :S(END)
```

simply express the fact that in case A<0 the program must end (of course, it may be also directed to any legal label, not necessarily to `END`); on the contrary it will simply step to the next source line.

---

[23] To tell the truth, I have noticed a strange behaviour in the current Spitbol and Snobol4 interpreters; if `GT(A,0)` fails, the two interpreters simply fail and that's good; if it succeeds, they break with an error message. Why?
This is apparently a violation of the Snobol protocol for test procedures, because if they fail, they make the whole clause fail, and in case they succeed, they become transparent to the whole clause. Or at least this is how it is seen in soft.

4.4. Language procedures

The procedures in this set perform a single operation that alters the normal program behaviour. In this set I put also the specific procedures that act as *containers* for other scopes (like *APPLY()* or *EVAL()*) that may return more than one type.

ANCHOR(arg_val)
enables or disables the Anchor Mode (that is, a pattern succeeds only if it matches the initial part of the base string). Its argument *arg_val* is any string or number which, if evaluated to a non-empty string or to a value different from zero, enables the Anchor Mode, otherwise disables the Anchor Mode; the form `ANCHOR()` (without arguments) disables the Anchor Mode. The Anchor Mode can also be activated by means of the `--anchor` option. *Berkeley Standard*.
Note: &ANCHOR is the Bell correspondent keyword of ANCHOR(), and can be used in read/write mode also in soft.

APPLY(fname_str, arg1_val, arg2_val, ...)
APPLY(selector_str, family_dat)
calls the procedure whose name appears as the first argument string *fname_str*, feeding it with the arguments supplied from the second argument on; the return value and type of *APPLY()* are the same value and type returned by the procedure call; it can be used also on user *DEFINE()* procedures. *APPLY()* is also equivalent to the Data selector identified by the first argument *selector_str*, and the family as the second argument *family_dat* (which is of type Data); the return value and type are the same as for the method in question. *Standard*.

ARRAY(proto_str[, arg_val])
is a procedure that creates an array; arrays are collections of values of different nature, they may have any dimension up to 16, and any width for any dimension up to the limit of the hosting computer. Arrays are initialized to the empty string and to zero, unless the second argument is used[24], and in this case the array item will have the type and value of that argument. *Berkeley Standard+*.

BACKSPACE(scope_str)
repositions backwards one record (i.e. backward one text line) the scope fileset specified by its argument *scope_str*, which is the scope fileset previously used as the second argument to the *INPUT()* or *OUTPUT()* procedures. The file is a text file. The method used is as follows: if an end-of-line character is found immediately preceding the current file position, it is ignored and discarded; the file is then scanned in reverse until another end-of-line character is found. The file is positioned just forward of the end-of-line character found. *BACKSPACE()* stops at the beginning-of-file. If the channel is not a text file, the procedure fails. If it succeeds, the null string is returned, and the next input procedure, which is directed on the scope fileset, will read the previous record again; conversely, in case of an output procedure, the end-of-line just read is re-emitted, to realign the output, and the next output procedure will overwrite the previous text instance that follows[25]. *Bell Standard**.
Note: the BACKSPACE() procedure does not erase the data after the new position, so that in an output procedure that emits lesser characters than the previous record, some of the old characters may survive in the file, and resurface when the file is opened in input again. To avoid this effect, assure you print more characters than the previous, or fill the output with spaces.

CLEAR(dummy_val)
sets the values of all natural variables to the null string, without affecting the values of keywords, created variables, I/O associations or procedure definitions[26]. It applies to the natural variables of type Integer, Real, String, Pattern, Name and Code, but not to the created variables of type Array or Data (included their various Data methods). It applies also to the natural variables *ABORT*, *ARB*, *BAL*, *FAIL*, *FENCE* and *REM*, and clears their predefined pattern values. The corresponding keywords *&ABORT*, *&ARB*, *&BAL*, *&FAIL*, *&FENCE* and *&REM* can be used to restore the predefined patterns. *Bell Standard**.

CODE(arg_str)
accepts as argument a string which is a Snobol4 program text (that is, a sequence of syntactically-correct Snobol statements), and returns as value the corresponding compiled type Code suitable for use with `:[code]`; its use,

[24] The Berkeley Standard didn't include the initialization parameter.

[25] The Bell syntax used the unit number, rather than the scope fileset. I had to adapt the original Bell procedure to the Berkeley structure, that does not make use of the unit number.

[26] The original Bell version cleared variables only at the level at which *CLEAR()* was called. soft instead clears all variables regardless of the level.

then, is to permit a program to extend itself while it's executing. All characters in the Snobol character set, including spaces, have their customary significance in the argument to *CODE()*. Statement separators are semicolons, but no final semicolon is required in the string. *Standard*.

CONVERT(arg_str)
CONVERT(arg_int)
CONVERT(arg_num)
(Berkeley version) returns the very same argument with the type converted to another type; this is the Berkeley procedure, which behaves as follows: if the single argument is a numeral String or Integer, it's converted into a Real number; if the single argument is a Real number, it's converted into String type. See the dedicated section for the relative problems. *Berkeley Standard*.

CONVERT(arg_val, arg_str)
(Bell version, with two arguments) returns the object in the first argument *arg_val*, which is potentially of any type, converted to the type defined in the second argument *arg_str*. See the dedicated chapter. *Bell Standard-*.
Note: If used with one argument only, the Berkeley version is applied, with different behaviours and return types.

COPY(arg_val) returns a distinct copy of *arg_val, which is a variable name of an Array, Code block, Pattern, or user-defined data type. For instance, If* `A` is an Array, the statement `B = COPY(A)` creates a new array `B`, whose initial contents are the same as array `A`. Their elements are independent; altering element `A` does not affect element `B` . By contrast, as reported elsewhere, the assignment `B = A` makes `A` and `B` alternate names for the same array. If *arg_val* is the name of a String or a number, the String or the number are returned unchanged.

DATA(proto_str) accepts as argument a prototype string in the form of a constructor with argument fields, as in `DATA('COMPLEX(R,I)')`, where the constructor `COMPLEX()` requires two fields, named respectively `R` and `I`. Any further assignment in the form `C = COMPLEX(2.1,-1.0)` will assign to `C` the data type `COMPLEX` with the two instantiated fields `R = 2.1` and `I = -1.0`. The fields of such data type can be access through methods of the same name, methods that must be used with the name of the instantiated variable as arguments; for instance, the imaginary component of `C` is accessed through `I(C)`. *Standard*.

DEBUG(arg_val)
enables or disables the debugging. Its argument *arg_val* is any string or number which, if evaluated to a non-empty string or not zero number, enables the debugging, otherwise disables the debugging; the form `DEBUG()` (without arguments) disables the debugging. *Not Standard*.

DEFINE(proto_str[, label_str])
accepts a prototype string *proto_str* in the form of a procedure, with a name and all necessary variables in parentheses, separated by commas, with possibly other variables specified afterwards, again separated by comma and without interspersed spaces, and creates a reference in the memory to the procedure; an optional second argument *label_str* defines the starting label of the procedure. After *DEFINE()*, the call of the procedure name with the proper arguments is ready for use; the call of the procedure body will execute the relative code until a *RETURN/FRETURN/NRETURN* is met, after which the normal program flow is re-established. See the chapter devoted to *DEFINE()*. *Standard*.

DETACH(arg_nam)
DETACH(arg_str)
breaks the association between the variable named by its argument and any fileset. There is no need to detach an associated variable before giving it a new association. (A variable may be associated with only one fileset at a time, but a fileset may have many variables associated with it simultaneously). If a variable should not be previously associated to a fileset, no warning is printed. *Standard*.

DUMP(arg_val)
enables or disables the variables dumping at the end of execution. The argument *arg_val* is any string or number which, if evaluated to a non-empty string or not a zero, enables the Dumping for all non-null variables, otherwise the Dumping is disabled; the form `DUMP()` (without arguments) disables the Dumping as well.
As an enhancement to the dumping of natural variables, the Data methods are printed as well; besides, if *arg_val* is 2, also the unprotected variables list is printed; if *arg_val* is 3, all protected and unprotected variables lists are printed; finally, if *arg_val* is 4, the default sic system variables and the non-null variables are printed as well. The dumping may also be activated by means of the `--dump` option, which corresponds to *arg_val* equal to 1; it can

also be activated by means of the `--dump=<n>` option, with `<n>` in the range `1..4`, that shapes the dumping according the user's needs without affecting the code. *Bell Standard**.
Note: &DUMP is the Bell correspondent feature of DUMP(), and can be used in read/write-mode also in soft, with values from 1 to 4.

ECHO(var_nam, arg_int)
ECHO(var_str, arg_int)
changes the echoing in input on the channel identified by the input variable *var_nam* (which is of type Name) or *var_str* (which is of type String), basing on the second argument *arg_int*, disabling echo if it's zero, enabling it otherwise. *Not Standard*.

END
is a system not mandatory label[27]; when found, if present as the last line of the program, it causes the immediate stop of the program loading, marking the end of the program; otherwise, if it is not present as a label in the program, it can be invoked by a go-to: the call `:(END)` will nonetheless cause a jump to the end of the program, otherwise the end of the program happens when running out of text. Remember though that, if you have text following the program, loaded through options `-r/--resource`, the *END* label must be present, in order to stop the file loading, assuring that what follows *END* is the expected text. For the previous reasons the *END*, even if not mandatory, cannot be used elsewhere as a simple label[28]. *Berkeley Standard**.

ENDFILE(scope_str)
closes the scope fileset specified by its argument *scope_str*, and detaches all variables that pointed to it. If the file is not opened, the procedure fails[29]. *Bell Standard**.

ENDGROUP(scope_str[, level_int])
writes an end-of-group mark on the scope fileset which is specified by its first argument *scope_str*. The level associated with the mark is specified by the second argument *level_int*, which must be an integer between 0 and 15 inclusive (zero if missing). Such a mark of any level will cause failure on input if later read by a Snobol4 program. *Berkeley Standard*.

ERRTEXT(arg_int[, arg_str])
returns as String the error text which in the system list corresponds to the argument *arg_int*. In doing this, if *arg_int* is in the range of the errors list, *&ERRTYPE* is reset to zero, otherwise a recoverable error is set and the clause fails. If the error requires a second argument (for errors 46,78, 79 and 107), this must be added as the second argument *arg_str*; it is in form of a String even for error 46, which requires a number. *Not Standard*.

EVAL(arg1_str[,arg2_str, ...argn_str])
evaluates the expression in the arguments *arg1_str, arg2_str, ...argn_str* that, after concatenation, must return a value of Integer or Real type (it can also process Strings that contain the deferred operator in cases like `*` or `*(..)`, even if they are Pattern figures, provided they return a number after the evaluation). Plain Integer or Real arguments are simply returned as values, without modification. This procedure, even if it seems quite useless, can be used to interpret mathematical expressions that are read from keyboard or from file, using the variables in memory, which operation could be problematic without *EVAL()*. *Bell Standard**.

FAMILY(arg_nam)
accepts as argument the Name of a created variable (array item, or field of a programmer-defined structure) and returns the object which is the family of variables to which the variable belongs. *FAMILY()* returns the Array or structure rather than the Name of the variable whose value is the Array or structure, and thus the value returned by *FAMILY()* is suitable for use as the first argument of *ITEM()*, or a second argument of *APPLY()*. In any other case it returns the name of the family as a String. See the dedicated chapter. *Berkeley Standard*.

FIRST(arg_pat)

[27] For the Bell version, on the contrary, the *END* label is mandatory.

[28] The Berkeley version had no `-r/--resource` option for the usage of the same file as a unique source and data container, and so it could use *END* as any other label; in this regard, in the original Berkeley version, `:(END)` does not lead to the program's end, but is a simple jump to the line labelled with *END*. In soft, the first *END* met causes the stop of parsing, and so it cannot be used in a line which is not the last. This is a minor difference with the Berkeley version,

[29] The Bell syntax used the unit number, rather than the scope fileset. I had to adapt the original Bell procedure to the Berkeley structure, that does not make use of the unit number.

evaluates the pattern argument *arg_pat*, which is an alternation of the form `X | Y | Z` or a concatenation of the form `X Y Z` (where `X`, `Y` and `Z` are proper patterns or pattern variables), and returns the first element of the pattern. *Berkeley Standard*.

FORWARD(scope_str)
performs a standard forward on the fileset specified by its argument *scope_str*, advancing the file pointer to the end-of-information point; if an end-of-group mark of level zero is found as the last written item, supposedly written by a *REWIND()* procedure, an implicit *BACKSPACE()* is performed on the file to ensure it can be rewritten in case. *Not Standard*.

FREEZE(arg_str)
lets a programmer suspend execution of a Snobol4 program, and then lately re-load it and re-commence execution, with all the variables-set intact and the same I/O devices of the original execution. The argument *arg_str* is a string which is a user text (not containing escape characters) that is printed both in the freeze banner and also in the reprise banner, to help identifying the two phases as one. See the dedicated chapter. *Berkeley Standard**.

FRETURN
is a system label. As such, it cannot be replicated and cannot be used in other places than into a programmer-defined procedure, to signal the return point with failure. *Standard*.

IF(arg_pat)
evaluates the arguments in *arg_pat*, and if the evaluation succeeds, it returns nothing, even if a value of any type is found; if the evaluation fails, *IF()* fails silently, without generating errors. This procedure works like a safe test procedure. The same principle applies to either expressions returning values which can similarly be converted into test procedures[30]. *IF()* is the Berkeley version of the *?* operator. *Berkeley Standard**.

INPUT
receives the next input line from the current input channel; if the input line is shorter than the input dimension of 2047 characters (plus the terminator), the input line is filled with spaces. Assignment to *INPUT* is ineffective, because the next invocation would set it to the next input line, resetting the assignment. If the current input channel is exhausted, *INPUT* is set to the empty string and the clause fails. *Standard*.

INPUT(var_nam, file_str[, len_int, echo_int])
INPUT(var_str, file_str[, len_int, echo_int])
associates a variable in a Snobol4 program with an input file. The first argument *var_nam* (if as Name) or *var_str* (if in String form) is the name of a variable to be used in the program; the second argument *file_str* specifies a scope fileset (in Berkely's jargon, it is a file name or a label); the third optional argument *len_int* specifies the number of characters to be read from each record on the file. (Excess characters are lost; missing characters are filled out with spaces.) The fourth optional argument sets the echoing in case it is from keyboard, disabling echoed if *echo_int* is zero, enabling it otherwise (the default state is to echo the input)[31]. If the variable is already associated with a file, it loses its previous association. If *file_str* is equal to the string `"KYB"`, the keyboard is set the input source. *Berkeley Standard+*.

ITEM(arg_arr, arg1_int, ... argn_int)
provides another method of referring to the items of an array when the array has been assigned to a variable of type String, not strictly subject to the identifiers naming rules. See the dedicated chapter. *Berkeley Standard*.

LEFT(arg_pat)
evaluates the pattern argument *arg_pat*, which is an immediate assignment of the form `X $ Y` or a conditional assignment of the form `X . Y` (where `X` is a pattern and `Y` is a variable name), and returns the (left) pattern element. *Berkeley Standard*.

---

[30] The original Berkeley *IF()* procedure had a wider definition: it was said to always succeed, but if "*...any part of that evaluation fails, that failure causes failure of the rule*". After thinking over this subject, I can tell that *IF()* always succeeds in evaluating its argument, but it returns the state (failure of success) that comes from the evaluation, causing the failure or success of the entire rule. This is a useful technique for testing a particular value, such as `ARR[N+1]` with `N` varying. Assigning `N = N + 1` may be safe or not, and accessing `ARR[N]` with `N` greater than the maximum index may mean a memory leak. In this regard, even Gaskins and Gould use the (famous) example:

```
N = IF(ARR[N+1]) N + 1
```

for the usage of *IF()* as a test procedure.
In any case, I'm not sure this is the real and attested behaviour of the Berkeley *IF()* procedure.

[31] See also the *ECHO()* command.

NEXTVAR(arg_nam)
NEXTVAR(arg_str)
accepts as its argument either the Name of a created variable (*arg_nam*), or the Name or String (*arg_str*) naming a natural variable. For created variables - array items or fields of Data structures - *NEXTVAR()* returns as Name the next member of the same family; for Arrays, names of items are returned in the order obtained by varying the rightmost index; for Data structures, names of fields are returned in left to right order of their appearance in the *DATA()* declaration which predefined the datatype; for natural variables, *NEXTVAR()* names are returned following the declaration order, empty variables included. In all cases, the order is cyclical: the name of the first member of a family under this definition follows the last member of the same family. See the dedicated chapter. *Berkeley Standard*.

NRETURN
is a system label. As such, it cannot be replicated and cannot be used in other places than into a programmer-defined procedure, to signal the return point, with success; the return value is in the form of a Name result. *Standard*.

OUTPUT
inherits the type and value of the assignment result, and redirects the meaningful output to the current output channel. *Standard*.

OUTPUT(var_nam, file_str[, app_str, app_int])
OUTPUT(var_str, file_str[, app_str, app_int])
associates a variable in a Snobol4 program with an output file. The first argument *var_nam* (if as Name) or *var_str* (if in String form) is the name of a variable to be used in the program; the second argument *file_str* specifies a scope *fileset* (in Berkeley's jargon, it is a file name or a token); the third optional argument *app_str* is a string that is appended to the output at each invocation; the default string is the Carriage Return character `\n`; the fourth optional argument *app_int* (not belonging to the Berkeley protocol) is a parameter with the following convention: if zero (the default state), the file is erased and the writing position is set to the start of file (overwrite mode); if not zero, the file is not erased and the writing position is set to the end of file (append mode). If *var_nam* is already associated with a file, it loses its previous association. If *file_str* is equal to the string `"TTY"` or the token `TTY`, the output is directed to the screen on the standard output. If *file_str* is equal to the string `"ERR"`, the output is directed to the screen on the standard error channel. If *file_str* is equal to the string `"PRN"` or `"PUNCH"`, the output is directed to a temporary file in the file system; if the printing was previously enabled with option `-e/--enable-print`, when the program terminates, it will send all printer files to the default printer, removing also the temporary file[32]. *Berkeley Standard+*.

PARAM(arg_nam)
accepts as its argument the name of a variable instantiated through an immediate, conditional or positional assignment, following one of the ten predefined pattern procedures; it returns the argument (parameter) with which one of those was called to construct the pattern. If the pattern is one constructed by *LEN()*, *POS()*, *RPOS()*, *TAB()*, or *RTAB()*, then *PARAM()* returns an Integer; if the pattern was constructed by *ANY()*, *NOTANY()*, *SPAN()*, or *BREAK()*, then *PARAM()* returns a String; if the pattern was constructed by *ARBNO()*, then *PARAM()* returns a Pattern, or of course a String or an Integer in simpler cases. See the dedicated chapter. *Standard-*.

RANDOMIZE(dummy_val)
selects a new seed for the generation of the pseudo-random numbers returned by *RANDOM()*. See the dedicated chapter. *Not Standard*.

REMARK(arg_str)
writes its argument string *arg_str* onto the special file which is the job log. Obvious uses are to preserve messages about the course of execution associated with timing information, and to decorate the day-file. E.g.:

```
REMARK('ENTERING FREEZE TO TAPE20.')
REMARK('ANNOUNCING THE BIRTH OF JOHN'S SON!')
```

The job log is the file `.soft_log.txt` in the directory where the user is operating. If the user wants to direct messages on the default output channel, the `-j/--job-to-output` option can be used, and in this case no record is written on the job file. If the user wants to use another file as the job log, the `--joblog=<file>` option will set `<file>` (possibly with path) as the current job log; in this case, using a shared resource located in the file system, many soft users can use the same job log and share the log messages for a communitarian experience, while

[32] The temporary file is created in the current directory with the following syntax: name of the associated variable, followed by two underscore characters, followed by `print_out`, followed by the channel number, with extension `.dat`; e.g. for the first invocation of *OUTPUT()* on variable `PRINT`, the created output file would be `PRINT__print_out2.dat`.

developing Snobol4 programs[33]! *Berkeley Standard+*.
Note: REMARK() is also a command of the debugger, so the programmer can records events on the job log, for example for marking one program's point, recording a thought for later intervention or asking something to someone in the soft*'s community, all through the job file.*

RENAME(old_str[, new_str])
looks for the scope file identified by the first String argument *old_str* and gives it the new name contained in the second String argument *new_str*. In case the original file is not found, or if the renaming cannot take place (for example because of missing rights), the procedure fails. If the String argument *new_str* is the empty String or is null, or is not specified, the file identified by *old_str* is deleted. *Not Standard*.

REST(arg_pat)
evaluates the pattern argument *arg_pat*, which is an alternation of the form `X | Y | Z` or a concatenation of the form `X Y Z` (where `X`, `Y` and `Z` are proper patterns or pattern variables), and return all but the first element of the pattern, losing the operator that separates the first and the rest of elements (the *blank space* for the concatenation, and the vertical bar `|` for the alternation. *Berkeley Standard*.

RETURN
is a system label. As such, it cannot be replicated and cannot be used in other places than into a programmer-defined procedure, to signal the return point, with success. *Standard*.

REWIND(arg_str)
performs a standard rewind on the resource input or output file whose name is specified by its argument string *arg_str*; the next read or write will respectively take place at the beginning of the file; if the last operation on this file was a write, an end-of-group mark of level zero is written before the file is rewound. Existing variable associations to the channel are unaffected. *REWIND()* fails if the channel is bound to the standard input and output (resp. `stdin` and `stdoud`), and breaks if no matching *arg_str* opened file was found. *REWIND()* always returns the empty string. *Berkeley Standard*.

RIGHT(arg_pat)
RIGHT(arg_nam)
is a procedure that has three fields of application. If *arg_pat* is a pattern which represents an immediate assignment of the form `X $ Y` or a conditional assignment of the form `X . Y` (where `X` is a pattern and `Y` is a variable name), it returns the variable name as a Name object (this is the complementary procedure of *LEFT()*), though this last returns a Pattern and not a Name. If *arg_pat* is a pattern which is of the form `*VAR`, it returns the variable name as a Name object. If *arg_nam* is a Name object, it returns the name of the object as a String object. *Berkeley Standard*.

SCREEN
is a system variable that inherits the type and value of the assignment, and redirect its content to the screen, independently of the current output channel. *Not Standard*.

SEEK(scope_str[, offset_int])
SEEK(scope_str[, offset_str])
sets the file pointer for the channel identified by *scope_str*, previously established by the *INPUT()* or *OUTPUT()* procedures. The new position, measured in bytes, is obtained by adding or subtracting *offset_int* bytes to the current position of the file. If a String is specified as the second argument *offset_str*, and this is equal to `"START"`, the file pointer is set to the start of file, and if equal to `"END"`, the file pointer is set to the end of file. *SEEK()* returns as Integer the current position of the file pointer if the second argument is zero or missing, otherwise it returns the updated position of the file pointer. See the dedicated chapter. *Not Standard*.

SELECTOR(arg_nam)
accepts as its argument the Name of a created variable, and returns a String which may be used to select that variable in its family. For Arrays, *SELECTOR()* returns a string which is a list of indices; for Data structures, *SELECTOR()* returns a string naming a field selection. The String returned by *SELECTOR()* is appropriate for use as the first argument of *APPLY()*, or a second argument of *ITEM()*. *Berkeley Standard*.

STLIMIT(arg_int)
sets the limit on the number of statements executed, iterations included, checking the value of *STCOUNT()*. The

[33] Illusions fill up one's life!

initial value of *STLIMIT()* is 1,000,000[34]; lower or higher limits may be set by the programmer by calling *STLIMIT()* with a non-negative integer argument. An execution-time error results if the value set by *STLIMIT()* is exceeded. If called with a null argument, *STLIMIT()* returns its current value and remains unchanged. If called with zero as argument, no further instructions are performed and the program stops. If called with a negative argument, no control is done upon the value of *STCOUNT()* and the execution continues indefinitely. *Berkeley Standard*.
Note: &STLIMIT is the Bell correspondent keyword of STLIMIT(), and can be used in read/write-mode also in soft.

TERMINAL
is a system variable that is instantiated with the next input line from the keyboard, independently of the current input channel; if the typed input line is shorter than the input dimension of 2047 characters (plus the terminator), the input line is filled with spaces. Assignment to *TERMINAL* is ineffective, because the next invocation would set it to the next input line, resetting the assignment. If the user types Enter on an empty line, *TERMINAL* is set to the empty string, but the clause succeeds; if the user types CTRL-D, it is set to the empty string and the clause fails. *Not Standard*.

TRACE(var_nam)
adds the variable whose name is in the argument *var_nam* to the traced variables list; whenever this variable is altered, a string appears marking the timing signature and showing the new content of the variable. *Bell Standard**.

TRIM(arg_str)
returns a string which is the same as its argument *arg_str*, but shorn of the trailing blanks (that is, the blanks that are appended to the end of *arg_str*; the blanks appearing at start are considered part of the string)[35]. *Standard*.

UNTRACE(var_nam),
STOPTR(var_nam),
remove the variable whose name is in the argument *var_nam* from the traced variables list. The two syntaxes perform the same task. *Bell Standard**.

[34] The same limit in the Berkeley Snobol4.

[35] With reference to the Bell version, it is not necessary to use the procedure *TRIM()* to delete trailing blanks from input program, since this operation is performed automatically according to the settings in the system variable *&TRIM*. So, from the historical point of view, the procedure *TRIM()* was more useful for the Berkeley version rather than the Bell version.

4.5. String procedures

The following procedures return in all contexts a String value.

ALPHABET(dummy_str)
returns the soft characters set from ASCII 1 to ASCII 254[36]. The argument, if given, is anyway ignored. The ASCII character at position zero is actually the ASCII 255 because, in C, the value zero would terminate the string. I warn that characters in the range ASCII 1 to ASCII 31 are quite unmanageable (apart from the `\t`, `\n`, `\r` and `\v`), and characters from 128 to 254 are not portable throughout all Operating Systems. *Berkeley Standard+*.
Note: &ALPHABET is the correspondent Bell keyword of ALPHABET(), and can be used in read-mode also in soft.

CHAR(arg_int)
returns a one-character string containing the *arg_int*-th character in the ASCII table in the range 1..255; if beyond this range, the modulo 256 of the argument is used. *Not Standard*.

CLOCK(arg_str)
returns an eight-character string representing the time of day at which the job is being run, in the form `HH:MM:SS`. Hours (`HH`) are counted from zero through twenty-three, minutes (`MM`) and seconds (`SS`) are counted from zero through fifty-nine. All numbers have a leading zero if lower than 10. If the argument *arg_str* is the string `'H'` or `'HOUR'`, the returned value is the hours count; if it's `'M'` or `'MIN'`, the returned value is the minutes count; if it's `'S'` or `'SEC'`, the returned value is the seconds count. Otherwise, the Berkeley clock time format is returned. *Berkeley Standard+*.

DATATYPE(arg_val)
returns the string of characters which is the name of the datatype of its argument *arg_val* (among the predefined types). It is used for controlling branching, and can be used with *IDENT()* to simulate other test procedures. An exhaustive listing of the returned strings is:

```
STRING      INTEGER     REAL        PATTERN
ARRAY       NAME        CODE        <constructor>
```

The type Data is never returned because in this case the name of the constructor of the used *DATA()* command is returned. *Standard*.

DATE(arg_str)
returns a nine-character string representing the current date, in the form `DD MMM YY`; the abbreviations for the month name is the first three letters of its name; the day has a leading zero in case it's lower than 10, and the year reports the last two digits (e.g. `2021` is `21`). If the argument *arg_str* is the string `'Y'`, the last two digits of the current year is returned; if it's `'YYYY'` or `'YEAR'`, the four-digit year is returned; if it's `'M'`, the zero-padded month number is returned; if it's `'MMM'` or `'MONTH'`, the returned value is the month name (first three characters); if it's `'D'` or `'DAY'`, the zero-padded day number is returned; if it's `'0'` or the Integer zero, the Spitbol format 'date-plus-hour' is returned in American format (month first), in a 17-character String (e.g. `03/14/22 18:31:46`); if it's `'1'` or the Integer one, the Spitbol format 'date-plus-hour' is returned in European format (day first), in a 17-character String (e.g. `14/03/22 18:31:46`). Otherwise, the Berkeley date format is returned. *Berkeley Standard+*.

DUPL(arg_str, arg_int)
replicates the string contained in the first argument *arg_str* as many times as what is specified by the second argument *arg_int*. *Bell Standard*.

LPAD(arg1_str, arg_int[, arg2_str])
performs a left-padding for columnar output. It returns the string *arg1_str* padded on its left end until the total size is *arg_int* characters. The pad character is the first character of the string *arg2_str*, if present, otherwise a blank (ASCII character 32) is used if it is absent or null. If *arg_int* is less than or equal to the length of *arg1_str*, the latter is returned unchanged. *Not Standard*.

PROTOTYPE(arg_arr)
PROTOTYPE(arg_dat)
PROTOTYPE(arg_pat)
PROTOTYPE(arg_nam)

[36] The Bell and Spitbol alphabets were formed by the whole ASCII table set, included the zero value at the start. The Berkeley alphabet, instead, as stated on page 140 of the GGM, included only 63 letters (presumably the 26 capital letters, the 10 digits plus a bunch of symbols). This is a minor deviation (an enhancement, actually) from the Berkeley protocol.

returns as its value a String representing the system definition of the Object which is the value of its argument. Its operation depends on the datatype of its argument. In each case, the string returned is convenient for investigation by Snobol pattern-matching[37]. See the dedicated chapter. *Berkeley Standard+*.

PUT(arg_val)
returns as a String type the evaluated result of the argument *arg_val*, which may be of any type. If *arg_val* is any variable, its content is returned as String; otherwise, it is evaluated as a result of an assignment, and returned as String as well. It also removes the surrounding parentheses of a pattern. See the dedicated chapter. *Not Standard*.

REPLACE(base_str, chr1_str, chr2_str)
returns the string in *base_str* with its original text, except that each occurrence of a character appearing in *chr1_str* is replaced by the corresponding character in *chr2_str*. If *chr1_str* and *chr2_str* have different length, it breaks and an error message is printed. *Not Standard*.

RPAD(arg1_str, arg_int[, arg2_str])
performs a right-padding for columnar output. It returns the string *arg1_str* padded on its right end until the total size is *arg_int* characters. The pad character is the first character of *arg2_str*, if present, otherwise a blank (ASCII character 32) is used if it is absent or null. If *arg_int* is less than or equal to the length of *arg1_str*, the latter is returned unchanged. *Not Standard*.

TYPE(arg_val)
returns the same result as *DATATYPE()* for objects of predefined types, and the string `DATA` for objects of programmer-defined types. Thus, an exhaustive listing of the returned strings is:

```
STRING      INTEGER     REAL        PATTERN
ARRAY       NAME        CODE        DATA
```

The constructor name does not appear here, where `DATA` is returned for all such objects. *Berkeley Standard*.

[37] It's worth noting here that the Bell version of *PROTOTYPE()* (as described in the GB, pages 120-121) is limited to return an Array prototype only.

4.6. Mathematical procedures

The following procedures return in all contexts a numeric value of Integer or Real type.

ABS(arg_num)
returns the absolute value of the number *arg_num* given as argument. *Not Standard*.

ACOS(arg_num)
ARCCOS(arg_num)
return the principal value of the arccosine of the number *arg_num* given as argument, that must respect the condition $-1 \le arg\_num \le 1$. The two syntaxes perform the same task. The Real result R is in radians, in the range $0 \le R \le \pi$. *Not Standard*.

ASIN(arg_num)
ARCSIN(arg_num)
return the principal value of the arcsine of the number *arg_num* given as argument, that must respect the condition $-1 \le arg\_num \le 1$. The two syntaxes perform the same task. The Real result R is in radians, in the range $-\frac{\pi}{2} \le R \le \frac{\pi}{2}$. *Not Standard*.

ATAN(arg_num)
ARCTAN(arg_num)
return the principal value of the arctangent of the number *arg_num* given as argument which belongs to $\Re$. The two syntaxes perform the same task. The Real result R is in radians, in the range $-\frac{\pi}{2} \le R \le \frac{\pi}{2}$. *Not Standard*.

CHOP(arg_num)
truncates the fractional part of the Real argument *arg_num*; the returned value is of type Real. *Not Standard*.

COS(arg_num)
returns the Real cosine of the radian value of the argument *arg_num*. *Not Standard*.

EORLEVEL(scope_str)
tests to see whether the fileset named by its String argument *scope_str* is positioned at an end-of-group mark; if so, it returns the level associated with the mark; the value returned by *EORLEVEL()* is the second parameter of the *ENDGROUP()* which wrote the mark, 0 to 15 inclusive. If no end-of-group mark is found, or the channel is bound to a system I/O channel, the value returned by *EOI()*[38] is -2. If the fileset is positioned at the end-of-information point (the case when the *EOI()* procedure would succeed) the value returned by *EORLEVEL()* is -1. *Berkeley Standard+*.

EXP(arg_num)
returns the Real exponential result of the numerical value *arg_num*. *Not Standard*.

FILESIZE(scope_str)
returns the Integer dimensions in bytes of the fileset named by the String argument *scope_str*. The fileset does not need to be previously opened with *INPUT()* or *OUTPUT()* and may be located anywhere, provided the entire path is given. *Not Standard*.

FNCLEVEL(dummy_val)
returns an Integer value to indicate the level of evaluation of nested or recursive user procedure calls. Its use is to provide a trace of the evaluation for debugging of program logic, or to preserve a record of the level of evaluation, causing a failure during execution. (At an execution-time error, this information is displayed by the system's error message.) *Berkeley Standard*.
Note: &FNCLEVEL is the Bell correspondent keyword of FNCLEVEL(), and can be used in read-mode also in soft.

LOG(arg_num),
LN(arg_num),
return the Real natural logarithm of the value in the argument *arg_num*. The two syntaxes perform the same task. *Not Standard*.

[38] This is an enhacement with respect to the Berkeley version.

MAXLNGTH(dummy_int)
returns the current string dimension as Integer type[39]. *Berkeley Standard-*.
Note: &MAXLNGTH is the Bell correspondent feature of MAXLNGTH(), and can be used in read-mode also in soft*; it can also be assigned, but without effect*.

ORD(arg_str)
returns the Integer ASCII value of the first character in the argument string *arg_str*. The character is searched in the whole ASCII table in the range 0..255; this may return different results in different systems. *Bell Standard*.

PI(dummy_val)
returns the Greek PI value 3.14159265358979323846; the argument *dummy_val* is anyway ignored. See also *&PI*. *Not Standard*.

RANDOM(arg1_num[, arg2_num])
returns a random number in a prefixed format according to the argument *arg1_num*, following these rules:

- if *arg1_num* is a negative Integer and no *arg2_num* is given, it returns a Real number in the interval]0,1[, limits not included; e.g. `RANDOM(-1);`
- if *arg1_num* is the Integer zero and no *arg2_num* is given, it returns an Integer number in the interval [1,32768], limits included; e.g. `RANDOM(0)`
- if *arg1_num* is a positive Integer and no *arg2_num* is given, it returns an Integer number in the interval [1,100], limits included. e.g. `RANDOM(1);`
- if both *arg1_num* and *arg2_num* are given, in case they are Real limits, *RANDOM()* returns a Real number in the interval]*arg1_num*,*arg2_num*[, limits not included; e.g. `RANDOM(-2.5,2.5);` in case they are two Integer limits, *RANDOM()* returns an Integer in the interval [*arg1_num*,*arg2_num*], limits included; e.g. `RANDOM(7,15).`

The random series is not automatically randomized, thus it returns the same series in successive equal sessions. To randomize, use the procedure *RANDOMIZE()*. See the dedicated chapter. *Not Standard*.

REMDR(arg1_int, arg2_int)
returns the Integer remainder of the division of *arg1_int* by *arg2_int*; in case *arg2_int* is zero, it breaks. *Bell Standard*.

ROOT(base_num, index_num)
calculates the *index_num*-th root of the number *base_num*. Special cases are taken in account if *base_num* is negative and *index_num* is Real, or *base_num* is null and *index_num* is also null (an error message appears), and if *base_num* is null, where zero is returned. A correct Real value is returned in case *base_num* is negative and *index_num* is of Integer type (or a Real with null decimals). *Not Standard*.

SIN(arg_num)
returns the Real sine of the radian value in the argument *arg_num*. *Not Standard*.

SIZE(arg_str)
returns the Integer length of the string *arg_str*. *Standard*.

SQRT(arg_num)
SQR(arg_num)
return the Real square root of the argument *arg_num*. In case *arg_num* is negative, it breaks. The two syntaxes perform the same task. *Not Standard*.

STCOUNT(dummy_val)
returns the Integer count of the number of statements executed so far. Since it is a 32-bit quantity, executing more than 2.147.483.647 statements will cause this value to overflow. No harm results, but its value will be a large negative number (this happens only if STLIMIT() was disabled setting it to a negative integer). *Berkeley Standard*.

[39] The Berkeley version was used also to assign a value; the definition was: *"MAXLNGTH() is used to set the limit on the length of strings which may be formed, in characters. Its initial value is 131.070: lower limits may be set by a programmer by calling MAXLNGTH() with a non-null integer argument* [...] *If called with a null argument, MAXLNGTH() returns its current value and is unchanged."*. soft hard-wires the string dimension to speed-up the execution, and the value cannot be changed. So the *MAXLNGTH()* procedure in soft is without effect (the argument is ignored). This a (minor) deviation from the Berkeley version.

Note: &STCOUNT is the Bell correspondent keyword of STCOUNT(), and can be used in read-mode also in soft.

TAN(arg_num)
returns the Real tangent of the radian value in the argument *arg_num*. *Not Standard*.

TIME(dummy_val)
returns the elapsed number of milliseconds from the start of the program execution, as an Integer value. *Berkeley Standard*.

4.7. Bitwise and logical procedures

The following procedures work in two modes: if arguments are of Integer type, they return in all contexts an Integer value as a result of the bitwise evaluation of the arguments, and behave as Integer functions. If all arguments are regular Snobol code enclosed in strings, they behave like a test procedure, in all context, for a success or failure final state of the Snobol clause, much like any test procedure.

If the second argument is omitted, it is set as zero (in case of bitwise evaluation) or the empty String (in case of logical evaluation, in which case it sets the failure state).

Each function in the following is accompanied by a truth table, where *T* stands for *true* and *F* stands for *false*; in case of the bitwise evaluation, the table shows what happens to the result bit considering the same bit position in A (first argument) and B (second argument); in case of the logical evaluation, the table shoes what happens when the two tests are compared.

Not all procedures are symmetric, and the symmetry is specified for each definition.

These functions are added as a homage to Fasbol, an advanced Snobol4 version.

AND(arg1_int, arg2_int)
AND(arg1_str, arg2_str)
returns the *and* result comparing the two arguments. It follows the "both true" rule. This procedure is symmetric. *Not Standard*.
It proceeds according to the following truth table:

A	B	A AND B
T	T	T
T	F	F
F	T	F
F	F	F

EQV(arg1_int, arg2_int)
EQV(arg1_str, arg2_str)
returns the *equivalence* result comparing the two arguments. This procedure is symmetric. *Not Standard*.
It proceeds according to the following truth table:

A	B	A EQV B
T	T	T
T	F	F
F	T	F
F	F	T

IMP(arg1_int, arg2_int)
IMP(arg1_str arg2_str)
returns the *implication* result comparing the two arguments. This procedure is not symmetric. *Not Standard*.
It proceeds according to the following truth table:

A	B	A IMP B
T	T	T
T	F	F
F	T	T
F	F	T

This function, in logical terms, is difficult to understand, because it does not proceed comparing the two arguments, but asserts the final truth value basing on different principles. See the chapter "Understanding the bitwise and logical procedures" in the "Understanding... what?" chapter.

NOT(arg_int)
NOT(arg_str)
returns the *not* result of the argument, inverting the bit states of the Integer argument *arg_num* (the width of the number is 32 bits, with 31 bits for the number value and the last bit for the sign), or the final state of the logical test *arg_str*. *Not Standard*.
It proceeds according to the following truth table:

A	NOT A
T	F
F	T

OR(arg1_int, arg2_int)
returns the *inclusive or* result comparing the two arguments. It follows the *at least one* rule. This procedure is symmetric. *Not Standard*.
It proceeds according to the following truth table:

A	B	A OR B
T	T	T
T	F	T
F	T	T
F	F	F

XOR(arg1_int, arg2_int)
returns the *exclusive or* result comparing the two arguments. This procedure is symmetric. It follows the *either one or the other, not both* rule. *Not Standard*.
It proceeds according to the following truth table:

A	B	A XOR B
T	T	F
T	F	T
F	T	T
F	F	F

Other procedures, like *NOR()* or *NAND()* are not implemented, but are easily emulated by `NOT(OR(...))` or `NOT(AND(...))`, without any loss of generality.

Besides, the following bitwise procedures are added, which work with Integers only and return an Integer result:

LSHIFT(arg1_int, arg2_int)
SHL(arg1_int, arg2_int)
operates a left shifting of the bits in *arg1_int* by the number of steps specified by *arg2_int* as an unsigned entity; it is equivalent to the mathematical operation $arg1\_int \times 2^{arg2\_int}$. The created bit positions are filled with zeros. The width of the number is 32 bits, and it is possible that the number overflows. *arg2_int* is calculated modulo 32. *Not Standard*.

RSHIFT(arg1_int, arg2_int)
SHR(arg1_int, arg2_int)
operates a right shifting of the bits in *arg1_int* by the number of steps specified by *arg2_int* as an unsigned entity; it is equivalent to the mathematical operation $\frac{arg1\_int}{2^{arg2\_int}}$. The width of the number is 32 bits. No overflow is possible, because the result flattens to zero. *arg2_int* is calculated modulo 32. *Not Standard*.

LROLL(arg1_int, arg2_int)
ROL(arg1_int, arg2_int)
operates a left rolling of the bits in *arg1_int* by the number of steps specified by *arg2_int* as an unsigned entity. The width of the number is 32 bits. The sign is retained. *arg2_int* is calculated modulo 32. *Not Standard*.

RROLL(arg1_int, arg2_int)
ROR(arg1_int, arg2_int)
operates a right rolling of the bits in *arg1_int* by the number of steps specified by *arg2_int*as an unsigned entity. The width of the number is 32 bits. The sign is retained. *arg2_int* is calculated modulo 32. *Not Standard*.

See also the chapter "Understanding the bitwise and logical procedures" in the "Understanding... what?" chapter, where all functions are explained and exemplified, along with some personal notes.

4.8. Bell keywords

As said in previous parts of this manual, soft recognizes a number of keywords, in order to perform common tasks. These keyword were not available in the Berkeley version, but I implemented them in soft to avoid the uneasy task to change or comment all the system variable references in a foreign program, and also because they are useful.

All these keywords are introduced by the operator `&` (ampersand), directly followed by the variable name.

4.8.1. Protected keywords

These keywords are protected[40]. Assignment to any of the following keywords causes an error. They can only be used in patterns and they can be assigned to natural variables.
Not all of these keywords come from the Bell tradition: some come from other Snobol4 environments, or are designed by myself. The usual scheme for *Standard* is used here.

&ABORT
contains the string `"&ABORT"`, the default behaviour for *ABORT*. The return value is Pattern. *Bell Standard*.

&ALPHABET
contains the available character set (from ASCII 1 to ASCII 255). The return value is String. *Bell Standard-*.

&ARB
contains the string `"&ARB"`, the default behaviour for *ARB*. The return value is Pattern. *Bell Standard*.

&BAL
contains the string `"&BAL"`, the default behaviour for *BAL*. The return value is Pattern. *Bell Standard*.

&DIGITS
contains the numeric digits in the range `[0..9]` as a String. It's a Vanilla Snobol4 heritage. *Not Standard*.

&ERRLINE
contains the line number where the last error which caused a failure and not a break did occur. At start, it contains zero. The return value is Integer. *&ERRLINE* is rewritten anytime a new error is found (to the new error code), and is reset when *ERRTEXT()* is invoked. *Not Standard*.

&ERRTYPE
contains the code of the last error that caused a failure and not a break. At start, it contains zero. The return value is Integer. *&ERRTYPE* is rewritten anytime a new error is found (to the new error code), and is reset when *ERRTEXT()* is invoked. *Bell Standard*.

&FAIL
contains the string `"&FAIL"`, the default behaviour for *FAIL*. The return value is Pattern. *Bell Standard*.

&FENCE
contains the string `"&FENCE"`, the default behaviour for *FENCE*. The return value is Pattern. *Bell Standard*.

&FILE
contains the name of the current file, as a String object. *Not Standard*.

&FNCLEVEL
contains the Integer value of the programmer-defined procedures execution level so far; each time such a procedure is started, *&FNCLEVEL* is incremented, and it is decremented when the procedure is terminated. At start, it contains zero. *Bell Standard*.

&LASTNO
contains as Integer the line number of the last completed statement executed. The number refers to the actual and real program line number (that is, comments and empty lines are counted)[41]. *Bell Standard**.

[40] It's worth noting that these keywords do not exist in soft in the same memory space of the user and system variables. They are only containers for registers of the system's memory, containers that are not in control of the programmer. The dumping with argument 3 or 4 will show all these system variables in a separate view.

[41] This may differ from other environments, where the count includes only the executable lines (comments and empty lines not included), but I

&LCASE
contains the alphabetic letters in the range `a..z` (lower letters) as a String. It's a Vanilla Snobol4 heritage. *Not Standard*.

&LINESNO
contains as Integer the total lines number of the current program, comments and empty lines included. *Not Standard*.

&MEMLIMIT
contains as Integer the available number of program lines (15000 in total for this version). *Not Standard*.

&PARM
contains the shell command line String that appears after the file name, to communicate with the program. This can be useful to transmit to the program any data directly from the console (it's a Vanilla Snobol4 heritage). *Not Standard*.

&PI
contains the Greek PI value 3.14159265358979323846 as a Real number. It's a Vanilla Snobol4 heritage. *Not Standard*.

&REM
contains the string `"&REM"`, the default behaviour for *REM*. The return value is Pattern. *Bell Standard*.

&RTNTYPE
contains the String value of the last return type in a user programmer-defined procedure, that is it contains `"RETURN"`, `"NRETURN"` or `"FRETURN"` accordingly; otherwise, it is the empty String. *Bell Standard*.

&SPACE
returns a String composed by the space (ASCII 32) and the tab character (ASCII 7). *Not Standard*.

&STCOUNT
returns as Integer the number of statements executed so far. At start, it contains zero. There is a Berkeley equivalent in the procedure *STCOUNT(dummy_val)*. *Bell Standard*.

&STFCOUNT
contains as Integer the number of statements that have failed so far. At start, it contains zero. *Bell Standard*.

&STNO
contains as Integer the line number of the statement currently being executed. The line number is the physical line number of the program text (to permit easy checks). Notice that in other old Snobol compilers, this number does not include commented or empty lines, but I thinks this is not a good technique nowadays (see also the note for *&LASTNO*). *Bell Standard*.

&UCASE
contains the alphabetic letters in the range `A..Z` (upper letters) as a String. It's a Vanilla Snobol4 heritage. *Not Standard*.

Much of the information brought by these keywords is the evolving version of the values shown by the statistic feature. In any case, **they are not guaranteed to bear the same effects as the Bell version**. Actually, in some cases, having no practical counterpart, I managed to yield some general info.

4.8.2. Unprotected keywords

These keywords are not protected[42], but in some cases remember there is a Berkeley procedure correspondence. The following keywords are all numeric registers, and if they are assigned anything than a number, this will be evaluated as zero.

&ABEND = arg_int
contains the value of the abort-end procedure, which can be changed. At start, it contains zero, which corresponds to

find that in this way it becomes quite hard finding the line reference in a long listing.

[42] Again, these keywords do not exist in the same memory space of the user and system variables.

normal ending; if set to 1, which is a request for an abnormal ending, a system core dump is given following the program termination[43] when the program terminates. The string that appears is written by your O.S., not by soft. *Bell Standard*.

&ANCHOR = arg_int
contains the value of the anchor mode, which can be changed. At start, it contains zero (anchor mode disabled). *Bell Standard*.

&CODE = arg_int
contains the value to be returned by soft when exiting. At start it is zero (that in Unix jargon means 'exit with success'). *Bell Standard*.

&DUMP = arg_num
contains the value for the dumping. At start, it contains zero (dumping disabled). *Bell Standard**.

&FTRACE = arg_int
contains the value of the number of user-function trace outputs permitted with the current execution. If *arg_int* is set to zero, no user-function tracing occurs. At start it is zero.

&FULLSCAN = arg_int
contains the value for the quick or full scan mode. At present in soft this variable can be read and assigned, but the system does not change the scan mode, which remains in full scan mode. *Not Standard*.

&INPUT = arg_int
contains the value for the input mode. At start it contains one (input enabled). If set to zero, the input is disabled. *Bell Standard*.

&MAXLNGTH = arg_int
contains the string length. At present in soft this variable can be read and assigned, but the system does not change the string length, which remains constant. *Bell Standard*.

&OUTPUT = arg_int
contains the value for the output mode. At start it contains one (output enabled). If set to zero, no output is performed. *Bell Standard*.

&REAL = arg_int
contains the value for displaying real numbers in real or exponential notation. At start it contains 1 (real mode enabled). If set to zero, the exponential mode is activated, e.g. 123,45 is printed as 1.2345+E02. The exponent is always in two (or more) characters, with a leading zero in case the exponent is lesser than ten; the sign of the exponent is always explicit[44]. *Not Standard*.

&STLIMIT = arg_int
contains the limit of execution. The initial value of *&STLIMIT* is 1,000,000. To disable the control of the number of statements, it must be set to a negative Integer; e.g. `&STLIMIT = -1`. *Bell Standard*.

&TRACE = arg_int
contains the value of the number of trace outputs permitted with the current execution. If *arg_int* is set to zero, no tracing occurs. At start it is zero.

&TRIM = arg_int
contains the value for trimming the input data implicitly. At start is contains 0 (trimming disabled). If set to 1, the trimming is enabled, making the usage of the procedure *TRIM()* superfluous. *Bell Standard*.

---

[43] Technically speaking, the normal ending is performed by the `exit()` procedure; if an abnormal ending is required, the `abort()` procedure will be used, which terminates the program and also dumps the memory, for subsequent analysis; this process is performed by the gcc, not by soft.

[44] The normal system subroutine prints numbers with the least digits after the dot, in decimal format, and if no decimal are present, print the integer part only (followed by the dot in case the type is Real). If the value cannot fit the decimal precision of 15 digits, the system automatically reverts to exponential mode, regardless of the state of *&REAL*.

The keyword *&ERRLIMIT* in the Bell version is not implemented in soft.

4.9. The procedures prototypes

Here follows the prototypes list of all procedures that accept a value and/or return a value; in the following, if a procedure has multiple argument values or multiple return values, they are all listed in different schemes.

Notes:

- The synopses report all the arguments, but in many cases not all are required; please refer to the previous parts of this chapter for this subject.
- The type Number may be either Integer or Real
- The word *any* identifies more that one data type, possibly all.
- The words *null value* identify the empty value of type String.
- The ellipsis ... means a variable number of items.

&ABEND	Returns: Integer
&ABORT	Returns: Pattern
&ALPHABET	Returns: String
&ANCHOR	Returns: Integer
&ARB	Returns: Pattern
&BAL	Returns: Pattern
&CODE	Returns: Integer
&DIGITS	Returns: String
&DUMP	Returns: Integer
&ERRLINE	Returns: Integer
&ERRTYPE	Returns: Integer
&FAIL	Returns: Pattern
&FENCE	Returns: Pattern
&FILE	Returns: String
&FNCLEVEL	Returns: Integer
&FTRACE	Returns: Integer
&FULLSCAN	Returns: Integer
&INPUT	Returns: Integer
&LASTNO	Returns: Integer
&LCASE	Returns: String
&LINESNO	Returns: Integer
&MAXLNGTH	Returns: Integer
&MEMLIMIT	Returns: Integer

&OUTPUT	Returns: Integer
&PARM	Returns: String or any
&PI	Returns: Real
&REAL	Returns: Integer
&REM	Returns: Pattern
&RTNTYPE	Returns: String
&SPACE	Returns: String
&STCOUNT	Returns: Integer
&STLIMIT	Returns: Integer
&STNO	Returns: Integer
&TRACE	Returns: Integer
&TRIM	Returns: Integer
&UCASE	Returns: String
ABORT	Returns: null value
ABS(Integer) ABS(Real)	Returns: Integer Returns: Real
ACOS(Number)	Returns: Real
ALPHABET(String)	Returns: String
ANCHOR(String)	Returns: null value
AND(Integer, Integer) AND(String, String)	Returns: Integer Returns: null value, or fails
ANY(String)	Returns: String
APPLY(String, any, ..., any)	Returns: value, or fails
ARB	Returns: String
ARBNO(Pattern)	Returns: String
ARCCOS(Number)	Returns: Real
ARCSIN(Number)	Returns: Real
ARCTAN(Number)	Returns: Real
ARRAY(String, any)	Returns: Array
ASIN(Number)	Returns: Real
ATAN(Number)	Returns: Real

BACKSPACE(String)	Returns: null value, or fails
BAL	Returns: String
BREAK(String)	Returns: String
CHAR(Integer)	Returns: String
CHOP(Real)	Returns: Real
CLEAR(String)	Returns: null value
CLOCK(String)	Returns: String
CODE(String)	Returns: Code
CONVERT(String)	Returns: Real
CONVERT(Integer)	Returns: Real
CONVERT(Real)	Returns: String
CONVERT(any, String)	Returns: any
COPY(any)	Returns: any
COS(Real)	Returns: Real
DATA(String)	Returns: null value
DATATYPE(any)	Returns: String
DATE(String)	Returns: String
DEBUG(String)	Returns: null value
DEFINE(String, String)	Returns: null value
DETACH(Name)	Returns: null value
DETACH(String)	Returns: null value
DIFFER(any,any)	Returns: null value, or fails
DUMP(String)	Returns: null value
DUPL(String, Integer)	Returns: String
ECHO(Name, Integer)	Returns: null value
ECHO(String, Integer)	Returns: null value
ENDFILE(String)	Returns: null value or fails
ENDGROUP(String, Integer)	Returns: null value
EOI(String)	Returns: null value, or fails
EORLEVEL(String)	Returns: Integer, or fails
EQ(Real, Real)	Returns: null value or fails
EQ(Integer, Integer)	Returns: null value or fails
EQV(Integer, Integer)	Returns: Integer

EQV(String, String)	Returns: null value, or fails
ERRTEXT(Integer,String)	Returns: String
EVAL((String, String, ..., String)	Returns: Integer or Real
EXP(Number)	Returns: Real
FAIL	Returns: null value and fails
FAMILY(Name)	Returns: Array or Data
FENCE	Returns: null value, or fails
FILE(String)	Returns: null value, or fails
FILESIZE(String)	Returns: Integer
FIRST(Pattern)	Returns: Pattern
FNCLEVEL(any)	Returns: Integer
FORWARD(String)	Returns: null value or fails
FREEZE(String)	Returns: null value
GE(Integer, Integer)	Returns: null value, or fails
GE(Real, Real)	Returns: null value, or fails
GT(Number, Number)	Returns: null value, or fails
IDENT(any, any)	Returns: null value, or fails
IF(any)	Returns: null value
IMP(Integer, Integer)	Returns: Integer
IMP(String, String)	Returns: null value, or fails
INPUT	Returns: String
INPUT(Name, String, String, Integer)	Returns: null value
INPUT(String, String, String, Integer)	Returns: null value
INTEGER(any)	Returns: null value or fails
ITEM(Array, Number, ..., Number)	Returns: any, or fails
ITEM(Array, String, ..., String)	Returns: any, or fails
LE(Integer, Integer)	Returns: null value, or fails
LE(Real, Real)	Returns: null value, or fails
LEFT(Pattern)	Returns: Pattern
LEN(Integer)	Returns: String
LGT(String,String)	Returns: null value, or fails
LN(Real)	Returns: Real

LOG(Real)	Returns: Real
LPAD(String, Integer, String)	Returns: String
LSHIFT(Integer, Integer)	Returns: Integer
LT(Number, Number)	Returns: null value, or fails
MAXLNGTH(Integer)	Returns: Integer
NE(Integer, Integer)	Returns: null value, or fails
NE(Real, Real)	Returns: null value, or fails
NEXTVAR(Name)	Returns: Name
NEXTVAR(String)	Returns: Name
NOT(Integer)	Returns: Integer
NOT(String)	Returns: null value, or fails
NOTANY(String)	Returns: String
OR(Integer, Integer)	Returns: Integer
OR(String, String)	Returns: null value or fails
ORD(String)	Returns: Integer
OUTPUT	Returns: null value
OUTPUT(Name, String, String, Integer)	Returns: null value
OUTPUT(String, String, String, Integer)	Returns: null value
PARAM(Pattern)	Returns: Pattern, String, or Integer
PI(any)	Returns: Real
POS(Integer)	Returns: null value, or fails
PROTOTYPE(Array)	Returns: String
PROTOTYPE(Data)	Returns: String
PROTOTYPE(Name)	Returns: String
PROTOTYPE(Pattern)	Returns: String
PUT(any)	Returns: String
RANDOM(Number, Number)	Returns: Integer or Real
RANDOMIZE(any)	Returns: null value
REM	Returns: String
REMARK(String)	Returns: null value
REMDR(Integer, Integer)	Returns: Integer
RENAME(String, String)	Returns: null value, or fails
REPLACE(String, String, String)	Returns: String
REST(Pattern)	Returns: Pattern

REWIND(String)	Returns: null value
RIGHT(Name)	Returns: Name
RIGHT(Pattern)	Returns: String
ROOT(Real, Real)	Returns: Real
ROOT(Real, Integer)	Returns: Real
RPAD(String, Integer, String)	Returns: String
RPOS(Integer)	Returns: null value, or fails
RSHIFT(Integer, Integer)	Returns: Integer
RTAB(Integer)	Returns: null value, or fails
SCREEN	Returns: null value
SEEK(String, Integer)	Returns: Integer
SEEK(String, String)	Returns: Integer
SELECTOR(Name)	Returns: String
SHL(Integer, Integer)	Returns: Integer
SHR(Integer, Integer)	Returns: Integer
SIN(Real)	Returns: Real
SIZE(String)	Returns: Integer
SPAN(String)	Returns: String
SQRT(Real)	Returns: String
STCOUNT(any)	Returns: Integer
STLIMIT(Integer)	Returns: Integer
STOPTR(Name)	Returns: null value
TAB(Integer)	Returns: null value, or fails
TAN(Real)	Returns: Real
TERMINAL	Returns: String
TIME(String)	Returns: Integer
TRACE(Name)	Returns: null value
TRIM(String)	Returns: String
TYPE(any)	Returns: String
UNTRACE(Name)	Returns: null value
XOR(Integer, Integer)	Returns: Integer
XOR(String, String)	Returns: null value, or fails

5. THE DEVIL IS IN THE DETAILS...

Designing Snobol required a lot of work, mainly for the conception and the project or the structures needed to accomplish the various programming tasks. (The design of soft does not strictly follow the SIL Macro Implementation guidelines).

I therefore decided to put in this chapter some news about these features, in order to explain also the internal mechanics of soft. These notes are not meant to teach the matter, they are just a simple chat about the implemented features.

5.1. The pre-analysis phase

When a program is loaded in memory, the program code is deconstructed and made more computer-friendly. It passes the following steps (pre-analysis phase):

- Each line is loaded from the program file; each line must be at most 2047 characters wide (plus the terminator); in case it's longer, the input may be buffered, with the risk of breaking lines into unwanted points. In case you know that some line is longer, break it.
- Any continuation line (starting with + or `.`) is appended to the previous line, resizing its dedicated memory and leaving the current line empty. The total appended line should of course not be longer than 2047 characters (plus the terminator).
- The semicolon (out of strings) is parsed and, if found, the parts in between are separated and stored in consecutive lines, each with its own label and go-to section, where specified. This eases the program execution, and is reflected on the listings that are executed after the program completion.
- Each label is stored in the labels list, giving it a numeric code associated to the line number, for a fast execution-time referencing; labels are then removed from the program line.
- Comments in the program are discarded.
- The first `END` label found terminates the process of loading; this preserves further info, that may be the content of the `INPUT` routines for option `--resources/-r`. If no `END` label is found, the loading goes on until the last line in the file.

5.2. The execution phase

The process of analysis of a single line (execution phase) proceeds along the following path:

- The execution starts always from the first non-null program line. As in the Bell Snobol4, it is possible to specify by use of an *END* statement which statement of the program is to be executed first (something that the Berkeley Snobol4 could not do), by writing this label's name after `END`, separated by at least one blank.
- The next program line is copied into a local buffer, where it passes the unveiling step (the variables in the line are converted to their contents); this is especially useful because system variables (e.g. `ARB`) are converted to their executable tokens (e.g. `&ARB`), or pattern variables of any kind are made explicit. The local buffer is also checked against unbalanced brackets of the type `(..)` and `[..]`.
- The buffer is analyzed and the first element is retrieved; if it is a variable, its numeric code is also retrieved. If the element begins with a parenthesis, the whole content between the opening and closing parenthesis (included) is taken as the first element.
- If the first element is followed by '=' it is an assignment; in this case the first element ***must be a variable***, (that is, its numeric code is not null) to hold the assignment content, which is gathered afterwards; in the course of the content retrieving, the final datatype of the argument is determined and also assigned to the variable. Mathematical expressions and strings concatenations are resolved before the assignment.
- If the first element is not followed by '=', it is a pattern evaluation; in this case, the first element is the *base string*, and what follows is taken as the patterns token to be matched against the base string. If a match is found, it succeeds; if a match is not found or a failure command is issued, the match fails (and a rematch may happen).
- If a successful pattern evaluation is followed by '=', the matching pattern in the first element is replaced by what follows the '=' symbol, and attributed to the variable, and in this case the base element ***must be a variable*** (that is, again, its numeric code is not null).

- After any process (assignment or pattern evaluation), succeeding or not, if there is a go-to part, this is examined and processed, setting the next execution line, otherwise the next non-empty line is processed.
- When the last line is examined and processed, or a jump to the `END` label is performed, the execution stops unconditionally.

If the user wishes to break a program execution before its natural end, for example in case of an infinite loop, the CTRL-C must be used. In this case, two kinds of messages appear; if run from console, the message

```
soft: CTRL-C pressed. Execution terminated.
```

In case of PIE, the message becomes:

```
soft: CTRL-C pressed. Execution terminated; PIE is restored.
```

Even if the default input and output channels are flushed, before terminating the program execution, it is possible that some character is output *after* the CTRL-C message. This does not mean any problem, it's just a feature that i cannot remove, because it depends on the Operating System.

5.3. Operators

In Snobol4, there are many unary and binary operators that govern the string world, the output, the mathematical expressions and even assignments.

Associativity for binary operators is defined as follows: An operation þ on a set S is said to be *Associative* or satisfy the Associativity Property if, for all $a,b,c \in S$, it follows that $a \text{ þ } (b \text{ þ } c) = (a \text{ þ } b) \text{ þ } c$.

−> Operators +, − and `*` are Associative, since for instance `2 + 3 + 5` is equivalent to `(2 + 3) + 5` and also to `2 + (3 + 5)`, giving the same result `10`.

A Non-Associative operator can maintain the associativity in one direction only, resulting in a Left- or Right-Associativity. An operator þ is said to be *Left-Associative* if $a \text{ þ } b \text{ þ } c = (a \text{ þ } b) \text{ þ } c$, and it is said to be *Right-Associative* if $a \text{ þ } b \text{ þ } c = a \text{ þ } (b \text{ þ } c)$.

−> The division operators `/` and `\` are Left-Associative; the case `50 / 5 / 2.5` is equivalent to `(50 / 5) / 2.5 = 4.` and not to `50 / (5 / 2.5) = 25.` as you may easily see. In mathematical formulas, $x/y/z$ is equivalent to $\frac{x/y}{z}$.

−> The exponential operators `^`, `!` and `**` are instead Right-Associative, and thus `2 ** 3 ** 3` is equivalent to `2 ** (3 ** 3) = 134217728`, and not to `(2 ** 3) ** 3 = 8 ** 3 = 512`. This is different from other languages that treat the exponential operator as Left-Associative (for instance BASIC). In mathematical formulas, the Right-Associativity of the exponential may be written as $x^{y^z} = x^{(y^z)}$ and this bears some sense, because it definitely matches the visual representation of the exponential function, while $(x^y)^z$ (applying the Left-Associativity) does not.

In writing your expressions, use as many parentheses as you need to resolve all possible ambiguities in correctly evaluating the mathematical meaning you want to build, if you feel uncertain about the associativity properties involved.

The following tables lists the mathematical operators understood by soft, with the convention that the *no space* feature means that the unary operator is attached to the following item; the *plus space* feature means that the unary operator is obligatorily detached from the following item.

Char	Name	Definition
$	*dollar*	Indirect assignment (plus space)
$	*dollar*	Value reference (no space)
+	*plus*	Positive value (no space, facultative)
−	*minus*	Negative value (no space)
.	*dot*	Indirect assignment on success (plus space)
.	*dot*	Name reference (no space)
@	*at*	Indirect assignment of cursor position (no space)

Table of Unary operators

Char	Name	Definition	Associativity
!	*exclamation mark*	Exponentiation	Right-Associative
*	*asterisk*	Multiplication	Associative
**	*double asterisk*	Exponentiation	Right-Associative
+	*plus*	Addition	Associative
−	*minus*	Subtraction	Associative
/	*slash*	Division	Left-Associative
=	*equal*	Assignment	No
\	*backslash*	Integer division	Left-Associative
^	*caret*	Exponentiation	Right-Associative
|	*vertical bar*	Alternation	Associative

Table of Binary operators

Note: the Berkeley version (and thus soft) does not feature the *OPSYN()* procedure to build new entities as synonyms for existing procedures or operators.

Note: The = sign for assignments does not strictly need to be surrounded by spaces, as was the Berkeley Snobol4 (page 176 of the GGM). Of course I suggest you to write it always surrounded by spaces, because other environments could be more adherent to the Snobol4 protocol and you make your program portable. Anyway, soft won't complain if you occasionally omit such spaces.

Note: for what about the mathematical operators, they instead, even if occasionally they do not need to be surrounded by spaces, they should, above all if parentheses or in case of mixed Strings and Mathematical Expressions; In any case, as for the = sign, I suggest you to write them **always surrounded by spaces**. This was required by the Berkeley version, anyway.

5.4. Precedence of operators

Operators do not have the same precedence. A classic examples is

```
OUTPUT = 2 + 3 * 4
```

which returns 14 and not 20, because this is equivalent to

```
OUTPUT = 2 + (2 * 4)
```

having the multiplication a higher priority, in math, than addition. In the following the table of precedences are shown for the mathematical operators of soft:

Operator	Precedence	Notes
unary -	5	evaluated immediately
** ! ^	4	exponentiation
( )	3	parenthesis calculus
* / \	2	multiplication and division
+ and binary -	1	addition and subtraction

Table of Operators Precedence

In the pattern evaluation there are two other operators that must be taken in account: the operators `$` for immediate assignment and `.` (the dot) for the conditional assignment. These operators have a higher priority with respect to the mathematical operators, but lower than parentheses. An evaluation like this:

```
X = "THE WEIGHT IS "
A = 34
B = 24

"IN THIS CASE THE WEIGHT IS 58" X A + B $ OUTPUT
```

prints `24` and is equivalent to the request:

```
"IN THIS CASE THE WEIGHT IS 58" X A + (B $ OUTPUT)
```

because the operator `$` has higher priority than the mathematical +. To request the complete math expression, use parentheses:

```
"IN THIS CASE THE WEIGHT IS 58" X (A + B) $ OUTPUT
```

that will print `58` instead of `24` or, to request the complete pattern, use one more parenthesis level:

```
"IN THIS CASE THE WEIGHT IS 58" (X (A + B)) $ OUTPUT
```

that will print `"THE WEIGHT IS 58"`.

Incidentally, I must note that the line without parentheses is not interpreted by current Snobol4 and Spitbol interpreters, that fail in the management of `X` followed by a numerical variable; this is because they join `X` (which is a String) and `A` (which is an Integer) *before* considering the +, and then try to add an Integer to a String (which is not a pure number); thus, they break with an error message.

soft, instead, considers mathematical expressions as a whole, and they are evaluated instantly. If you want to separate the calculus, and fall back to the classic Snobol4 behaviour, use parentheses as much as needed, because they don't interfere with pattern evaluations. See also the next chapter.

5.5. Integer mathematics

Any mathematical expression involving only Integer numbers uses an integer rendition of the Real number that is used to calculate the expression and returns an Integer result. For instance:

```
OUTPUT = 2 ** 31 - 1
```

returns 2147483647, which is a correct Integer rendition of the number. But this cannot be extended to include the 64-bit Integer representation. In particular, the Real intermediate result can host a *real* Integer (in terms of integer representation) up to the limit 2^{49}, which is about 5,62949953421e+14; for greater Integers, the exponential representation prevails, even if the type remains Integer.

This is due to the algebraic expressions evaluator, which deals with Reals, and is used also to evaluate the Integer expressions.

I think this is not a real problem for Snobol, and anyway, soft can master 32-bit Integer representations, but I know modern Snobol and Spitbol interpreters handle full 64-bit Integer representations, which is something I have to

aspire to in the future.

5.6. Real mathematics

Any mathematical expression involving real procedures, though not a main issue in Snobol, is taken with care in soft. Mathematical calculations return accurate values, and all special cases of dominion holes are considered.

For instance, the *SQRT()* procedure returns correct values as long as the argument is positive or null. In case it's negative, there are no results in $\mathfrak{R}$ - the *real numbers* dominion - and the solution must be searched in $\mathfrak{R} + \mathfrak{I}$ - the *complex numbers* dominion - which is out of scope here.

Analogously, the *LOG()* procedure returns correct values as long as the argument is positive, because the logarithmic function is not defined for arguments which are negative or null.

In case values should make the mathematical procedure fail, the following error message appears:

```
REAL ARITHMETIC OVERFLOW.
```

This message wants to underline the fact that the result lies out of $\mathfrak{R}$.

The type of the result is the type of the first numeric element in the expression; look at the examples:

```
$ output = ln(45.67)/ln(2)
5.51317488460363
```

The following expression calculates the logarithm in base 2 of the number 45.67. So, we expect that the following expression holds:

```
$ output = 2**5.51317488460363
45
```

The type of the result is the type of 2, which is an integer, and so the final result (perform in the real domain) is trimmed at the final assignment. If you want to obtain the real result, you have to turn 2 in decimal form:

```
$ output = 2.0**5.51317488460363
45.67
```

5.7. Assigning natural and created variables

Assigning a natural variable to another variable means creating a mere **copy** of the original variable. The following little program shows this copy process:

```
* ASSIGN A
        A = 3
* COPY A TO B
        B = A
* REASSIGN A
        A = 5
* PRINT RESULTS: A CHANGED, BUT B DIDN'T
        OUTPUT = "A = " A
        OUTPUT = "B = " B
END
```

Its execution produces, as expected:

```
        A = 5
        B = 3
```

because `A` and `B` are two distinct elements.

Instead, assigning a *created* variable to another variable creates a **reference** to the original variable, rather than a

copy. For instance, see the following code snippet:

```
* ASSIGN A
        A = ARRAY(4)
* REFERENCE A TO B
        B = A
* REASSIGN AN ELEMENT OF B
        B[2] = 24
        OUTPUT = A[2]
* THE SAME ELEMENT OF A WAS REASSIGNED AS WELL
END
```

In this example, `B` is a *reference* of `A`, not a mere copy. An effect of this property is that altering the second element of `B` means also altering the second element of the array `A`, because both `A` and `B` refer to the same memory area. Both variables have type Array, but while `A` has a proper array structure reference in it, `B` hasn't, and rather *points* to A. The output of this program is of course:

```
24
```

Keep this in mind when you assign created variables.

5.8. The Directives

The directives managed by the Berkeley version were the following (see page 156 of the GGM): `-SPACE`, `-EJECT`, `-UNLIST` and `-LIST`.

For soft I decided to increase the number of directives because they are useful, bringing some inspiration from the Spitbol family, plus some ideas from mine. They stand alone on a line and are not followed by regular rule or go-to sections, because they are evaluated immediately.

The available directives in soft are:

-COPY
a synonym for `-INCLUDE`, provided for back-compatibility with Spitbol compilers.

-EJECT
Print the page number line at the bottom of the current page and leaves a form feed in the listing[45]. What follows stands on top of the next page. It works only on lines for which `-LIST` is active. Not to be confused with `-FORMAT`.

-FAIL
-NOFAIL
When the `-FAIL` mode is set (the default), a failure in a statement without a go-to is ignored, and execution continues with the next statement in sequence. This can lead to undetected errors, especially when array subscripts are out of bounds, or there are pattern matches which fail when you were sure they would always succeed. Including a `-NOFAIL` in your program changes this. If a statement without a go-to fails, you'll get an execution error message. `-FAIL` and `-NOFAIL` can be intermixed in the same program; statements will be compiled in the mode currently in effect.

-FORMAT
On a single page, print all but two lines of listing plus one empty line plus one line of paging (with filename, username and page number)[46]. This produces a textual file that can be printed of a pdf with page numbering.

---

[45] The number of lines in a page is contained in the constant EJPAGE, in `soft.h`, and it is currently 60. Change it and recompile if you have a printer with a different value.

[46] See the note for `-EJECT`.

-INCLUDE 'file'
Compile program code from the specified text file, that must contain valid Snobol4 code. The file name must be enclosed in single or double quotes. When the End-of-File is encountered, compilation resumes with the line following the `-INCLUDE` statement, which is not included in the listing.

-LIST LEFT
Activates the numbered listing for the current line and all subsequent; the listing produces statement numbers and level index at the left-end of each program line (as does, in a different fashion, the `-n/--numbered-list` option); any not null specifier different from `LEFT` will be interpreted as `RIGHT` (actually it's a 'not `LEFT`').

-LIST RIGHT
Activates the numbered listing for the current line and all subsequent; the listing produces statement numbers at the right end of each program line, eventually breaking up lines longer than the maximum width of the current dimension of the screen, minus 10 characters (which hosts the line number).

-LIST
Equivalent to `-LIST RIGHT`.

-NOEXECUTE
Prevents the execution of the program. This directive is useful if `-LIST` in any form was set, and the listing is the only required process (the execution is of no importance at this level).

-NOLIST
Prevents the listing in any form. This directive sets the flag that starts the listing to false; it is analyzed during the program loading, and thus before any execution, so it can be put in any place in the listing.

-PRAGMA <text>
Invokes console options for the current execution, basing on the argument <text> in the form of successively dashed console options separated by blanks (as if written from the console); for instance, `-PRAGMA -r --dump` would establish the resource file reading and the variable dumping, without typing options directly from console. This is useful for distributed programs, that may be simply executed without the need to know which options have to be enabled. Every token not beginning with a dash stops the options parsing. The `-PRAGMA` directive is ignored by other compilers/interpreters, so that it is fully compatible for distributed compatible programs.
If you anyway want to avoid running the `-PRAGMA` options (for your inscrutable needs), use the console option `-quell-pragma` (see the man page), which suppresses the `-PRAGMA` call and let overwrite the needed options.

-SINGLE
-DOUBLE
Listings are normally built filling each line of output. These directives allow you to double-space all or part of a program listing or to revert to the no-spaced lines; the double-space adds a blank line after each program line printing.

-SPACE[<n>]
Leaves <n> blank lines in the listing; if no argument is specified, leaves one blank line. Of course, it works only on lines for which `-LIST` is active.

-TITLE text
-STITLE text
Title and Subtitle lines for the program listing; both are processed when met, so there may be more than one instance of them, but they are subjected to the `-LIST` directive (they are not printed if `-LIST` is not active for the titles lines). The text `text` follows freely, no single or double quotes. `-TITLE` prints its text in capitalized form, `-STITLE` maintains the text as written on the program text.

-UNLIST
Disables the numbered listing for the current line and all subsequent.

5.8.1. Notes about directives

For the optional directives, soft defaults to `-LIST RIGHT`, `-SINGLE` and `-FAIL`.

The directives normally don't appear in the program listing, unless they influence the listing itself, with the

following properties: The directives `-PRAGMA` and `-NOLIST` are executed immediately during the loading phase and appear in the listing. The directives `-INCLUDE` and `-COPY` are executed immediately during the loading phase. They are substituted by the included program, and that's why they won't appear in the global listing. The directive `-SPACE` alters the listing output, but don't appear in the listing, being sunstituted by the empty lines it stands for. The directive `-FORMAT`, instead, appears in the listing. The directive `-NOEXECUTE`, which prevents the execution of the program, appears in the listing. The directives of the `-LIST` family, which activate the listing before the execution of the program, must be used in the position where you want the effect to start or end, and there can be multiple instances of them. The parts of code where the effect is set will follow the list right or left specific rule. This directives family appears in the listing, because they influence the listing itself. The listing has lines numbers and levels, so that it cannot be stored into a file and re-executed; moreover, in case of `-LIST RIGHT`, the breaking of lines longer than the available width is brutal (for instance, strings are not broken and recomposed). The `-LIST RIGHT` option should be used on wide screens (80 characters or more).

The directives `-SINGLE` and `-DOUBLE` must be used in the position where you want the effect to start or end, and there can be multiple instances of them. The parts of code where the effect is set will follow the single line/double line specific rule. This directives family appear in the listing, because they influence the listing itself.

The directives `-TITLE` and `-STITLE` must be used in the position where you want the title and/or subtitle to be, and there can be multiple instances of them. They won't be part of the final listing.

The directives `-FAIL` and `-NOFAIL` must be used in the position where you want the effect to start or end, and there can be multiple instances of them. The parts of code where the effect is set will follow the fail-on-no-jump/don't fail-on-no-jump specific rule. They won't be part of the final listing.

Don't be confused by the `-UNLIST` and `-NOLIST` directives; the first disables the listing flag for the lines that follow, should be used in conjunction with `-LIST`, and must be repeated for any list/unlist part; the second prevents the listing in any form, and it's not necessary to repeat it: one instance suffices.

5.9. The essence of the pattern variables

The six pattern variables *ABORT*, *ARB*, *BAL*, *FAIL*, *FENCE* and *REM* are special variables that contain a specific system pattern; for instance, read carefully the following example:

```
VAR = ARBNO(LEN(1))
"VACUUM CLEANER" "A" ARB . OUTPUT "ER"
"VACUUM CLEANER" "A" VAR . OUTPUT "ER"
```

Both evaluations return the string `"CUUM CLEAN"`, because *ARB* is equivalent to `ARBNO(LEN(1))`; but *ARB* does not contain the pattern `ARBNO(LEN(1))`; rather, it contains the token `&ARB` that, when invoked, performs a slightly different procedure than `ARBNO(LEN(1))` (actually, it has the same effect and is a bit more faster). The same can be said for the other system variables.

This is why I prevent by default the reassigning of these system variables, because the original behaviour could be lost and they could be misused. But if you absolutely want to reassign them, you must enable the session by means of the `--alter` option, and then assign them to whatever is your need, for instance

```
ARB = ARBNO(LEN(2))
```

The pattern variable *&ARB* contains the pattern `"&ARB"`, which is the pattern identifying the original *ARB* procedure and that can be used to restore the original *ARB* pattern. The same for the other pattern variables. So, in case you reassign *ARB* or in case you use the procedure *CLEAR()*, you can reassign the original value with

```
ARB = &ARB
```

For their implementation in soft I found them not easy to understand, but the pattern variables are very common in Snobol4 programming, and learning their usage is one of the main goals of Snobol. In particular, I underline here the scope of three of them: *FAIL*, *FENCE* and *ABORT*.

As said in a previous chapter, these three pattern variables influence the pattern evaluation result, with *FAIL* that forces a *rematch*, and *ABORT* that forces a *failure*.

These concepts are plain and need no further explanation. Anyway, the third variable *FENCE* deserves more talk, because it is not easy to understand, so that you are invited to see also the chapter "Understanding *FENCE*" in this manual.

For what about the seventh variable *SUCCEED*, not implemented, see also the chapter "Understanding the missing *SUCCEED*" in this manual.

6. ...AND THE CREATOR IN THE MATTER

The created variables are generated by special procedures that, when invoked, they not only assign a special type to the destination variable, but they also perform some tasks in order to set up internal values and structures that help in performing the behaviour of the procedure itself. They are different from the *natural* variables: where the latter are put in a common space, using consecutive indices, the created variables become special pointers to different memory spaces where the data are stored.

The present chapter describes the creation process of arrays, data structures, programmer-defined procedures and code lines, which extend the power of Snobol.

6.1. Creating ARRAY() structures

The *ARRAY(proto_str[, arg_val])* command is used to create arrays, which are a special class of variables; the values they refer to are stored elsewhere, out of them (unlike natural variables), and these 'external' objects are referred through an index (called *selector* in Berkeley Snobol4 jargon) which is specified in square brackets. Dimensions may be more than one, and indices can vary from any range, and in case of multiple dimensions, all indices are stored inside one pair of brackets, and separated by comma. If the second argument *arg_val* is given, this is the value with which all elements in the array are instantiated[47]; this value may be a string or a number.

For example, the following procedure is used to create a three-dimensional array:

```
ARR = ARRAY('0:10,5:10,-5:5')
```

This assignment creates an object `ARR` of type Array, with prototype `0:10,5:10,-5:5`; the range can vary from positive or negative lower limits, and there is virtually no limits in the dimensions number or in their width but the computer's memory[48]. The name of the array is as usual a legal name, which starts with a letter and contains only letters, digits and the dot.

A simpler form is

```
ARR = ARRAY(5)
```

This is equivalent to

```
ARR = ARRAY('1:5')
```

This is a declaration with a starting value:

```
ARR = ARRAY(5, "DONE")
```

Array elements are accessed through the following scheme:

```
<name>[<ind>]
```

where `<name>` is the name of the array, and `<ind>` is the indices list (*selectors*, in Berkeley jargon), used to select one and specific object, enclosed in square brackets and separated by commas in case of dimensions greater than one. For instance, the latter example could use:

```
ARR[3] = "VALUED"
OUTPUT = ARR[2]
```

The first example could be used as such:

```
ARR[0,5,-5] = SQR(SIN(I)) + SQR(COS(I))
OUTPUT = ARR[10,10,5]
```

As natural variables, array items may be of any basic type (Integer, Real, String or Pattern), but they are not natural

[47] Not in the original Berkeley version.

[48] The original Berkeley superimposed limits on the array creation.

variables.

6.1.1. Special problems on arrays

The GGM, on page 107, dedicates a paragraph to the so-called *special problems concerning array references*, that is the practice to use *any* name for the arrays, which leads to unexpected problems.

For instance, you can declare such arrays:

```
$'A/1' = ARRAY('10')
LIST[1] = ARRAY('20')
$WORD = ARRAY('30')
```

The second and third examples presume that

```
LIST = ARRAY(...)
WORD = <string>
```

do precede the arrays declarations, but I give it for sure.

Now, what if one item of these special arrays is to be instantiated?

```
$'A/1'[1] = "ANNA"
LIST[1][2] = 24
$WORD[3] = 4.56
```

These operations are forbidden. In a Berkeley engine, the first and second lines lead to compile-time errors, while the third passes the compilation, but then, during execution-time, it would fail in interpreting the array name; instead of `$WORD` it would take as name the figure `$WORD[3]`, but no variable has this name, and so the statement fails.

The same problems could arise if one element of these special arrays must be used:

```
OUTPUT = $'A/1'[1] " IS BEAUTIFUL"
OUTPUT = 3 * LIST[1][2]
OUTPUT = $WORD[3]
```

These operations are forbidden as well, in the Berkeley Snobol4.

The solution (ignoring the fact that using such names can lead to confusion) is to assign a reference to these *special* arrays, a reference with a legal name. The GGM suggests using the following technique:

```
TEMP1 = $'A/1'
TEMP2 = LIST[1]
TEMP3 = $WORD

TEMP1[1] = "ANNA"
TEMP2[2] = 24
TEMP3[3] = 4.56

OUTPUT = TEMP1[1] " IS BEAUTIFUL"
OUTPUT = 3 * TEMP2[2]
OUTPUT = TEMP3[3]
```

Incidentally, I must observe that, while using arrays of arrays is forbidden in the Berkeley Snobol4 (thanks Berkeley!), this is not the case of the Bell version, where the form

```
LIST[1][2]
```

is perfectly accepted and does not lead to any compile-time error. But I also must observe that using arrays of arrays, where multidimensional arrays at any level are available, is pure self-punching. I also suggest using normal names

for the arrays, to live a long and serene life.

6.1.2. The ARRAY() mechanics

Let's see in detail what happens when *ARRAY()* is used. Let's consider the following declaration:

```
LIST = ARRAY('3:4,0:10',X)
```

First of all, the assigning procedure collects the array prototype and stores it into a special register of the variable `LIST`; then it collects the instantiation value (here represented by the value of `X`). The analysis of the prototype follows, and the specific array data (dimensions, offsets and indices) are pre-calculated and stored for fast execution-time reference.

A memory zone, capable of holding all the array items in a single row, is then reserved. If an instantiation value was provided, all elements are initialized with that value, otherwise they are reset to the empty string.

The array memory is independent and separated from the natural variable space. A link to the array variable (`LIST` in the example) is made, connecting this array memory to the natural variables space.

When an element is requested, the respective indices are collected, to build a reference in the memory row where the element is stored, both for setting or reading it.

When a copy is made to one array variable, as for instance:

```
ELEM = LIST
```

there is no copy of the array memory. Rather, the variable `ELEM` is set to the type Array and is built to point to the same array memory row of `LIST`: from now on, calling an element of `ELEM` is equivalent of calling the very same element of `LIST`.

6.2. Creating DATA() structures

The *DATA(proto_str)* command is used to create specific data structured types, basing on the prototype contained in the string *proto_str*. The prototype specifies a *constructor* and one or more *methods*, in the form:

```
DATA('constructor(method1,method2,method3,...)')
```

where `method`*n* are the means used to access the values. Methods are special kind of variables in the form of a procedure that are created when the constructor is created, and that will accept as argument the name of their constructor. There is no virtual limit to the number of methods that can be associated to a constructor.

Let's use the same examples of the GB, which are illuminating. As a first example of a programmer-defined data type, consider complex numbers. A complex number consists of two numbers, one real and one imaginary. The call

```
DATA('COMPLEX(R,I)')
```

defines a data type `COMPLEX`, with two fields `R` and `I`. In this example, `COMPLEX` is the constructor, and `R` and `I` are the methods. To create an object with the data type `COMPLEX`, a call of the form *COMPLEX(e1, e2)* is made, where `e1` and `e2` are expressions that result in numerical values. For example, to assign the complex number $1.5 + 2.0i$ to the variable `C`, the statement

```
C  =  COMPLEX(1.5,2.0)
```

is executed. Each call of the constructor `COMPLEX` creates two new variables corresponding to the real and imaginary parts, initialized with the couple as arguments[49]. These variables, that are not *natural* variables, may be nonetheless referenced by using the field name as a procedure name and the constructor as argument. After executing the statement above, the value of `C` is a complex number; the real part is referenced by `R(C)` and the imaginary part by `I(C)`. Thus, with reference to the previous example,

[49] The variable created using a constructor is not a natural variable. In particular, it has no string or numeric part. It serves as a reference for getting to the content of the methods. See the next paragraph.

```
        A  =  R(C)
```

assigns the value 1.5 to A. Since R(C) is a variable, it may be assigned a value. So

```
        R(C)  =  3.2
```

assigns 3.2 to the real part of C, which results in the end as the complex number $3.2 + 2.0i$.

(You can find the source of this program in the samples/ *directory of the installation package in the file* complex.sno*).*

A more advanced example uses linked lists to read textual lines and store them into a proper structure. The structure can be predefined using the TEXT datatype:

```
        DATA('TEXT(LINE,N,NEXT)')
```

The first field is a line of text, the second field stores the number of the current line of text, and the third field points to the next line of text. A passage of text is read using the following program chunk:

```
        I = 1
        HEAD = TEXT(TRIM(INPUT),I)              :F(PRINT)
        CURRENT = HEAD
LOOP    I = I + 1
        NEXT(CURRENT) = TEXT(TRIM(INPUT),I)     :F(PRINT)
        CURRENT = NEXT(CURRENT)                 :(LOOP)

PRINT   .....
```

This piece of code initializes the linked list with HEAD, which is bound to the first input line. The variable CURRENT is set to *point* to HEAD (assignment to Data elements results in assigning a pointer to the element). Methods not specified in the constructor are implicitly initialized to the empty string.

The loop uses CURRENT to access its NEXT() part, set it a new TEXT reference, initialized with the next textual line, and stopping only when INPUT terminates the text lines.

The stored linked list is accessed using the following program chunk:

```
PRINT   CURRENT = HEAD
TEST    LINE(CURRENT) 'EVERY'                   :F(BUMP)
        OUTPUT = N(CURRENT) ':' LINE(CURRENT)
BUMP    CURRENT = NEXT(CURRENT)
        IDENT(CURRENT)                          :F(TEST)
        OUTPUT = "DONE"
END
```

Again, CURRENT is used as the pointer that traverses all the linked list, starting from HEAD, to which CURRENT is reset as first thing. Next, the line content (accessed through the method LINE()), is examined to see if it matches the string 'EVERY'. If so, it is printed. Anyway, CURRENT is set to point to the next list element, until no more elements are present, and CURRENT is equal to the empty string (it means it was not initialized).

(You can find the text of this program in the samples/ *directory of the installation package in the file* data.sno*, that contains also some text lines after the last* END*; to execute it, step into this directory from the console and type*

```
$ soft -r data.sno
```

and you will see the power of Snobol4 in action.)

6.2.1. The DATA() mechanics

Let's see in detail what *DATA()* and the constructors do. Let's consider the following *DATA()* declaration:

```
DATA('COMPLEX(R,I)')
```

When *DATA()* is invoked, it sets some inner registers saving the name, the prototype and the number of methods, under a unique code number (called *DATA item number*). This register is invisible to the user.

In this example, the name of the constructor is `COMPLEX`, the prototype is the string `R,I` and the number of methods is 2. Let's suppose the *DATA item number* is 1.

When a constructor is invoked, some special elements are created. Let's consider the following program line:

```
C = COMPLEX(1.2,-1,6)
```

This instruction first generates the mother-variable `C` of type Data, with inner reference *DATA item number* 1 (which bounds it to the type `COMPLEX`), with its own variable code (say 9). The mother-variable generated by the constructor does not use the string and numeric parts, but since the initialization must set the string content to something not null (otherwise `IDENT()` or `DIFFER()` won't be able to detect if this variable points to `NULL` or not), the string part is filled with a string which corresponds to the numeric code of the *DATA item number*. This string is a service string that is inaccessible to the user.

Then, two special method-variables are created; they are of the form `R(9)` and `I(9)` where 9 (it's just an example) is the variable code of the mother-variable. Note that whenever you try to directly access these variables, you fail to get the proper value, because these are not natural variables, though they can host strings, patterns and numbers as usual. The only way to access these methods is by using as argument the name of the mother-variable that generated them (`C` in this case)[50]:

```
R(C)
I(C)
```

Please note that this feature permits the usage of the same method name for different types, provided you address the argument to the proper mother-variable. E.g.

```
DATA('COMPLEX(R,I)')
C = COMPLEX(2,3)
DATA('SORT(R,T)')
F = SORT('EVERY',0)
OUTPUT = R(C) ': ' R(F)
```

```
2: EVERY
```

As already said in the chapter "Assigning created variables", whenever you set one generic variable to be equal to a Data variable (a mother-variable), this generic variable is not a copy of the Data variable, but rather it's a reference to it. Whenever you access this generic variable item, you also access the same data of the mother-variable. E.g.

```
DATA('COMPLEX(R,I)')
C = COMPLEX(2,3)
D = C
OUTPUT = R(D)
```

```
2
```

This is the useful feature that allows traversing a linked-list with a pointer that is not a list element *per se* (because it was not created via a constructor), a pointer that inherits all the references to treat the list element to which it points to as if it was a list itself.

[50] To tell the truth, there is another way... look at the debugger. In any case, using directly `R(9)` or `I(9)` does not work...

6.3. Creating DEFINE() user procedures

Besides the data creators Array and Data, which extend the data types that can be treated by the Snobol4 language, there is also another important procedure that extends the language, letting create programmer-defined procedures. Here it is. The definition is as follows:

```
DEFINE(proto_str[, label_str])
```

This definition is used to create a specific user procedure, basing on the prototype contained in the string *proto_str*, setting facultatively the starting point of the procedure from the label specified by the string *label_str*. The prototype specifies a *procedure name* and one or more *argument variables*, in the form:

```
DEFINE('name(var1,var2,var3,...)acc1,acc2,acc3...'[,'labelname'])
```

where `var1..3` are the names of the argument variables used to access the values, `acc1..3` are the accessory variables used in the procedure, and `labelname` is the label from where the procedure will start its execution. The number of argument or accessory variables is not fixed, and they may also be missing. Even the label name may remain undeclared. The rule in the Berkeley Snobol4 requires that no spaces exist in the string after the ')' defining the accessory variables, and soft conforms to this rule.

For what about *labelname*, there may be two different syntaxes:

- The label is explicitly declared, and in this case the body of the user procedure starts from that label;
- The label is not given, and in this case a label with the same name of the procedure is searched and set as the starting point of the procedure.
- In any case, if the label is not found, an error is raised.

All cautions must be given to avoid executing the body of the *DEFINE()* procedure, by skipping its body in the normal execution, with proper go-tos. For instance, let's see a procedure that adds three numbers:

```
* FIRST CASE WITH LABEL
        DEFINE('SUM3(A,B,C)','FUNC')
        ...
        ...
        ...                                 :(SKIP)
* THE FOLLOWING LINE IS SKIPPED
FUNC    SUM3 = A + B + C                    :(RETURN)
SKIP        ...
        OUTPUT = SUM(1,2,3)
```

`FUNC` is set as the starting label, and the body of `FUNC` is regularly skipped.

The same program, but without specifying a label:

```
* SECOND CASE WITH IMPLICIT LABEL
        DEFINE('SUM3(A,B,C)')
        ...
        ...
        ...                                 :(SKIP)
* THE FOLLOWING LINE IS SKIPPED
SUM3    SUM3 = A + B + C                    :(RETURN)
SKIP    ...
        OUTPUT = SUM(1,2,3)
```

`SUM3` is searched and found, and this will be the starting label; the body of `SUM3` is again skipped.

The accessory variables are declared in the definition of the procedure to ensure that they can be used in the body of the procedure itself without altering the value of existing variables with the same name in the main program.

The call of the procedure may happen in any place of the program text (remember that, in Berkeley Snobol4,

everything is a procedure, so that it always returns at least the empty string), but *after* the procedure declaration, of course. Arguments are always passed by value, and it's not required to specify all of them: those not specified in the list of the caller are instantiated to the empty string.

Assigning a result to a variable with the same name of the procedure, `SUM3` in the example, means fixing also the return value of the procedure itself. The type of this special variable will also be the type of the procedure. This is a feature of Snobol4, because the same procedure may return an Integer, a Real or a String in three different moments, without requiring three different procedures, each with its fixed return type.

In the body of the procedure, the end of the procedure must be signalled with a jump to one of the predefined labels `RETURN`, `FRETURN` or `NRETURN`; these labels have such a special meaning in Snobol4 that it's forbidden to use these names as labels (an error is raised)[51], and they follow these rules:

- The `RETURN` label in the procedure's body is used to signal the successful end of the procedure. The return value may be safely used in any assignment or pattern matching. This case resets the final status of the procedure to *success*, because any failure in the user-procedure is not passed back to the caller, because the procedure succeeds.
- The `FRETURN` label in the procedure's body is used to signal the failure of the procedure (the procedure will fail even if this was not explicitly derived from the procedure body). This case resets the final status of the procedure to *failure* even in case of success, because it's specified that the user-procedure fails (for example in a test procedure).
- The `NRETURN` label in the procedure's body is used to signal the successful end of the procedure, fixing also the return type to Name, with a string result that is also the name of a variable. This lets using the result value to be instantiated within an assignment. This case resets the final status of the procedure to *success* as in the `RETURN` case.

The values supplied as arguments may be of any kind, as in the following example:

```
        OUTPUT = SUM3(23.5,6.78,3.0/23.8)
```

where the printed value is

```
30.4061
```

You may have noticed that the third argument is a mathematical operation. Since values are passed by value, I extended this feature as to accept *any* number or mathematical expression whereas a number is expected[52].

When the procedure stops, variables are restored to the state before the call, the program counter is restored to the line from where it started and the execution continues.

6.3.1. Passing variables to a programmer-defined procedure

The variables passed to a *DEFINE()* procedure are generally passed as values. So that, if you have the following definition:

```
        DEFINE('CUBE(X)')              :(CUBEND)
CUBE    CUBE = X * X * X               :(RETURN)
CUBEND
```

each time you invoke the procedure for the cube calculation, you must pass a value suitable for the equation in the procedure's body, for instance:

```
        OUTPUT = CUBE(3.2)
```

The *value* 3.2 is evaluated and assigned to `X` with Real type, and this makes the equation effective: the variable `CUBE` is assigned with the result of the equation.

[51] The original Berkeley compiler, if detected outside of a user-procedure, considered such labels as *normal* usable labels, but the architecture built-in in soft prevents this kind of usage. In any case, this is a minor difference.

[52] In reality, the standard Snobol4 interpreters and compilers don't accept a mathematical operation here, because they perform a one-to-one value for the procedure's arguments. I think this is a useful enhancement for soft.

But what if you have to pass variables as entities? Take a look at this example:

```
        DEFINE('SWAP(X,Y)T')          :(SWAPEND)
SWAP    T = X
        X = Y
        Y = T                         :(RETURN)
SWAPEND
```

Whenever you invoke this procedure, the only thing you will achieve is to swap the values of the internal variables, that will be destroyed when the procedure ends, and leaving the rest unchanged:

```
        VARX = 34
        VARY = 67
        SWAP(VARX,VARY)
        OUTPUT = VARX ", " VARY
```

This problem is solved passing variables names as strings, and using referenced variables in the body of the procedure:

```
        DEFINE('SWAP(X,Y)T')          :(SWAPEND)
SWAP    T = $X
        $X = $Y
        $Y = T                        :(RETURN)
SWAPEND

        VARX = 34
        VARY = 67
        OUTPUT = VARX ", " VARY
        SWAP('VARX','VARY')
        OUTPUT = VARX ", " VARY
```

with output:

```
34, 67
67, 34
```

With the previous `SWAP()` function format, values can be passed also as Name entities: they will be evaluated through the `$` mechanism when used into the function body; see the following example:

```
        DEFINE('SWAP(X,Y)T')          :(SWAPEND)
SWAP    T = $X
        $X = $Y
        $Y = T                        :(RETURN)
SWAPEND

        LIST = ARRAY(5)
        LIST[1] = "FIRST"
        LIST[5] = "LAST"
        OUTPUT = LIST[1] ", " LIST[5]
        SWAP(.LIST[1], .LIST[5])
        OUTPUT = LIST[1] ", " LIST[5]
END
```

The output is of course:

```
FIRST, LAST
LAST, FIRST
```

6.3.2. Local variables in full

All variables that are created in the execution of a user procedure are *internal variables*. There are four types of internal variables:

- argument internal variables
- explicit internal variables
- implicit internal variables
- the service internal variable

The *argument internal variables* are special internal variables that are also the arguments of the called procedure. This means that they are created, when the procedure starts, and they are instantiated to something that is supposedly not an empty string.

Accessory variables, instead, are declared in the body of the procedure to avoid reassigning existing variables with the same name, but they are not arguments of the procedure. They are called *explicit internal variables*, because they are known in the declaration phase.

There is also another class of internal variables: those declared as new in the body of the procedure and having a different name than that of the argument or accessory variables. These variables are called *implicit internal variables*, because they are created in the body of the procedure and they cannot be known by the declaration phase, but they are nonetheless destroyed when the procedure ends.

Finally, there is also another internal variable that must be considered. It is the *result variable*, that is the variable that permits the return value of the procedure, the one with the same name of the procedure. Assigning this variable means also setting the return type of the procedure.

All types of internal variables are destroyed when the procedure ends, and the original variables set, which existed before the procedure was executed, is restored.

6.3.3. The returned values

The Berkeley and Bell version could nominally return any value[53]. This version of soft restricts return values to Integer, Real, String, Pattern, Name and Array items, but prevents returning Code items, because this can interfere with the memory in an uncontrolled way (imagine if an infinite loop is associated to such a procedure...); Data items can be returned, but since the structure built in the procedure is destroyed when the procedure terminates, this cannot be safely used in the caller environment, or associated to a possible variable.

In particular, the following must be observed:

- If Integer or Real values are set for return in a *DEFINE()* procedure, they substitute in any context the procedure with the result; for instance:

  ```
  LENGTH = SUM(SIZEA,SIZEB,SIZEC) * 3
  ```

  returns a value that is the result of `SUM()` multiplied by 3, which is assigned to `LENGH`. If found in a pattern evaluation, the result is checked against the current cursor position, as if the very same number was typed in the clause.
- If a String value is set for return in a *DEFINE()* procedure, it substitutes in any context the procedure with the result String; for instance:

  ```
  WORD = CUTOUT(BASE, "HELL") " STEPS TO HEAVEN"
  ```

  returns a string that is the result of `CUTOUT()`, added to the terminal String. In a pattern evaluation, the String is checked against the current cursor position, as if the very same String was typed on the line.
- If a Pattern value is set for return in a *DEFINE()* procedure, it is treated differently in assignments or in pattern evaluation.

[53] Not really true, since Data item cannot be returned neither in modern Bell versions, nor in Spitbol...

In an assignment, it is treated as any other pattern; for instance:

```
        PAT = BREAKX('E')
```

sets PAT to be a Pattern, with the result of BREAKX() as content; PAT can from now on be used as a pattern in any context where patterns are used.

In a pattern evaluation, the Pattern figure is substituted to the procedure, and re-evaluated, as to make the Pattern result effective; for instance:

```
        "SEE NOISES" LEN(1) BREAKX('E') "S"
```

acts in two steps: firstly, the BREAKX() procedure is invoked, which returns a Pattern value; secondly, the phrase BREAKX('E') is substituted with the Pattern value and re-executed from the position of BREAKX(); in the example, supposing that BREAKX() return BREAK('E') ARBNO(LEN(1) BREAK('E')), the process acts as if the line was:

```
        "SEE NOISES" LEN(1) BREAK('E') ARBNO(LEN(1) BREAK('E') "S"
```

without re-executing LEN(1) but restarting from the next BREAK() (the first patch of the Pattern).

- If an Array value is set for return in a *DEFINE()* procedure, it may be used to declare an array, but this is the only case when this has some meaning; other usages are not forbidden but they may cause a failure of the clause. The classic examples is:

```
        DEFINE('NEWARR()')              :(NEWARR_END)
NEWARR  NEWARR = ARRAY(5)
        NEWARR[1] = 'ANNA'
        NEWARR[5] = 24                  :(RETURN)
NEWARR_END

        F = NEWARR()
        OUTPUT = F[1]
        OUTPUT = F[5]
END
```

In this case, the procedure NEWARR() uses the inner variable NEWARR to declare an array and populate it with some scattered values. When the procedure returns, it can be used in the assignment to F, which becomes a proper array (actually a *copy* of the procedure array, which is destroyed when exiting from the procedure), and inherits also the values that were assigned in the original array; the program returns, as expected:

```
ANNA
24
```

6.3.4. The DEFINE() mechanics

Let's see in detail what happens when *DEFINE()* is used. Let's consider the following declaration:

```
        DEFINE('SUM3(A,B,C)Y','PR.SRT')    :(PR.END)
PR.SRT  Y = A + B
        SUM3 = Y + C                       :(RETURN)
PR.END

        OUTPUT = SUM3(1,3,5) END
```

When *DEFINE()* is invoked, the prototype string and, if found, the label string, are analyzed and stored in a proper structure under a unique number (called the *DEFINE item number*). This register is invisible to the user. If the label string was given, the address of the jump, for future reference is calculated and an error is raised if the label is not found. The prototype string generates two distinct registers: the argument prototype, and the accessory prototype.

In this example, the name of the procedure is SUM3, the argument prototype is A,B,C, the accessory prototype is Y,

the label prototype is `PR.SRT`,

When `SUM3` is invoked, in some other place, a number of things happen. First of all, the service variable `SUM3` is instantiated as an empty string. This will serve as the return value holder. Then, the three argument variables `A`, `B` and `C` are instantiated, but they are filled with the values given in the `SUM3` call; in the example, they are set as three integers with values `1`, `3` and `5` respectively. Finally the accessory variable `Y` is instantiated to the empty string. All variables are built queueing them to the current variables list and all previously declared variables are still available.

Then, the execution of the procedure body is invoked, starting from the line with the label `PR.SRT`. The execution terminates when the first *RETURN/FRETURN/NRETURN* is met. After the execution is done, the current value of the service variable gives the result value and type of the procedure. If no assignment was done to the service variable, the procedure acts like a no-return value procedure or, if needed, as an empty string value procedure. In the example, the service variable `SUM3` is set to hold the integer 9.

Finally, the program pointer, and the variable index are restored to the state prior to the execution, getting rid of all the internal variables, which are no more necessary. Since a call to `RETURN` is done, the procedure terminates with success.

The procedure should have at least one return point. If the `END` label is met before a return keyword, the program terminates.

6.4. Creating CODE() program sections

The *CODE(arg_str)* command accepts as argument a String which is a regular Snobol4 program text, with labels and go-tos, that is, a sequence of syntactically-correct Snobol4 statements[54], and returns as value the corresponding compiled code as type Code, which can be assigned to a natural variable whose type will also be set to Code; its use, then, is to permit a program to extend itself while it's being executing. All characters in the Snobol4 character set, including space, have their customary significance in the argument to *CODE()*.

Statement separators are semicolons, but no final semicolon is required in the string.

Since *CODE()* adds lines after the last, to avoid memory leaks, a final `END` label is introduced automatically in case it's not found in the source (this operation is revealed by the --listing option, for instance).

Let's examine an example:

```
[1]          WORD = "ANNA BALALAIKA"
[2]          OUTPUT = "START"
[3]          CODE('LOOP  WORD "A" =           :S(LOOP) ;'
[4] +'       OUTPUT = "WORD = " WORD')
[5]          OUTPUT = "WORD = " WORD      :(LOOP)
[6]          OUTPUT = "END"
[7] END
```

The code to add to the program is the following:

```
LOOP    WORD "A" =                       :S(LOOP)
       OUTPUT = "WORD = " WORD
```

The program becomes the following (when the *CODE()* line is executed):

```
[1]          WORD = "ANNA BALALAIKA"
[2]          OUTPUT = "START"
[3]          CODE('LOOP  WORD "A" =           :S(LOOP) ;'
    +'       OUTPUT = "WORD = " WORD')
[4]          OUTPUT = "WORD = " WORD      :(LOOP)
[5]          OUTPUT = "END"
[6] END
```

[54] Substantially, this means that the program in the *CODE()* argument could be a program by itself, with the only exception that it can jump to labels not available in the code piece itself, because present in the original listing where *CODE()* is contained.

```
[7] LOOP    WORD "A" =                          :S(LOOP)
[8]        OUTPUT = "WORD = " WORD
[9] END
```

This is the only case when labels may be duplicated, and in effect the final `END` in line 9 is not contained in *arg_str* but is attached by soft.

The program flow, after the `CODE` line is executed, at line 4 makes a jump to `LOOP`, which is the first line in the *CODE()* argument (line 7). The program steps to `LOOP` and executes the pattern matching, removing all 'A's from the string, and then prints the result. The output is:

```
START
WORD = ANNA BALALAIKA
WORD = NN BLLIK
```

Notice that the *CODE()* statement does not prescribe getting back to the point of the jump to resume the original flow, and this does not emerge from the Berkeley GGM but from the GB, where it's stated in par. 6.3.1. that "*...Flowing off the end of a block of compiled object code results in normal termination, just as if there were an* `END` *statement. If the label* `END` *is encountered in the string being compiled, compilation ceases at that point...*". This means that after `WORD` is printed in line 8, the program reaches the second `END` label in line 9 and terminates.

The jump technique can be used to override this behaviour. See the following example:

```
[1]          WORD = "ANNA BALALAIKA"
[2]          OUTPUT = "START"
[3]          CODE('LOOP  WORD "A" =            :S(LOOP) ;'
[4] +'       OUTPUT = "WORD = " WORD         :(PRINT)')
[5]      OUTPUT = "WORD = " WORD      :(LOOP)
[6] PRINT   OUTPUT = "END"
[7] END
```

(Notice the `:(PRINT)` go-to appended to the *CODE()* string, and to the label `PRINT` at line 6.) As in the previous case, the *CODE()* statement creates two additional lines of code, but the second line has a final go-to.

This program runs the same process of the previous example, but eventually it jumps back to label `PRINT` when the final word is printed, causing the printing of the string `"END"` at line 5; After this, the original label `END` in line 6 is met, and the program terminates after having printed:

```
START
WORD = ANNA BALALAIKA
WORD = NN BLLIK
END
```

Snobol4 provides another way to jump to a specific `CODE` section, by means of the go-to itself. The go-to

```
:[NULP]
```

encountered in a normal go-to section, makes the program flow jump to the first line of the code originated by the former assignment

```
NULP = CODE('...')
```

Of course, the variable in the `:[...]` part must be a Code variable, or an error is raised. The programmer does not need to know a label address, but just the variable which was assigned the code section.

The `:[...]` jump can be effectively operated in conjunction with `:F[...]` and `:S[...]` for making a conditioned jump in case of failure or success only, respectively.

Note: there's a limitation in using this go-to within soft: the GB[55] shows another way of setting a jump:

```
: [CODE(' OUTPUT = "RECOMPILED" :(RESTART)')]
```

This causes the CODE instruction to return a type Code, adding the line, and returning the type Code in order to make an effective jump, in substitution of the variable name of type Code. This structure is NOT currently implemented in soft; this must be re-written as:

```
STEP = CODE('  OUTPUT = "RECOMPILED" :(RESTART)')  :[STEP]
```

This workaround is even easier to read, compared to the previous line.

6.4.1. The CODE() mechanics

When an assignment like

```
VAR = CODE('...')
```

is encountered, soft analyses the argument string, counts the lines to be added to the current program text, compiles them (labels are added to the current labels list) and adds a final END label (not strictly required but it's safer).

The variable VAR is created with type Code and some inner registers are stored in it with the starting and final added code lines (the starting address is the *CODE item number*). These registers are invisible to the user. The Code variable generated by this phase does not use the string and numeric parts, but since the initialization must set the string content to something not null (otherwise IDENT() or DIFFER() won't be able to detect if this variable points to NULL or not), the string part is filled with a string which corresponds to the *CODE item number*. This string is a service string that is also inaccessible to the user.

6.5. Creating variables copies

The procedure COPY() is a very advanced procedure that not only returns a copy of the argument but it creates all necessary structures required by the copy process. Its syntax is simple:

```
A = COPY(B)
```

sets A to be not a reference to B, whatever it is, but a real clone of it, so that there are two objects with the same content. Notice also that the name of the variable B is written directly, not a Name or a String object, and this is peculiar in Snobol.

The rules are:

- If B is a Data object, A is assigned the same type and values, and all its methods are created, with the same values of the correspondent methods in B.
- If B is an Array object, A is assigned the same type and prototype of the original Array, and a new memory space is generated where the same content of the original is copied back, to assure that A treats the same objects of B.
- If B is a Code object, an error message is printed and execution stops; the copying of a Code object would add to the listing the same lines already added by B, causing memory problems.
- If B is a Name, a Pattern, a String, a Real or an Integer, A is set to be a copy of B, but in this case this is a rewriting of the simple assignment A = B, which produces the same results.

6.6. Using creator procedures

Creators procedures are those that usually perform creation of structures that are required by the process, as seen previously in this chapter; they are:

- *ARRAY()*, which reserves a memory portion for saving the array items.
- *DATA()*, which creates a data type; when later executed, this data type creates the associated methods and instantiate them with the given values.

[55] I don't know if the Berkeley version supported this extension.

- *CODE()*, which adds to the listing the Snobol4 lines contained in its argument.
- *COPY()*, when used to copy Array, Data or user-defined type objects.
-

The usual way of using such procedures is directly and alone after the equal sign; e.g.:

```
LIST = ARRAY(5)
```

When used this way, the previous procedures return a type which is given to the addressing variable (`LIST` in the example), after performing their specific task. But what if such procedures are used in different contexts? For instance, what would you expect as output in the following line?

```
OUTPUT = "THIS IS AN " ARRAY(5)
```

The normal and common behaviour is that such a construction returns the string `"[ARRAY]"`, as when required to associate to a String any Array object. But not all creators behave in the same manner; actually:

- *DATA()* is normally executed, because all the information it needs is contained in the argument, and it does not return a variable code and/or type. It always returns the empty String, that can be attached to any other output.
- *CODE()* is normally executed, because all the information it needs is contained in the argument; in this case, it does not return a variable code and/or type and returns the empty String, that can be attached to any other output.
- All Data objects return they class name (the one defined within *DATA()* clauses) as String, enclosed in square brackets (e.g. `"[COMPLEX]"`.
- *ARRAY()* returns the String `"[ARRAY]"`.
- *FAMILY()* returns the family name as String.

7. UNDERSTANDING... WHAT??

The title of this chapter is misleading: It's not that I have understood the matter and I want to teach you anything; rather, I'm not sure I've correctly understood the matter, so I report here the things I've learned, hoping in a confirmation on your part, or a more correct explanation where I fail.

Let's go.

7.1. Understanding BREAK() and SPAN()

The two procedures *BREAK()* and *SPAN()* are the most frequently used (and thus perhaps the most important) procedures in Snobol, used in practically all programs. Their comprehension is not difficult, but it required a lot of efforts in implementing them, because of their different scopes, and in the present chapter I will expose their internal mechanisms.

The procedure *SPAN()*, the most simple, adds to the pattern figure all characters that belong also to the argument string. Its main characteristic (valid also in the Bell version, in Spitbol and even in Fasbol) is that it cannot return the empty string (this means that the current position, specified by the cursor, points to a character that does not belong to the argument string, and that its pattern figure is empty), and fails in case.

The procedure *BREAK()*, instead, adds to the pattern figure all characters that do not belong to the argument string. It's a negative version of *SPAN()*, but as such it can return the empty string. This is useful because the most common figure for parsing a String line for a given `SET` of characters is the following:

```
LINE BREAK(SET) SPAN(SET) . WORD
```

Whenever `BREAK(SET)` parses all useless characters (and possibly returning the empty String if `LINE` begins with a character in `SET`), it leaves the cursor in the correct position for `SPAN(SET)`. If there are no more characters in `LINE` that also belong to `SET`, `SPAN(SET)` fails. That's why *BREAK()* can return the empty String.

Anyway, the match of *BREAK()* must always be followed by a character in the base String that do belong to the argument string, or it will fail. This is not specified in all manuals, and only the Spitbol manual throws some light to it.

A consequence of these properties is that both procedures fail in case the base string is the empty String; *SPAN()* fails because the pattern is of course empty, *BREAK()* fails because the cursor can never point to a character in the argument set.

Finally, the two procedures print an error message, and stop, in case the argument String is empty.

7.2. Understanding TAB() and RTAB()

The two procedures *TAB()* and *RTAB()* are not usual, because their engines operate in a different way, with respect to other simpler procedures that return a fixed string (like *SIZE()*, for instance) or a given match (for instance a literal string).

The best definitions of the two procedures is in the GB, here summarized: *TAB(N)* matches the number of characters from the current cursor position up to position *N* in the base string; the GB: "[...] *Stated another way, TAB(N) insures that N characters are matched by positioning the cursor to the right of the Nth character* [...]".

RTAB(N), instead, matches the number of character from the current cursor position up to, but not including, the *N*th position, counting from the last character in the base string; the GB: "[...] *RTAB(N) insures that all but N characters are matched by positioning the cursor to the left of the Nth character from the end* [...]".

For both procedures, if the argument *N* is lower than zero or greater than the maximum capacity of a string, an error message is printed and the execution stops.

The procedure *TAB(N)* fails if *N* is lower than the current cursor position (making any matching impossible), or if greater than the current base string length *L*.

The procedure *RTAB(N)* proceeds by calculating a new reference value, $R = L - N$, and this is the corresponding value that acts just like the argument in *TAB()*. It fails if *R* is lower than the current cursor position (including the

case of *N* greater than *L*, in which case *R* is negative).

The usual way for showing the characters mapping of *TAB()* and *RTAB()* in quite all Snobol manuals is depicted this way:

cursor	0 1 2 3 4 5 6 7 8
	M O U N T A I N
TAB() count	1 2 3 4 5 6 7 8
RTAB() count	8 7 6 5 4 3 2 1

TAB() starts its counting from the current cursor position (initially zero) and proceeds forward, including the number of characters specified by the argument; for instance, `TAB(2)` returns `MO` (two characters), and the scanner is then advanced by two characters, and becomes 2.

RTAB() instead starts its counting from the last character in the base string, and proceeds forward, as seen, from the current cursor position until, but not including, the character specified by the argument; for instance, imagining that `RTAB(2)` is executed after the previous `TAB(2)`, with cursor equalling 2, the characters `UNTA` are returned (the characters at the absolute positions 3, 4, 5 and 6, that are also, from the *RTAB()* point of view, all the remaining characters but those at the backward positions 1 and 2).

RTAB() has one interesting property; the call:

```
BASE LEN(2) RTAB(0)
```

attributes to `LEN(2)` the first two characters in `BASE`, and the remaining characters, until the end of `BASE` to `RTAB(0)`; it acts just like *REM*: the following line produces identical results:

```
BASE LEN(2) REM
```

and effectively, in some manuals, *REM* is explained as containing the pattern `RTAB(0)`.

7.3. Understanding POS() and RPOS()

The two procedures *POS()* and *RPOS()* do not return a pattern (they always return the empty string); they use the current cursor position and, if equal to the argument value (that follows the same rules of *TAB()*/*RTAB()* with reference to the characters mapping), they let the evaluation proceed, otherwise they cause a rematch.

They are not easy to be understood, because they are not used to increase the matches number (like all other pattern functions), but to verify the cursor position and, in this regard, they should be considered *test* procedures, like *DIFFER()*, for instance; instead, in all Snobol manuals they are classified as regular pattern procedures, and so it is in this manual.

This property is interesting; for instance, the call:

```
BASE POS(0) 'A' ARB $ OUTPUT FAIL
```

fails if `BASE` does not begin with an `'A'`, and this is an implicit anchor mode.

Consequently, a call like:

```
BASE 'A' ARB 'M' RPOS(0)
```

succeeds only if the whole match from `'A'` to `'M'` extends until the end of `BASE`. Or, more stringently:

```
BASE POS(0) 'A' ARB 'M' RPOS(0)
```

succeeds only if `BASE` starts with `'A'` and terminates with `'M'` (no substrings are allowed).

The use of *POS()* and *RPOS()* is very special, not available in other programming languages; they require a steeper learning curve. At least that was valid for me, and I'm not sure I've reached the top yet.

7.4. Understanding PARAM()

The procedure *PARAM()* is one of the most mysterious procedures in the Berkeley Snobol4, and it was neither in the Bell version nor in the Spitbol family (as far as I know).

I report here the definition in the GGM (page 131):

PARAM() accepts as its argument only a pattern returned by one of the ten predefined pattern procedures; it returns the argument (parameter) with which one of those was called to construct the pattern. If the pattern is one constructed by LEN(), POS(), RPOS(), TAB(), or RTAB(), then PARAM() returns an integer; if the pattern was constructed by ANY(), NOTANY(), SPAN(), or BREAK(), then PARAM() returns a string of characters in their standard collating sequence (the sequence defined by ALPHABET()). If the pattern was constructed by ARBNO(), then PARAM() returns the pattern that was its argument, which may of course be of datatype String or Integer in simple cases.

The key phrase is or course: "[It] *accepts as its argument only a pattern returned by* [something]". One (me!) may imagine that *PARAM()* could be used to intercept one call, as in

```
"VACUUM CLEANER" LEN(3) PARAM(LEN(5)) . OUT
```

and to obtain that `OUT` is instantiated with the integer 5. But this makes the pattern fail, rather than succeed, because the token `PARAM(...)` returns 5 even if the content `LEN(5)` is matched correctly against `"UUM C".` So `OUT` is never instantiated, because `"VAC5"` cannot match the base string.

The only solution that I could conceive for the usage of *PARAM()* is to filter the information though a variable to be successively passed to it. For instance:

```
"VACUUM CLEANER" LEN(3) LEN(5) . OUT
OUTPUT = PARAM(OUT)
```

The assignment has stored the parameter used, and when *PARAM()* is called, it recognizes that one of the ten *PARAM()* pattern procedure was used. In this way, the pattern regularly succeeds, and then *PARAM()* can recover the data that `LEN(5)` did use for the match, that is the integer 5. The reference of the previous `LEN(3)` is lost, because there is no implicit assignment after it, but the process can be repeated on more variables:

```
"VACUUM CLEANER" LEN(3) . OUT1 LEN(5) . OUT2
OUTPUT = PARAM(OUT1)
OUTPUT = PARAM(OUT2)
```

So, the two arguments are both saved and can be used later.

In case *PARAM()* is called instead with a direct pattern argument, like in the following cases :

```
VAR = POS(4)
OUTPUT = PARAM(VAR)
OUTPUT = PARAM(LEN(5))
```

the two values 4 and 5 are returned, and this seems reasonable.

If you can show me how the *PARAM()* procedure really worked, I'd be glad to change the current implementation.

7.5. Understanding PROTOTYPE()

From the GGM (page 110), I read the following definition:

The PROTOTYPE() procedure can accept as its single argument any expression whose value is of datatype Array, and returns as its value a String giving the prototype of that array. This prototype will be the same as the one specified in the call to the ARRAY() procedure, which caused the array to be created, except that the lower bound for each dimension is always explicitly

```
expressed, and the integers specifying the bounds are in canonical form (a sign retained only for
negative numbers, leading zeroes suppressed, and zero represented by the single character 0).
```

Does this state that *PROTOTYPE()* works only for the *ARRAY()* procedure, as in the Bell version? No, in Appendix A, section II.B (pages 137-139), it is shown that the *PROTOTYPE()* statement can be used also for Structures, Patterns and Names. The following is a list of the usages of *PROTOTYPE()*, in addition to the *ARRAY()* property described earlier, using the same list of the GGM, with the integrations for soft as well as my personal observations.

Array and Data structures:

```
A = ARRAY("<proto>") ; PROTOTYPE(A)       returns "<proto>"
D = DATA("<proto>") ;  PROTOTYPE(D)       returns "<proto>"
```

As stated for *PROTOTYPE()*, the prototype of an Array has always explicit lower bounds. This is not guaranteed for the Bell version or in the Spitbol family.

Code elements:

```
C = CODE("<code>") ; PROTOTYPE(C)         returns "<code>"
```

Predefined pattern variables:

```
P = ARB ;    PROTOTYPE(P)                 returns "ARB()"
P = REM ;    PROTOTYPE(P)                 returns "REM()"
P = BAL ;    PROTOTYPE(P)                 returns "BAL()"
P = FENCE ; PROTOTYPE(P)                  returns "FENCE()"
P = FAIL ;  PROTOTYPE(P)                  returns "FAIL()"
P = ABORT ; PROTOTYPE(P)                  returns "ABORT()"
```

Predefined pattern procedures
where `S` is any numeric expression or string, `P` is a pattern of any type, and `I` is the numeric result from the numeric expression `S`:

```
P = LEN(S) ;     PROTOTYPE(P)             returns "LEN(I)"
P = POS(S) ;     PROTOTYPE(P)             returns "POS(I)"
P = RPOS(S) ;    PROTOTYPE(P)             returns "RPOS(I)"
P = TAB(S) ;     PROTOTYPE(P)             returns "TAB(I)"
P = RTAB(S) ;    PROTOTYPE(P)             returns "RTAB(I)"
P = ANY(S) ;     PROTOTYPE(P)             returns "ANY(S)"
P = NOTANY(S) ; PROTOTYPE(P)              returns "NOTANY(S)"
P = SPAN(S) ;    PROTOTYPE(P)             returns "SPAN(S)"
P = BREAK(S) ;  PROTOTYPE(P)              returns "BREAK(S)"
P = ARBNO(P) ;  PROTOTYPE(P)              returns "ARBNO(P)"
```

Alternation and concatenation
where `Fi` is the first item (see *FIRST()*), `Re` is the list of the rest of the items (see *REST()*):

```
P = 'A' | 'B' | 'C' ; PROTOTYPE(P)        returns "ALT(Fi,Re)"
P = 'A' ANY(S) 'C' ;  PROTOTYPE(P)        returns "CAT(Fi,Re)"
```

Conditional assignment operators
where `Le` is the left part (see *LEFT()*, and `Ri` is the right part (see *RIGHT()*):

```
P = SPAN(S) . VAR ;  PROTOTYPE(P)         returns "PRD(Le,Ri)"
P = BREAK(S) $ VAR ; PROTOTYPE(P)         returns "DOL(Le,Ri)"
P = @VAR ;           PROTOTYPE(P)         returns "AT(Ri)"[56]
```

[56] This is a soft proper feature non present in the Berkeley version.

Deferred evaluation
where `Ri` is the right part:

```
P = *VAR ; PROTOTYPE(P)                    returns "STAR(Ri)"
```

Name structures
where `Ri` is the right part, `Fam` is the family (see *FAMILY()*) and `Sel` is the selector (see *SELECTOR()*):

```
VAR = .VOWELS ;          PROTOTYPE(VAR)    returns "INDIRECT(Ri)"
VAR = .LIST[I,J] ;       PROTOTYPE(VAR)    returns "ITEM(Fam,Sel)"
VAR = .RLINK(CURRENT) ;  PROTOTYPE(VAR)    returns "APPLY(Sel,Fam)"
```

7.6. Understanding NEXTVAR()

The command *NEXTVAR()* accepts as its argument either the Name of a created variable or the Name or String naming a natural variable, and returns the name of the next member of the same family.

From the GGM (pages 141-143), I read the following definition excerpt:

> For created variables -- array items or fields of data structures -- NEXTVAR() returns, the name of the "next" member of the same family. For Arrays, names of items are returned in the order obtained by varying the rightmost index most rapidly. For data structures, names of fields are returned in left to right order of their appearance in the DATA() declaration which defined the datatype. In both cases, the order is cyclical, the name of the "first" member of a family (under this definition) being the value of NEXTVAR() applied to the name of the "last" member. [...] The more important use of NEXTVAR() arises from the fact that it also treats the set of all natural variables as a "family", and thus when given a String or a Name which names a natural variable, NEXTVAR() returns the name of another natural variable.

What is underlined here is that there are three fields of application:

1 For Arrays, names of items are returned in the order obtained by varying the rightmost index, restarting from the lower limit when the upper limit is reached.

2 For data structures, names of fields are returned in left-to-right order of their appearance in the *DATA()* declaration which predefined the datatype.

3 For natural variables, the next variable in the declaration order is returned, including null variables; the Berkeley protocol requires at least that all variables with non-null values are included in the list, and thus soft agrees with the Berkeley protocol.

In all cases, the order is cyclical; the name of the "first" member of a family (under this definition) being the value of *NEXTVAR()* applied to the name of the "last" member. Thus, if the rule

```
LIST = ARRAY('O:2,O:2')
```

has been executed, the value of `NEXTVAR(.LIST[0,0])` is the name of the array item referred to as `LIST[0,1]`, and the value of `NEXTVAR(.LIST[2,2])` is the name of the array item referred to as `LIST[0,0]`.

Similarly, if the rules

```
DATA('INODE(LLINK,RLINK, INFO)')
CURRENT = NODE()
```

have been executed, the value of `NEXTVAR(.LLINK(CURRENT))` is the name of the field referred to as `RLINK(CURRENT)`, and the value of `NEXTVAR(.INFO(CURRENT))` is the name of the field referred to as `LLINK(CURRENT)`.

(The previous examples are taken from the GGM.)

7.6.1. NEXTVAR() in depth

The GGM reports some useful notions about *NEXTVAR()*. If a statement such as

```
NEXT = NEXTVAR(NEXT)
```

is written in a loop, then the names of all the members of the family to which the value of NEXT belongs will be returned in order; but unless the program is instructed to see where it started, the loop will be infinite. A suitable loop for going once only through the fields of a node is the following:

```
SAVE = LLINK(CURRENT)
NEXT = SAVE LOOP
<statements to process a field>
NEXT = NEXTVAR(NEXT)
IDENT(NEXT, SAVE)                    :F(LOOP)
```

NEXTVAR() is convenient for referring in turn to all the variables of an array or a data structures, but Gaskins and Gould underline that its effect can be programmed in Snobol4 using *PROTOTYPE()*, *ITEM()*, and *APPLY()* (see for instance the examples in Chapter 7 of the 1972 edition).

Another point is the following: if *NEXTVAR()* is called within the body of a programmer-defined procedure, the names returned will refer to the variables which are internal to the programmer-defined procedure.

7.7. Understanding APPLY() and ITEM()

The two procedures *APPLY()* and *ITEM()* (called *collective procedures*) use strings and names and recompose their arguments to form an equivalent Array or Data object.

APPLY() is defined as follows (page 144 of the GGM):

> APPLY() provides the only way to write procedure references for procedures chosen at execution-time. The first argument of APPLY() must be a string which names a procedure; the Snobol system calls that procedure, using as its arguments the remaining arguments of APPLY(), and observing the usual conventions for extra or missing arguments. APPLY() returns the value returned by the procedure it calls, using the same return (RETURN, NRETURN, or FRETURN). If APPLY() is used to call a field selection procedure, then its use is analogous to the use of ITEM() for item references; the Snobol system forms a field reference using the first argument as the selector and the second argument for the family, and NRETURNs the field so selected.

For instance, given

```
DATA('COMPLEX(R,I)')
C = COMPLEX(1.2,3.4)
```

the following lines produce identical results:

```
OUTPUT = R(C)
OUTPUT = APPLY('R',C)
```

The *APPLY()* procedure can be applied to any procedure, with any number of arguments; for instance:

```
APPLY('INPUT','READ','KYB',25)
APPLY('TRIM',INPUT)
```

The definition of *ITEM()* can be found on page 108 of the GGM:

> The ITEM () procedure, like the APPLY() procedure [...], has a varying number of arguments, usually one more than the number of dimensions of the array involved. The first argument must be an expression whose value is an array; the remaining arguments may be any integer-valued expressions, usually one for each dimension of the array, given in the appropriate order. ITEM()

```
returns as its value (by NRETURN) the variable specified by using its first argument to indicate
a family and its remaining arguments together to form a selector.
```

The *ITEM()* procedure in the Bell version (that complies with the Spitbol family behaviour) and that, as seen earlier, intersects *APPLY()*, has a syntax that the Berkeley version plainly accepts; the indices (*selectors*, in Berkeley jargon) could be input as digits or strings, and strings can host more that one index, also separated by commas. For instance, the two calls:

```
OUTPUT = ITEM(A,3,3,3)
OUTPUT = A[3,3,3]
```

produce two identical results, because they are equivalent; and even if some value is given as a literal string, this does not change the final result:

```
OUTPUT = ITEM(A,3,'3',3)
OUTPUT = A[3,3,3]
```

This is what the Berkeley and Bell version have in common. But the Berkeley version increases the *ITEM()* possibilities, enabling one more feature; the following calls:

```
B = '3,3,3'
OUTPUT = ITEM(A,B)
C = '3,3'
OUTPUT = ITEM(A,C,3)
```

are perfectly equivalent calls as the previous ones, but it can be seen that the indices are given in various and different ways (the latter examples cannot be run on a Bell engine...).

APPLY() and *ITEM()* are strictly bound to the meta-analysis enabled in the Berkeley version; the *FAMILY()* and *SELECTOR()* commands (and many others) can be used to decompose any item, and *APPLY* and *ITEM()* recompose them.

The pride of the Gaskins and Gould can be read on page 182: "*ITEM() has been made more flexible and more useful in the Berkeley version than it is in the Bell version*".

7.8. Understanding FAMILY()

Another procedure that presents some difficulties in its usage is *FAMILY()*. The GGM definition on page 133 is the following:

```
FAMILY() accepts as argument the Name of a created variable (array item, or field of a program-
mer-defined structure). It returns the object which is the family of variables to which the Named
variable belongs. [...] Since FAMILY() returns the Array or structure rather than the Name of the
variable whose value is the Array or structure, the value of FAMILY() is suitable for use as the
first argument of ITEM(), or a second argument of APPLY().
```

The definition is straightforward: it behaves like *ARRAY()* (if a member of an array is passed as argument) or like a *DATA()* object assignment (if a method of a Data object is passed). The following must succeed:

```
LIST = ARRAY('0:10,0:10')
A = FAMILY(.LIST[5,5])
OUTPUT = A[5,5]
```

and would set `A` as a reference to `LIST`. But it must be usable also in other situations. Let's see this example.

```
LIST = ARRAY('0:10,0:10')
LIST[5,5] = 24
OUTPUT = ITEM[LIST,5,5]
ELEMENT = .LIST[5,5]
```

```
OUTPUT = ITEM[FAMILY(ELEMENT),5,5]
```

In case of Data objects the following must succeed:

```
DATA('COMPLEX(R,I)')
C = COMPLEX(1.2,3.4)
D = FAMILY(.R(C))
OUTPUT = R(D)
```

and would set D as a reference to C. Let's see another example:

```
DATA('COMPLEX(R,I)')
C = COMPLEX(1.2,3.4)
OUTPUT = R(C)
ELEMENT = .R(C)
OUTPUT = APPLY['R',FAMILY(ELEMENT)]
```

All these example show that *FAMILY()* does not return a simple number or a simple string or whatever. It returns an object of type Array or Data, and this object is available both in assignments and as argument of other very specific procedures (namely, *ITEM()* and *APPLY()*). Basing on the previous examples, it's easy to notice that *FAMILY()* cannot be used elsewhere. A composition like:

```
LIST = ARRAY(10)
ELEMENT = .LIST[5]
OUTPUT = "VAR" FAMILY(ELEMENT)
```

would output `VARLIST`, because *FAMILY()*, other than the places where it is expected, returns the name of the family as string. But beware! The construction:

```
LIST = ARRAY(10)
ELEMENT = .LIST[5]
OUTPUT = FAMILY(ELEMENT) "VAR"
```

would fail, because it is interpreted as a *FAMILY()* assignment, but followed by something which is not a go-to. If this is precisely what you mean to get, simply make precede the assignment with an empty string:

```
OUTPUT = "" FAMILY(ELEMENT) "VAR"
```

7.9. Understanding the file procedures

The original Snobol4 (in either Berkeley or Bell version) was not file-based, or at least leaned on Operating Systems different from Unix.

So, after the implementation of all the *regular* file procedures, I realized there remained a lot of missing tasks that soft couldn't manage, like: file renaming; file deletion; setting the file pointer in detailed positions; getting some basic information about the file.

In the following I present a small survey of all the implemented procedures, not because you need to know, but because I have to put it down on paper, if you pass me the term.

7.9.1. The I/O system

Designing the I/O system was not an easy task for me. It was directly inspired from the GGM, but after a bunch of failed attempts. So, essentially for myself, I'm going to explain how it works.

As a premise, a file is opened both for writing and reading, but a channel is opened in one direction only. As an example, consider the following lines (in Berkeley format):

```
INPUT('READ','testfile.dat')
OUTPUT('WRITE','testfile.dat')
```

Both channels refer to the same file. So, the following things are possible, for instance:

- to write something to the fileset with *WRITE()*, calling *REWIND()* on the fileset and then re-read the whole data with *READ()*
- (if the file pre-exists) to read the content of a file with *READ()*, calling *REWIND()* on the fileset and then proceed to overwrite it with *WRITE()*
- treat a data that has just been processed and written with *WRITE()* on file, possibly re-elaborate it and rewrite it in replacement of the previous one, after calling *BACKSPACE()*; or using *READ()* to show the data just written, once again after calling *BACKSPACE()*.

What is important to know, in these cases, is that there may be more channels on the same file, with different I/O scopes, but the file is one, and the file pointer is the same for all channels.

7.9.1.1. Input sources

The input of a Snobol4 program has two sources:

- The keyboard, through the default input channel (using the procedure *INPUT*);
- Any files, each coordinated by a specific channel identified by a variable (using the *INPUT()* procedure).

soft enhances these features with some additional tools:

- Option `--input=<file>`, that sets the default input channel from `<file>`; this input maybe iterated, and each file is queued to the previous to assure a continuous reading;
- Option `--resource` queues what follows the final *END* in the source file (until the end of file or until the first `'#'` character found at the beginning of a line) to a temporary input file that is set as last in the input chain;
- The procedure *TERMINAL*, that accepts input always from `stdin` (the default UNIX input channel).

These tools extend the programmer's possibilities, because a program could expect some input from some specific files (through the `--input` option) and some data asked to the user can always be typed from keyboard.

The procedure *INPUT()* increases the programmer's possibilities, because the input can be set from more than one file or file series, or from the keyboard, by means of the dedicated variables. See it in detail in the dedicated section.

7.9.1.2. Output sources

The output of a Snobol4 program has two directions:

- The screen, through the default output channel (using the procedure *OUTPUT*);
- Any files, each coordinated by a specific channel identified by a variable (using the *OUTPUT()* procedure).

soft enhances these features with some additional tools:

- Option `--output=<file>` redirects the default output channel to `<file>`
- The procedure *SCREEN*, redirects the output to `stdout` (the default UNIX output channel)

These tools extend the programmer's possibilities, as can be seen in the following cases:

1. In case the `--output` option is not used, *OUTPUT* and *SCREEN* work in the very same way, redirecting the output to `stdout`; the output of the program can be thus saved to file through the shell:

   ```
   $ soft prog.sno > out.txt
   ```

   In this case, both the *OUTPUT* and the *SCREEN* text is saved to `out.txt`.

2. In case the `--output` option is used, the *OUTPUT* text is redirected to the specified file, while *SCREEN* keeps working on `stdout`. So, typing

```
$ soft --output=out.txt prog.sno
```

you will see that the *OUTPUT* text is saved to `out.txt`, while *SCREEN* keeps printing on the screen.

3. In case `--output` is used, along with a redirection, the *OUTPUT* text is saved on the specified file, while *SCREEN* is saved on the redirected file. E.g.

```
$ soft --output=out1.txt prog.sno > out2.txt
```

After this command, `out1.txt` will contain the *OUTPUT* text, and `out2.txt` will contain the *SCREEN* text.

The procedure *OUTPUT()* increases the programmer's possibilities, because the output can be directed to more than one file, or to the standard output or the standard error channels, by means of the dedicated variables. See it in detail in the dedicated section.

7.9.2. INPUT()

The procedure *INPUT()*, as stated previously on page and also on page , is useful for establishing a connection with a file or the keyboard, to accept String items that can be successively processed.

Without it, a Snobol4 program would be deaf and dumb, and all input data would necessarily be put in the source itself.

The original Berkeley *INPUT()* definition (on page 146 of the GGM) is:

```
INPUT(String, String, String)       Returns: null value
INPUT(Name, String, String)         Returns: null value

INPUT() is used to associate a variable in a Snobol4 program with an input file. The first argu-
ment is the name of a variable to be used in the program; the second argument specifies a scope
fileset; the third argument specifies the number of characters to be read from each record on the
file. (Excess characters are lost; missing characters are filled out with spaces.) If the vari-
able is already associated with a file, it loses its previous association. It is through INPUT()
-- and OUTPUT() -- procedures that the Snobol4 program establishes contact with the files system.
```

This version of soft respects the original *INPUT()* structure, but adds a fourth option; the complete prototype for *INPUT()* in soft is the following:

INPUT(Name, String, String, Integer) *Returns: null value*
INPUT(String, String, String, Integer) *Returns: null value*

The first three arguments respect the original definition; let's see them in detail.

- The name of the input variable must be given in String or Name format
- The scope fileset is: 1) the name of an input textual file, complete with path if necessary, as a String item; 2) the string `"KYB"`, that connects the input to the default input channel, the keyboard
- The number of characters that are to be read; by default it is set to 2048 (2047 for text and 1 for the terminator), but you can size your input. As for the Berkeley version, characters that exceed the imposed limit are lost. And as for the Berkeley version, if the characters read are less than the imposed limit, the string is padded with blank spaces.
- The fourth argument, not available in the Berkeley version, is the echoing flag (by default it is set to one, i.e. true). If this is evaluated to an Integer different from zero, the echoing is enabled. Otherwise it is disabled. Of course, this is meaningful (and has effect) only if the scope fileset is the keyboard, because the input from file is not typed, and thus it is already *invisible*.

7.9.3. OUTPUT()

The procedure *OUTPUT()*, as stated on page and and following, is useful for establishing a connection with a file or to the screen, to reverse the result of a process.

Without it, a Snobol4 program would not be able to express any result, making the language quite useless.

The original Berkeley *OUTPUT()* definition (also on page 146 of the GGM) is:

```
OUTPUT(String, String , String)      Returns: null value
OUTPUT(Name, String, String)         Returns: null value

OUTPUT() is used analogously to INPUT(), to associate variables in Snobol4 programs with scope
filesets which are to be used for output; The first argument is the name of a variable to be used
in the program; the second argument specifies a scope fileset; the third argument is the carriage
character which will be concatenated at the head of every record written. (If omitted, none will
be concatenated.) If the variable is already associated with a file, it loses its previous asso-
ciation.
```

This version of soft respects the original *OUTPUT()* structure, but also adds a fourth option; the complete prototype for *OUTPUT()* in soft is the following:

OUTPUT(Name, String , String, Integer) Returns: null value
OUTPUT(String, String, String, Integer) Returns: null value

The first three arguments respect the original definition; let's see them in detail.

- The name of the output variable must be given in String or Name format
- The scope fileset is: 1) the name of an output textual file, complete with path if necessary, as a String item; 2) the string `"TTY"`, that connects the output to the default output channel, the screen; 3) the string `"ERR"`, that connects the output to the default error channel, usually on the screen; 4) the string `"PRN"` or `"PUNCH"`, that establishes a printing process (see ahead)
- The appended string; The Berkeley suggest that this is either the end-of-line character or the empty String. soft will accept any String item of any length; the end-of-line character is the escape sequence `"\n"`.
- The fourth argument, not available in the Berkeley version, is the append mode flag (by default it is zero, append mode disabled); If this is evaluated to an Integer different from zero, the append mode is enabled. Otherwise it is disabled, and in this case the file is created anew, and emptied if existing.

7.9.3.1. Using OUTPUT() with printers

soft cannot send data to printer, because the modern printers do not print line by line, but sheet by sheet. So the possible printer data is collected in a temporary file that will eventually be sent to printer after the program completion. The process of printing can be accomplished in two ways:

1) The first method uses the classic Unix/Linux piping feature to send the output to printer, provided the output is sent to the default output `stdout` (using *OUTPUT* or *SCREEN*); for instance:

   ```
   $ soft prog.sno | lpr
   ```

   All the output will be collected and sent to the default printer through the bash application `lpr`.

2) If the program is someway more complicated, with input and output channels directed from and to files, to screen, from keyboard and some to printer, the process is the following:

   a) A channel, among all, must be opened for printing:

      ```
      OUTPUT(.PRINT,'PUNCH')
      ```

      A temporary file is created in the current directory, and all the printer output is directed there using the dedicated variable:

```
PRINT = "THIS IS PAPER OUTPUT"
```

b) The printer is disabled by default, so the file is retained and can be examined; it can be included is any word processor and if the printed is not needed, the process ends here.

c) When the printing is needed, provided the program returns correct results, the printer must be enabled:

```
$ soft --enable-printer prog.sno
```

The screen and file output will be executed as designed, and the printer will print the desired text. When soft ends, it takes care of removing all the temporary files.

The procedure *OUTPUT()* for printers recognized the following strings that enable the printer channel: `'PRN'` and `'PUNCH'`, with identical results; besides, in these cases, the attach parameter (third parameter of the *OUTPUT()* procedure) is automatically set to the end-of-line sequence '\n', in order to separate the printing lines.

7.9.4. BACKSPACE()

The procedure *BACKSPACE()*[57] operates on the file source set as its argument, whatever position the file pointer is pointing to, setting the file pointer one line back, assuring to go not past the start of file. It can be used to re-read a line, or to re-write a line. See at the following example:

```
* OPEN CHANNELS
        INPUT('READ','text1')
        OUTPUT('WRITE','text2')

* READ TWO LINES AND WRITE THEM DOWN
        OUTPUT = TRIM(READ)
        WRITE = OUTPUT
        OUTPUT = TRIM(READ)
        WRITE = OUTPUT

        OUTPUT = "NOW WE BACKSPACE THE INPUT;
+ WHAT WILL BE READ NEXT TIME?"
        BACKSPACE('text1')
        OUTPUT = TRIM(READ)

        OUTPUT = "NOW WE BACKSPACE THE OUTPUT; LOOK AT THE PRINTED FILE"
        BACKSPACE('text2')
        WRITE = "PALE SUNSET!"
END
```

Suppose the file `text1` contains the following:

```
THESE HORSES ARE BLACK AND WHITE
THE DOG IS BARKING ON THE SOFA
THE CAT IS SLEEPING
```

the output of the program is:

```
THESE HORSES ARE BLACK AND WHITE
THE DOG IS BARKING ON THE SOFA
NOW WE BACKSPACE THE INPUT; WHAT WILL BE READ NEXT TIME?
THE DOG IS BARKING ON THE SOFA
NOW WE BACKSPACE THE OUTPUT; LOOK AT THE PRINTED FILE
```

and the content of `text2` is:

[57] This procedure belongs to the Bell version, and was not available for the Berkeley version.

```
THESE HORSES ARE BLACK AND WHITE
PALE SUNSET!
RKING ON THE SOFA
```

Seen? The second line `THE DOG IS BARKING ON THE SOFA`, that was written to file on the first run, is backspaced and another (shorter) line is written in its place, leaving the final part of the previous line in the file.

The usage of *BACKSPACE()* can be avoided with a more modern command, like *SEEK()*, that can operate in a more precise way, moving the file pointer back and forth.

7.9.5. DETACH()

Whenever you connect a variable to an I/O source, it continues to operate in read or write mode in connection with that source. But it may happen that this source is closed by some other procedure, or that you need the variable in some other context.

A variable can be disconnected from the source using the *DETACH()* procedure. The variable itself is not removed or erased, and retains its content and type (which most of the cases is String), but it loses its I/O state, and from now on behaves just like any other variable of the same type.

There is no need to detach one I/O variable to assign it a new I/O channel, because any new assignment with *INPUT()* or *OUTPUT()* overwrites the previous I/O state and resets the file pointer.

7.9.6. ECHO()

Any input from a file is automatically hidden, because no evidence of the input is known at the execution state, unless an *OUTPUT* is used, whereas the keyboard input is there for all to see.

In cases where you want to *hide* the typing, you can either do it directly in the *INPUT()* declaration (fourth argument), or you can enable/disable the echoing later, when needed, using *ECHO()**[*].

ECHO() has two arguments, the first being the name of the input variable used in the *INPUT()* declaration, and the second is an Integer which is either zero (echo disabled) or not zero (echo enabled).

See the following example, that performs username and password getting:

```
* SET I/O channels
       INPUT('ENTER',"KYB")
       OUTPUT('WRITE',"TTY","")

* INPUT USERNAME DATA
       WRITE = "Enter username: "
       NAME = TRIM(ENTER)

* REMOVE ECHOING FROM INPUT SOURCE
       ECHO('ENTER',0)

* INPUT PASSWORD DATA
       WRITE = "Enter password: "
       PASS = TRIM(ENTER)

* OPTIONAL OUTPUT: PASSWORD DATA (KEPT HIDDEN) IS MADE EXPLICIT
       OUTPUT = "\nHello user " NAME "; you have password: " PASS
END
```

All the input is coordinated by `ENTER`, which is declared with echo enabled (suitable for the username input); then, the password is asked, that must be hidden, and so *ECHO()* is called upon `ENTER`. The result is:

```
Enter username: guiafra
```

Not present in the Bell nor in the Berkeley versions.

```
Enter password: *******
```

But of course the password was stored in PASS, so it can be also printed; the result is:

```
Hello user guiafra; you have password: ghwye34
```

Notice also that WRITE was declared with attach null, to avoid the printing of the new-line character after the Enter... lines.

7.9.7. ENDFILE()

Whenever a program has ended its duty upon a fileset, it's good norm closing it (even this is not strictly necessary because soft does it automatically at its termination).

For this, the *ENDFILE()* procedure can be used. This procedure was not in the Berkeley version, but was present in the Bell version, with argument equal to the numeric channel used for opening the /O. Since the Berkeley version made no use of the numeric channel, and since it was based of the fileset concept, I turned *ENDFILE()* to work with a fileset rather than with a numeric channels, and so *ENDFILE()* in soft accepts only a file name as argument, a file name among the ones opened with *INPUT()* or *OUTPUT()*.

It then searches all the instances of this fileset in the opened channels list (taking care of not closing the standard input and output sources), and detaches all variables connected to the same fileset[58].

From this moment, the file is not available anymore, and all variables connected to it are to be considered normal variables (see also *DETACH()*).

ENDFILE() fails if requested to close a file not regularly opened.

7.9.8. ENDGROUP()

The procedure *ENDGROUP()* was conceived to protect file data from being read again (for instance, this may be the case of teachers that want to hide part of the file to students). To do this, *ENDGROUP()* can be used. This procedure is peculiar of the Berkeley version and was not present in Bell.

As with *ENDFILE()*, the procedure *ENDGROUP()* accepts, as the first argument, a file name among the ones opened with *INPUT()* or *OUTPUT()*. The second argument is an integer between 0 and 15 (if exceeding these limits, the argument is resized accordingly), called *level*. See for instance the following program:

```
        OUTPUT('OUT','oldtoread')
        SPACE = " "
        VAR = "VACUUM CLEANER"
LOOP    VAR LEN(1) . OUT =              :F(PROS)
        IDENT(OUT,SPACE)                :S(QUIT) F(LOOP)
QUIT    ENDGROUP('oldtoread',7)         :(LOOP)
PROS    ENDFILE('oldtoread')
        INPUT('READ','oldtoread')
GOON    OUTPUT = TRIM(READ)             :F(END) S(GOON)
```

This program, writes all the letters of the string VAR to the file oldtoread, and when the blank is found, it adds the mark. Then is proceeds with the rest of the string. At the end, it starts to read the data, but stops when the mark is found; the output is:

```
V
A
C
U
U
```

[58] I underline here that "KYB", "TTY" and "ERR", respectively connected to stdin, stdout and stderr, are not closed, and variables connected to them are not detached.

```
M

WARNING: ATTEMPT TO READ PAST END-OF-INFORMATION. LEVEL IS 7
```

The operation performed by *ENDGROUP()* is a mere writing of the code `"#@||@#"` followed by the level in two digits (if lower than 10 a zero is added) on a new line at the current file position.

The same code is checked in every input routine (with *INPUT* or through any input variables) and if such a mark is found, it makes the clause fail with a warning, and all the information written past the code mark cannot be read again by a Snobol4 Berkeley program. Of course, if the file is opened with an editor (vim or emacs, for instance) or through another Snobol4 (Spitbol or a Bell Snobol4 version), the mark becomes visible, as long as all the information that follows, as seen in the previous examples.

This is the effective file `oldtoread`:

```
V
A
C
U
U
M

#@||@#07
C
L
E
A
N
E
R
```

7.9.9. EOI()

The procedure *EOI()* tests if the file argument has its file pointer set to the end-of-information point (in C it would sound end-of-file, or EOF)[59]. In this case, it succeeds, otherwise it fails.

The argument of *EOI()* must be a file already opened with `INPUT()` or *OUTPUT()*, otherwise it breaks. If invoked upon a default I/O channel (`stdin`, `stdout` or `stderr`) it fails.

This function has meaning both for an input process (to test the end of input *before* the possible failing of the *INPUT* procedure) and for an output process (used in conjunction with `BACKSPACE()`, in case the file pointer for next writing is not set to the end-of-information point.

7.9.10. EORLEVEL()

The numeric procedure *EORLEVEL()* checks if the file in its argument (which must have been opened through the *INPUT()* or *OUTPUT()* procedures) points to an end-of-group mark; this mark may have been written on the file by a *REWIND()* (see the dedicated chapter) or explicitly with *ENDGROUP()*; in any case, if found, it returns the level that was written in the mark, from 0 to 15.

If the file pointer is set to the end-of-file, the value -1 is returned, and this is precised also in the GGM.

If the file pointer is set to any line which does not present the end-of-mark (a regular readable line), -2 is returned. This is not specified in the GGM, but I have added this value to distinguish between an end-of-file and the absence of the end-of-group mark.

Practically speaking, if *EORLEVEL()* returns a positive value, the end-of-group mark was found, and the positive value is the relative level. If it is negative, the end of mark was not found; any subsequent reading can find a regular String (-2) or the end-of-file (-1).

[59] It is actually the equivalent of the C library function `feof()`.

7.9.11. FILE()

The test procedure *FILE()* checks if the file whose name is given as argument String (complete with path, if necessary) is found in the underlying file system.

In Unix/Linux this can mean the user has not all the privileges needed to access the file or the directory where the file is hosted. The failure of this procedure is thus to be further investigated.

The name of the file can be given with path, in absolute or relative format. The `~` and `$HOME` symbols are substituted with the user home directory, but this feature has meaning only for Unix/Linus.

FILE() was neither in Bell nor in Berkeley.

7.9.12. FILESIZE()

The numeric procedure *FILESIZE()*, as in pair with *SIZE()* for strings, analyses the content of the file given as argument (with all the path, if necessary), file which must be a textual file, and returns its size in bytes.

The file does not need to be previously opened with *INPUT()* or *OUTPUT()* and maybe any legal file in the file system.

Being *FILESIZE()* a numeric procedure, it can enter any mathematical expression.

The dimension returned by *FILESIZE()* is the same of the `ls` Unix console command.

7.9.13. FORWARD()

The procedure *FORWARD()* sets the file pointer of the file given as argument (a fileset, in Berkeley jargon), to the end of file. It's a painless operation, that fails only if the reposition of the file pointer is not possible (for example, because the file was deleted by some other process).

In case the fileset was not previously opened with *INPUT()* or *OUTPUT()*, the program breaks, an error message appears and the program stops, because the procedure *FORWARD()* must be used only on previously open files.

In case, after setting the forward, an end-of-group mark of level zero is found as the last written item (supposedly put by a *REWIND()* procedure, an implicit *BACKSPACE()* is performed on the file, to ensure that the next writing will overwrite the mark.

The usage of *FORWARD*, neither in Berkeley, nor in Bell, was designed to work in parallel with *REWIND()* and it is its actual reverse procedure.

See also the procedure *SEEK()*, which is a generalization of this one.

7.9.14. RENAME()

The procedure *RENAME()* is a tool for renaming a file, or for deleting it. It works in this way: the first argument is the existing file, given with the whole path if necessary. The second argument is the new name to be given to the file. The path is important to define the new name, because a different path means also the relocation of the file. For instance, with a file called `blue.sno` in the subdirectory `example/`, the execution of the following procedure:

```
RENAME('examples/blue.sno','redred.sno')
```

causes the renaming of the file `blue.sno` to `redred.sno`, and also the moving of the file in the current directory (the one where soft is running).

If the second argument is missing, or is null, the file is simply deleted, as in the following example:

```
RENAME('examples/wid.sno')
```

Remember that the fourth option of *OUTPUT()* lets you decide whether the file is created anew or if it must be opened for appending, so that deleting a file before opening it for writing is not properly necessary.

The procedure *RENAME()* fails if the renaming or the deletion cannot take place (for instance, the file does not exists, or it has not the necessary permissions).

7.9.15. REWIND()

The procedure *REWIND()* sets the file pointer of the file given as argument (a fileset, in Berkeley jargon), to the start of file. It's a painless operation, that fails only if the reposition of the file pointer is not possible (for example, because the file was deleted by some other process).

In case the fileset was not previously opened with *INPUT()* or *OUTPUT()*, the program breaks, an error message appears and the program stops, because the procedure *REWIND()* must be used only on previously open files.

In case the last operation was a write, *REWIND()* writes an end-of-group mark of level zero before rewinding; this assures that a following reading can stop at the mark, and this can be checked with `ENDGROUP()` *before* getting to the end-of-information point.

The example in the premise is an example of usage of *REWIND()*.

The *FORWARD()* procedure has a symmetric reverse behaviour (see).

See also the procedure *SEEK()* which is a generalization of this one.

7.9.16. SEEK()

The procedure *SEEK()* is taken in weight and size from the C language, because of its versatility in managing a file pointer. The file is identified by the first argument, one among the ones opened with *INPUT()* or *OUTPUT()*. The command is:

SEEK(scope_str[, offset_int])

The new position of *SEEK()*, measured in bytes, is obtained by adding or subtracting the number of bytes specified by the second argument *offset_int*, measured from the current position in the file. The offset may be positive (to advance the pointer) or negative (to backspace). The final calculated position is never lesser than zero, and if greater than the file size, it is reset to the file size in append mode. In case of positive result, the current updated position is returned as Integer. As such, *SEEK()* can be used to return the current position in the file (at the moment of the request, acting like the C `tell()`), by omitting *offset_int* (`SEEK('file')`) or setting it to zero (`SEEK('file',0)`).

An alternative syntax can be used:

SEEK(scope_str[, offset_str])

If *offset_str* is the String `'START'`, the pointer is set to the start of file; if it is the String `'END'`, the pointer is set to the end of file.

SEEK() is quite difficult to manage, because the right quantity of bytes is not easy to found, and many tests must be accomplished in order to find the right quantity. In any case, if you use `SEEK('file',-1000)` and you are sure that the file size is lesser than 1000, this command performs a simple rewind, because the final calculated position is 0 (i.e. the start of file).

Watch out! The offset used in *SEEK()* is in bytes, but any line of test is closed by the end-of-line character which is invisible, but counts as one byte, or even, in Windows, by the end-of-line `\n` and the carriage-return `\r` characters, which count as two. The offset may thus be found by some trial and error attempts, if you want to consider only to printable characters.

7.10. Understanding type conversions

The conversion of objects of one type to other types is a process that is usually performed implicitly in the procedures.

For explicit conversions, the *CONVERT()* procedure is added. This procedure was not the same in Bell and in

Berkeley. They bear the same name, but the different prototypes make the two mutually not compatible.

I present here the differences of the two procedures, as implemented in soft, with the warning that the Bell version used to treat more types which are not all present in the Berkeley (and soft) version.

7.10.1. The Berkeley CONVERT()

The Berkeley procedure has a limited range of conversion types, depending on the type of the argument; its prototype is:

CONVERT(arg_val)

with *arg_val* an object of type Integer, Real or String. Using an adapted table as in the GB, a table that summarizes the data type conversions that can be performed by the Berkeley version, the result is quite poor:

	Data type of Object Returned								
Data type of Argument	I	R	S	P	N	C	A	d	D
INTEGER	-	C							
REAL		-	C						
STRING		F	-						
PATTERN				-					
NAME					-				
CODE						-			
ARRAY							-		
defined type								-	
DATA									-

In the previous table, `C` means that the conversion occurs, `F` that the conversion occurs but it may fail, − that the conversion is impossible, a blank means that the conversion certainly fails.

In general this type of conversion is useful for converting number representations to Strings, and to convert an Integer type to Real.

The String to Real conversion, instead, is useful only if the string contains other than digits, the dot, the plus or minus or the letter `E` (exponential format) because, as you may expect, something like

```
OUTPUT = CONVERT("34.56")
```

does not work as expected. In fact, even if in String form, the value `"34.56"` is actually an object of Real type, and the Berkeley *CONVERT()* diligently converts a Real to String, leaving things unchanged. Its main utility is in detecting strings that are not fully numbers, like `"1200 KB"` or `"A23.45"`. In this case, *CONVERT()* fails.

From the table above, it also follows that there is no way to convert a Real to Integer. Peace.

7.10.2. The Bell CONVERT()

The Bell *CONVERT()* is another story. Its prototype is:

CONVERT(arg_val, arg_str)

The second argument *arg_str* indicates the type to convert the first argument to. The conversion can fail, of course, but any combination is theoretically possible and can lead to some (possibly even useless) results.

The following table is the very same one present in the GB (page 141), adapted to use only the types treated by the Berkeley version (and soft), and taking also care of the fact that the type Name has a wider scope in Berkeley than in Bell:

Data type of Argument	Data type of Object Returned							
	I	R	S	P	N	C	A	d
INTEGER	*	C	C	C				
REAL	C	*	C	C				
STRING	F	F	*	C	C	C		
PATTERN			C	*				
NAME			C	C	*			
CODE			I			*		
ARRAY			I				(*)	
defined type			I					(*)

where C means that the conversion occurs, F that the conversion occurs but it may fail, * that the conversion occurs but things remain unchanged, I means that the conversion returns a textual representation of the item type, (*), for created objects, means that the conversion occurs and returns a pointer to the converted objects, a blank means that the conversion certainly fails.

In general, it must be noted that:

- The conversion between String instances (String, Integer or Real) is always performed, but if the object of type String contains other than digits or numeric components (other than the dot, +, −, the letter E or e), the conversion to number fails. The conversion between String instances (String, Integer or Real) to Pattern or Code is always performed; any error is detected in other places of the program (for instance, a wrong Pattern or Code value causes a break or a failure).
- The conversion of String items and literal String items to Code is performed directly; it returns a pointer to the Code instance; this can be assigned to a variable that will be of type Code and that can be used to jump to the Code section.
- The conversion of objects of Pattern and Name types is meaningful only if they are converted to String or Pattern. The conversion to a numeric type is likely to fail.
- The conversion of objects of type Code to String type returns the string "[CODE]"; conversion of objects of type Code to type Code returns a reference to the Code value, that can be stored in another variable.
- The conversion of created objects of type Array to type String returns the string "[ARRAY]"; conversion of created objects of type Array to type Array returns an Array pointer, that can be stored in a variable that becomes a reference to the array.
- The conversion of user-defined data types to String type returns the data name as String enclosed in brackets; conversion of user-defined data types to the same user-defined data type returns a pointer that can be stored in a variable that becomes a reference to the user-defined data item.

It should be noted here that the Bell version can convert a Real to Integer, which the Berkeley version fails dramatically.

7.10.3. Conversions in soft

For the execution of explicit conversions, soft supports both versions of *CONVERT()*, with the limitation of the unsupported types Expression and Table.

The Berkeley side is activated if there is one argument only; if the second argument is specified, the Bell version is activated. soft adds a conversion feature: if the string 'NUMERIC' is used as the conversion type, the item is converted to Integer if it yields a number without decimals, or to Real if it yields a number with a decimal part or in exponential form; otherwise the conversion fails.

Please, refer to the previous sections for the complete usage of *CONVERT()*.

Note: since there is no Table type in Berkeley (and in soft), the type Array cannot be converted to anything but String, with the known output.

For what about the implicit conversions, we must first know exactly what types are available. In soft they were chosen accurately, to make conversions an easy-to-understand task. The types and their weights are ordered in the

following table:

Type	Weight
INTEGER	1
REAL	2
STRING	3
PATTERN	4
NAME	5
CODE	6
ARRAY	7
DATA	8

When two objects of different types are combined, the lesser is converted to the greater type and the resulting type is the greater weight; for instance:

```
OUTPUT = 2 + 3.
```

combines an Integer and a Real, but since the Integer is converted to Real, the final result is Real, even if there are no decimals involved. Or, if a String is associated to an Integer, the final result is String:

```
OUTPUT = "THE SCORE IS " 35
```

because the Integer is converted to String and is attached to the literal string that precedes it.

A characteristic of soft is that if the assign operation cannot understand the tokens, they are classified as Pattern, leaving a next pattern evaluation fail or succeed. For instance, considering that the `!` symbol is not used outside of mathematical expressions, the assignment:

```
OUTPUT = "STR1" ! "STR2"
```

will assign to `OUTPUT` the sequence `'"STR1" ! "STR2"'` as a Pattern. If it is to be used in a pattern evaluation, welcome. If it should fail, the programmer must know what to do in this case.

A note about the Data type is worthwhile: variables created through *DATA()* have inner type Data, but their real type is the constructor's name. So there is no conversion to Data or from Data: there is conversions from constructor's name to String or whatever.

7.10.4. The implicit conversions

The weight table in a previous chapter is useful also for understanding the implicit conversion processes that happen in various parts of the program without the intervention of the programmer. I list these implicit conversions, loosely following the spitbol report format, and adapting the procedures to the available types, as one more tool for the programmer.

INTEGER → REAL
: It occurs in mathematical expressions. The number is not changed.

INTEGER → STRING
: It occurs in String compositions. Leading zeros are suppressed, and a minus sign appears in case the Integer is negative.

INTEGER → PATTERN
: It occurs in pattern evaluations, where the Integer is converted to String (following the previous rules) and used in the evaluation as a Pattern.

INTEGER → NAME
: It occurs in procedures which accepts Names or Strings, where Strings which contain an Integer are converted to Names. This is allowed by the fact with the *$* operator (variable referencing) such a name can be used.

REAL → INTEGER
: It occurs in procedures which accept an Integer, by truncating the decimal part. No rounding is performed.

REAL → STRING
: It occurs in String compositions. Leading zeros are suppressed, and a minus sign appears in case the Integer is negative. The number is generally written with the minimum decimals, and if it exceeds the length of 16 characters, it is converted to exponential (20 characters length at most).

REAL → PATTERN
: It occurs in pattern evaluations, where the Real is converted to String (following the previous rules) and used in the evaluation as a Pattern.

REAL → NAME
: It occurs in procedures which accepts Names or Strings, where Strings which contain a Real are converted to Names. This is allowed by the fact that the dot and digits are parts of a name and with the *$* operator such a name can be used.

STRING → INTEGER
: It occurs in mathematical expressions where a String value contains a number. Leading zeros are suppressed, and a minus sign appears in case the Integer is negative. The empty String is counted as zero.

STRING → REAL
: It occurs in mathematical expressions. Leading zeros are suppressed, and a minus sign appears in case the Integer is negative. The number is generally written with the minimum decimals, and if it exceeds the length of 16 characters, it is converted to exponential (20 characters length at most). The empty string is counted as zero.

STRING → PATTERN
: It occurs in pattern evaluations, where the String is used in the evaluation as a Pattern.

STRING → NAME
: It occurs in procedures which accepts Names or Strings, where Strings are converted to Names.

STRING → CODE
: It occurs in the evaluation of the argument of `CODE()`.

NAME → STRING
: In occurs in String compositions, where the name of the variable is returned.

NAME → PATTERN
: It occurs in pattern evaluation, where the Name is converted to String, and the String is used in the evaluation as a Pattern.

CODE → STRING
: It occurs in assigning a code value to `OUTPUT`: the String `"[CODE]"` is returned.

ARRAY → STRING
: It occurs in assigning an array reference to `OUTPUT`: the String `"[ARRAY]"` is returned.

DATA → STRING
: It occurs in assigning a user-defined type to `OUTPUT`: the name of the constructor enclosed in square brackets is returned as String.

7.11. Understanding RANDOM() and RANDOMIZE()

The random numbers generation was not a main issue in the Seventies, and Snobol had no procedures for generating random numbers. Of course, this can be devised and programmed in Snobol (it's always possible to follow this path), but I feel that a decent and modern programming language cannot miss a command to generate random numbers.

So I installed in soft a random generator, creating the two procedures *RANDOM()*, that returns a pseudo-random number (Real or Integer) and *RANDOMIZE()*, which changes the generation order.

The argument of *RANDOM()* shapes the output:

- if the argument is negative, *RANDOM()* returns a Real number in the range]0,1[, limits excluded.
- If the argument is zero or absent, *RANDOM()* returns an Integer number in the range [1,32768], limits included.
- If the argument is positive, *RANDOM()* returns an Integer number in the range [1,100], limits included.
- If two arguments are given, they are interpreted as two domain limits *a* and *b*: if both arguments are of Integer Type, the calculated random number *n* satisfies the rule $a \le n \le b$ and is of type Integer; if at least one domain

limit is of Real type, the calculated random number x satisfies the rule $a < x < b$ (limits excluded) and is of type Real.

As an example, look at the following program:

```
* CASE OF REAL NUMBER IN THE RANGE ]0,1[
        OUTPUT = RANDOM(-1)
        OUTPUT = RANDOM(-1)
        OUTPUT = RANDOM(-1)
        OUTPUT =
* CASE OF INTEGER NUMBER IN THE RANGE [1,32768]
        OUTPUT = RANDOM(0)
        OUTPUT = RANDOM(0)
        OUTPUT = RANDOM(0)
        OUTPUT =
* CASE OF INTEGER NUMBER IN THE RANGE [1,100]
        OUTPUT = RANDOM(1)
        OUTPUT = RANDOM(1)
        OUTPUT = RANDOM(1)
        OUTPUT =
* VARIOUS CASES OF DIFFERENT REAL AND INTEGER RANGES
        OUTPUT = RANDOM(2.5,5)
        OUTPUT = RANDOM(2.5,5)
        OUTPUT = RANDOM(2.5,5)
        OUTPUT = RANDOM(-2.5,2.5)
        OUTPUT = RANDOM(-2.5,2.5)
        OUTPUT = RANDOM(-2.5,2.5)
        OUTPUT = RANDOM(0,15)
        OUTPUT = RANDOM(0,15)
        OUTPUT = RANDOM(0,15)
END
```

The output is:

```
0.229446446118573
0.153569358903519
0.754849183969924

29911
513
23818

25
28
22

4.72118075069356
3.74111742760704
2.72263249348399
-0.862706952866894
-0.0245669712843721
-1.1825353521575
6
14
1
```

Technically speaking, the generation is built around two variables: the *seed* and the *divisor*. The algorithm is the following:

1. The seed is divided by the divisor and the digits starting from third are taken as the decimal number to be returned. E.g., if the division yields 0.29253401, the returned number is 0.253401;
2 A new seed is calculated basing on the current random number;
3 A new divisor is calculated basing on the multiplication of the seed by the divisor, modulo 65636.
4 Proceed from step 1 to generate the next random number.
5 The randomization changes the divisor basing on current time, which alters the normal succession of the generator.

To ensure a wider range of different numbers, two values are added both to the new seed and the new divisor, values that increase at each invocation up to a maximum value, to be reset when reached, and the process continues.

To avoid also *unfair* random numbers (like 0.2500000, for instance), the seed is always odd, the divisor is always even.

The returned value is controlled: if lower than 2^{-56} (about $1.39x10^{-17}$) or greater than or equal to 1, a new value is tried.

The algorithm behind this pseudo-random numbers generator is simple and naive (you can check its sources for the technical matters), but has shown, in spite of this, a very good behaviour. From the point of view of the pseudo-random number generation frequency, the probability for each number is scattered along the whole spectrum in a very homogeneous distribution. If the generation of one billion numbers is performed, selecting only some scattered values with `RANDOM(0)`, it follows that:

Total calculated:	1000000000
Total extracted:	283466
Theoretical Recurrence:	30517
Deviances Sum:	-21704

Number	Recurrence	Percentage	Deviance
1	24298	8.57	-6219
4000	33540	11.83	+3023
8000	26761	9.44	-3756
12000	31118	10.98	+601
16000	30635	10.81	+118
20000	32084	11.32	+1567
24000	31117	10.98	+600
28000	26752	9.44	-3765
32767	27708	9.77	-2809
32768	19453	6.86	-11064

Table of recurrence for generated random values

The above table shows that the algorithm is quite stable, with a slight tendency to reduce the frequency for the terminal numbers 1 and 32768, which is not a real issue. The distribution is homogeneous, and if different numbers are tried, the same results follow. The percentage of the frequency is very close to the mathematical average value of 10% for ten numbers. The deviance is contained, and in the central part is quite optimal.

Summing up, I know that my random-numbers generator could be bettered, but I'm quite proud of it. This said, the algorithm is not to be used for serious cryptography or important cyphering/deciphering programs, because it does not guarantee unquestionable results for a huge amount of pseudo-random numbers. If you need such a generator, build your own: it's a very interesting intellectual challenge, believe me!

The procedure `RANDOMIZE()` simply generates a new divisor basing on the current clock value and therefore, by invoking it even in two next instants, two different divisors are anyway generated. The argument of the procedure is always ignored and the null String is returned.

7.12. Understanding IF()

The procedure *IF()* is atypical. It's definition follows (page 144 of the GGM):

```
IF() always succeeds. Since it is defined to have no arguments, any arguments in a reference to
IF() are evaluated but otherwise ignored. Thus if any part of that evaluation fails, that failure
causes failure of the rule. If a reference to a procedure returning a non-null value is written
as an argument of an IF() procedure, the combination will work like a test procedure. The same
principle applies to ether expressions returning values which can similarly be converted into
test procedures.
```

The contradiction is in the premise: "*IF() always succeeds*". If it was true, any *IF()* clause would succeed and be completely transparent (and useless). But what follows is illuminating: "*...Thus if any part of that evaluation fails, that failure causes failure of the rule...*". So it's not true that *IF()* always succeeds; the real definition should be: "*IF() always succeeds in evaluating what is passed as argument, and returns the status of the evaluation (failure or success)*". The example given in the GGM is quite clear:

```
N = IF(ARR1[N+1]) N + 1       :F(OUT)
```

The clause `ARR1[N+1]` may succeed or may fail. If the current `N` makes the value `N+1` overflow the array limit, the clause would fail. Otherwise it would succeed. *IF()* evaluates the clause, and finds a state report of failure or success. In case of success, `N` is instantiated to the new value `N+1`, otherwise the whole line fails, and the go-to redirects the program flow to the label `OUT`.

So *IF()* succeeds as long as what is passed as argument succeeds[60].

The *IF()* procedure is the Berkeley version of the Bell function called *Interrogation operator* (`?`) (See page 174 of the GGM), which is not implemented neither in the Berkeley nor in the soft version.

7.13. Understanding the test procedures

The usage of the test procedures is not the same of other programming languages.

A typical `if-then-else` condition in C is

```
if (A > 0) B = 140;
else B = 10;
```

The instantiation of `B` is made in any case, depending on the positive or not positive content of `A`.

There is no equivalent in Snobol4. The same task is applicable, for instance, in this way:

```
B = 10
B = GT(A,0) 140
```

The condition is expressed in the second line; what happens?

- The procedure *GT()* returns the empty string, which does not influence the result.
- If `A` is greater than zero, the state of the clause is not changed to *failure*, and the value `140` is added to the empty string. The evaluation of the final result `"140"` is interpreted as an Integer and stored in `B`, overwriting the previous value.
- If `A` is not greater than zero, the state of the clause is changed to *failure* and the assignment does not take place. `B` retains the value `10`.

Another common usage is in pattern evaluations, where a test procedure influence the failure or the success of a pattern evaluation. The following line is used in the A-CAPPELLA CHOIR program on page 230 of the GB:

```
SONG   PAUSE IDENT(SING) OUTPUT('SING', ACAPPELLA.CHOIR,
```

[60] I hope I have understood the problem in the right way.

```
+       "(' " PAUSE"', 100A1)") = '                    : (RETURN)
```

In case the variable SING is null, an output channel is created through *OUTPUT()*, but if *IDENT()* should fail, no output channel is created.

Another common usage of a test procedure is a test on its own, whose state can influence the go-to that follows; e.g.:

```
VER     GE(A,0)                                        :F(RE.ASK)
GO.ON   ... {use A}
        ...                                            :(END)
RE.ASK  ... {ask or calculate a new A }                :(VER)
```

This piece of code shows that the test procedure *GE()* is written on its own; in case if succeeds, the process continues from GO.ON; if it should fail (because A<0), a new A is calculated or asked and the process repeats the verification, until a valid A is produced. The equivalent C excerpt is:

```
while (A<0) reask();
else goon();
```

More compact, yes, but C is a structured language.

The test procedures in soft are:

DIFFER()
EOI()
EQ()
FILE()
GE()
GT()
IDENT()
IF()
INTEGER()
LE()
LGT()
LT()
NE()

In the vocabulary, all these test procedures are explained. Read also the GGM and the GB.

7.14. Understanding the bitwise and logical procedures

The bitwise and logical procedures work in two modes: with Integer arguments in bitwise mode, returning an Integer result, and with String arguments, representing Snobol clauses, and working as test procedures.

Let's review them in order.

7.14.1. The bitwise procedures

The bitwise procedures accept as arguments Integer values, and return an Integer value. They behave in all context as Integer functions. What is the proper usage of such procedures?

Well, to answer this question I will use a method which is common in C programming. A *flag* is a two-state item, which can be set or unset. In particular, we can use the single bits of an Integer value to hold many flags.

Suppose we set the first bit of an Integer as our flag, and that this integer is called FLAGSET. To test if the first bit is set we compare our number with an Integer with these features: the first bit set and all other bits unset; this is the

number 1. Then we can test for the first bit.

```
STATE = EQ(AND(FLAGSET,1),1) "STATE SET"
```

The two bit states may be, for instance:

```
10110001
00000001
```

The *AND()* procedure returns 1 for a bit correspondence of both ones, and zero for any other bit correspondence. The returned number is

```
00000001
```

which means the bit was set. If the bit is not set:

```
10110000
00000001
```

the returned number is

```
00000000
```

which means the bit was not set.

Any other bit can be considered (the values are all powers of 2: 2^0, 2^1, 2^2, 2^3 etc). For instance, to test for the fourth bit we use

```
STATE = EQ(AND(FLAGSET,8),1) "STATE SET"
```

The bitwise procedures in soft are:

AND()
EQV()
NOT()
OR()
IMP()
XOR()
LSHIFT() with alias *SHL()*
RSHIFT() with alias *SHR()*
LROLL() with alias *ROL()*
RROLL() with alias *ROR()*

Notes for the shifting and rolling procedures:

- The sign of the first argument is not used in the calculation,, but stored apart. The bitwise calculation is operated to the positive value; the sign is reapplied at the end, when the result is returned.
- The second argument is always interpreted in unsigned mode, so it cannot be a negative Integer; moreover, it is always calculated modulo 32, in order to consider only movements within the 32-bit range.

7.14.2. The logical procedures

The proper structure of the program flow in Snobol4 is not classic, and follows a loose procedural scheme, because the program flow is deeply influenced by the failure/success state of each executed line.

Sometimes it is desirable to process a number of tests, in order to proceed. For instance, suppose that a particular

assignment can take place only if `A>0` and `B<0`. The proper way to write such an assignment is, for instance:

```
LENGTH = GT(A,0) LT(B,0) A * -B
```

The condition expresses the logical condition "*if A > 0 and B < 0 then...*. If one or both tests should fail, the assignment wouldn't happen.

But what if some peculiar condition arises? For instance, one could consider the case *if A > 0 or B < 0* (asking for at least one truth) or even *either A > 0 or B < 0* (asking for one truth only)? The common Snobol4 structures cannot help. For example, using alternatives for an *or* condition:

```
LENGTH = (GT(A,0) | LT(0,B)) A * -B
```

is of no use, because then the alternative fails in either case, the whole clause fails.

That's where the logical expressions come in handy. They analyse their arguments, and return a success if they conform to the true state of the logical expression, or return a failure, and so making the Snobol4 clause fail. Using a logical expression on the previous instance gives

```
LENGTH = OR("GT(A,0)","LT(0,B)") A * -B
```

The *OR()* procedure takes the two tests, evaluates them separately (not influencing the final state) and then compares them for the *or* logical expression, that is true if at least one condition is true. If so, it proceeds to instantiate `LENGTH`.

The logical procedures in soft are:

AND()
EQV()
NOT()
OR()
IMP()
XOR()

The arguments of these procedures are all signed Integers.

7.14.3. The special procedures

The *AND()*, *OR()*, *NOT()* and *XOR* procedures present no conceptual difficulty, because they are of common usage in all programming languages.

The procedures *EQV()* and *IMP()*, instead, deserve some more talks, because they are not common; this is why they are here defined as *'special'*.

The procedure *EQV*, which tests for the equivalence, is not interested in the relative state of its arguments, but only if they have the same value (both true or both false). This is the contrary of the *XOR()* procedure, and actually, the truth tables of *EQV()* and *XOR* are opposed: where *EQV()* succeeds, *XOR()* fails and vice versa.

The procedure *IMP()*, which tests for the implication, has a unique feature: it is not symmetric; this means that the two arguments lie in a fixed position (they cannot be exchanged), and this is why they have deserved, in some texts, a proper name (like *premise* and *consequence*, or *antecedent* and *subsequent*, and so on).

The definition of *IMP()* was not built on a mathematical point of view. First of all, the implication is set to succeed if the consequence (the second argument) is true; this is straightforward: if the consequence is true, no matter what the premise is, we can always say that the implication holds.

If the premise is false and the consequence is also false, the implication holds again; it's like asserting that the implication *'if London is the capital of Italy, then donkeys fly'* is true (it may take a little time to fully understand this

clause).

The only case of failure is when the premise is true and the consequence is false; this is straightforward too, because it is the key of *false reasoning* (if the premise is good and the consequence is not, then the reasoning is missing some points...).

It's obvious that the logical procedures here presented are not really necessary in Snobol4. A good programmer can ignore them and build different structures to get the same effect. I added them to soft because I think that is some occasions, they can be the key for a faster programming, but if you want to ignore them, ignore them.

7.15. Understanding FENCE

The scope of the pattern variable *FENCE*[61] is prevent further backtracking leftward its position, but letting the pattern evaluate what is rightward. The definitions in the GGM and in the GB are not as clear as the one in the Spitbol manual, here reported:

```
Pattern FENCE matches the null string and has no effect when the pattern matcher is moving left
to right in a pattern. However, if the pattern matcher is backing up to try other alternatives,
and encounters FENCE, the match fails.

FENCE can be used to lock in an earlier success. Suppose we want to succeed if the first 'A' or
'B' in the subject is immediately followed by a plus sign.  In the example below, the 'A's match,
we go through the FENCE, and find '+' does not match the next subject character, 'B'. SPITBOL
tries to backtrack, but is stopped by the FENCE and fails:

? '1AB+' ? ANY('AB') FENCE '+'
  Failure

If FENCE were omitted, backtracking would match ANY to 'B', and then proceed forward again to
match '+'.

If FENCE appears as the first component of a pattern, SPITBOL cannot back up through it to try
another subject starting position. This results in an anchored pattern, even if the &ANCHOR key-
word specifies unanchored mode:

 ? 'ABC' ? FENCE 'B'
   Failure
```

These examples in the Spitbol manual are clear. Other examples are here exposed.

The clause

```
'AB' 'A' FENCE 'B'
```

succeeds, because the order of evaluation of the patches is:

- `'A'` matches; the cursor is advanced by 1
- `FENCE` matches; the cursor is not advanced
- `'B'` matches; the cursor is advanced by 1
- final state: *success*

and *FENCE* is never passed leftward. The following clause, instead, fails:

```
'ACAB' 'A' FENCE 'B'
```

Why? The failure is summarized as follows:

[61] The pattern variable *FENCE* took a lot of study, experimentation, remake, trial, frustration and failure before it was implemented, and I'm still not sure it's correct. Imagine.

- 'A' matches; the cursor is advanced by 1
- FENCE matches; the cursor is not advanced
- 'B' does not match; the cursor is set to 1
- current state: *rematch*
  But the rematch cannot take place, because *FENCE* is passed leftward
- final state: *failure*

A more complex example involving alternations is the following:

```
        WORD = 'BERATES GETTERS'
REDO    WORD  (('BE' | 'GE' | 'FRE') ('TS' | 'T')) =      :S(REDO)
        OUTPUT = WORD
```

This first version (not involving *FENCE*), searches for the first match, which is GET, and erase it, after failing all 'BE' clauses. No more instances are present, and the string BERATES TERS is printed.

Now suppose that we require that if the first word fails, then the whole clause must also fail; the usage of *FENCE* solves the question:

```
        WORD = 'BERATES GETTERS'
RED1    WORD  (('BE' | 'GE' | 'FRE') FENCE ('TS' | 'T')) = :S(RED1)
        OUTPUT = WORD
```

In this case, after trying to match 'BE' and failing (no BETS or BET are found in WORD), the scanner is forced to backtrack to the alternation to find another item, as in the previous example, but in doing this it passes *FENCE* leftward, and the clause fails. So WORD is printed unchanged.

In short, I found *FENCE* simple to understand, difficult to use, challenging to implement. A real mess!

7.16. Understanding the missing SUCCEED

The system variable *SUCCEED* is a mystery. The Berkeley version didn't implement it (and on the other hand neither soft), with the motivation (GGM, page 181): "*The predefined pattern variable SUCCEED, which always matches the null value (and which has very limited practical application) is not available.*"

Moreover, the GB, at page 61, offers one and only example, here reported:

```
        SAWTOOTH = SUCCEED (LEN(1) ARB) $ OUTPUT FAIL
        'XXXXXX' SAWTOOTH
```

that produces an infinite (and useless) cycle:

```
X
XX
XXX
XXXX
XXXXX
XXXXXX
X
XX
XXX
XXXX
XXXXX
XXXXXX
.
.
.
```

The definition of *SUCCEED* in the GB is in facts the following cryptic assertion: "*The variable SUCCEED has a pattern structure as its initial value. SUCCEED matches the null string when first encountered by the scanner*

moving left to right through a pattern. If a subsequent failure causes the scanner to back up to SUCCEED, seeking an alternative, SUCCEED again matches the null string. Thus, SUCCEED always matches the null string, both in the forward direction and when alternatives are sought.

This definition seems to assert that matching a null string is the only task accomplished by *SUCCEED*, something that does not seem plausible.

The Spitbol manual reports instead very important considerations about *SUCCEED*. Here is an extract of the definition in the Spitbol manual (pages 126-127) .

(from the Spitbol manual)

```
SUCCEED           Match always
```

This pattern matches the null string and always succeeds. If the scanner is backtracking when it encounters SUCCEED, it reverses and starts forward again. Placing a pattern between SUCCEED and FAIL causes the pattern matcher to oscillate. We used to say that there were no serious uses for the SUCCEED pattern, but discussions in September, 1996 on the SNOBOL4 mailing list server have effectively countered that assertion. While the on-line list archives can be consulted for the full discussion (file list9609.txt), they can be summarized as follows:

SUCCEED can be used as part of a loop control structure within a pattern when it appears in the form:

```
        SUCCEED something_with_possible_ABORT FAIL
```

The pattern matcher will continually move back and forth between SUCCEED and FAIL until something in between causes the match to abort. This example

```
        ? P = FENCE(TAB(*(N + 1)) $ OUTPUT @N | ABORT)
        ? "abcd" ? POS(0) $ N SUCCEED P FAIL

 a
 ab
 abc
 abcd
 Failure
```

displays successively larger subject substrings. The FENCE function (discussed in the next section) is necessary to prevent the pattern matcher from seeing the ABORT pattern when it is backing up. The substrings could be presented to a user-defined function which would determine when to end the loop (by returning the null string to continue or the ABORT pattern to terminate).

```
        DEFINE('FN(T)') :S(FN_END)
 FN     FN = GT(SIZE(T), 4) ABORT :(RETURN)
 FN_END

        P = TAB(*(N + 1)) $ T *FN(T) @N
        S ? POS(0) $ N SUCCEED P FAIL
```

Here, the pattern would continue to oscillate until the matched substring contained more than four characters. The ABORT pattern has been moved to within the function, and the FENCE function is not needed because there are no alternatives within P.

Peter-Arno Coppen has devised these rules for the use of SUCCEED:

1. If SUCCEED can be replaced by the null string, it is superfluous.
2. SUCCEED without an ABORT or FENCE, and without a deferred expression will either never terminate or it is superfluous.
3. No sensible use of SUCCEED is possible without the unevaluated evaluation operator.

These considerations in the Spitbol manual are quite final. The Berkeley version lacks the *FENCE()* function (that has not to be confused with the system variable *FENCE*), and cannot use the deferred operator (called unevaluated evaluation operator in Bell jargon) on objects that are not natural variables and therefore points 2 and 3 are impractical; the only point left is the first; therefore, in these exquisite simple terms, *SUCCEED* is *superfluous* for the Berkeley version and its heirs.

7.17. Understanding FREEZE()

When *FREEZE()* is encountered during execution, the engine writes out a copy of the entire job data onto a predefined text file, and execution is terminated with the string:

```
System frozen on: DD-MM-YYYY  HH:MM:SS
<user code string>.
```

where the <user code string> is the argument string to *FREEZE()*, which is printed both in the freeze banner (as seen) and in the reprise banner, to help identifying the two processes (that may occur not consecutively) as one.

When the user re-executes the Snobol4 program, soft finds the predefined file (called *job data file*, built by soft basing on the current file name), reloads from it the whole job data, re-establishes the I/O system and execution continues from the point where it was frozen. The execution of the program should be done from the same point in the file system, in order to let soft recognize the predefined file[62].

The restart prints the following *reprise* banner:

```
Frozen on    : DD-MM-YYYY  HH:MM:SS
Restart now  : DD-MM-YYYY  HH:MM:SS
With message :<user code string>.
```

The procedure *FREEZE()* does not guarantee that the re-start takes effectively place if it is put in a *DEFINE()* user procedure, a loop or a pattern evaluation; it should rather be put as a single instruction into one specific line in the main level. Moreover, no go-to should follow *FREEZE()*, because the program is immediately interrupted and the restart takes place from the line that follows *FREEZE()*, and the go-to would be ignored.

After the system has loaded the job data, the predefined job data file is deleted, and unless a new *FREEZE()* is encountered, the next restart will be normal.

Notes:

- If invoked from within PIE, *FREEZE()* will be simply ignored.
- All console options invoked at the reprise take are overwritten by the registers restored from the job data file; this means that the console option of the original execution (before the freeze) are maintained.
- The job data file is a pure textual file, which can be read by any text editor but **it shouldn't be altered in any way**: if the reload should find some unaligned data, an error is printed and the reload fails, with unpredictable results.
- The total timing calculated by soft does not include the intermediate time between the two phases, but only the effective total time summing the time of the first phase and the time of the second. The intermediate time may be calculated from the reprise banners time reports, if needed.

Use *FREEZE()* with care. I have tested it along, and the job data file contains practically everything, but execution parameters substitution is never a safe operation. If you experiment some bug, write to me.

7.18. Understanding PUT()

The apocryphal procedure *PUT()* returns as a String type the evaluated result of its argument, which may be of any type. If it is any variable of any type, its content is returned as String; if it is an expression of any kind, it is evaluated and the result is returned as String as well. It also removes the surrounding parentheses of a pattern.

[62] The original definition, using job cards, required a user string to set the file scope, and this string was the argument of *FREEZE()*; when a later job card required the file scope to be loaded again, the program restarted (as explained on page 148 of the GGM) from where it was frozen. Of course, since soft runs on file-based Unix/Linux machines, I had to redesign the whole process, and now the user does not set up the file scope anymore.

See the following example from PIE, where option `printc` is enabled through the debug command `SET` (corresponding to the console option `--print-complete`):

```
$ P = 'A' | 'V' | 'Z'

Yes.

$ set
Set flag : printc
Set state: on

$ OUTPUT = P
[PATTERN] = ('A' | 'V' | 'Z' )

Yes.

$ OUTPUT = PUT(P)
'A' | 'V' | 'Z'

Yes.

$ OUTPUT = PUT('AA' 'BB')
AABB

Yes.

$ OUTPUT = PUT(3 * 12 / (1 +2.4))
10.5882352941176

Yes.
```

It was conceived during the design and implementation of soft, for debugging purposes, and I maintained it in the final release because there is no equivalent command in Snobol4, and it may prove useful.

8. DEBUGGING AND TRACING

Sometimes, even well-conceived programs don't work at the first execution; it may depend on some typing error, on some omission, on some wrong reference, something that often is not detectable in a glance.

For this, I have introduced in soft some tools that can help checking the program flow: the debugger and the variables tracer. The debugger acts in three different ways: 1) executing lines one by one (the *line debug*); 2) executing the program as a whole with the possibility to analyze the memory (the Pattern Interactive Evaluator - PIE); 3) the pattern evaluation analysis (the *pattern debug*).

8.1. The line debug and PIE environments

The debugging engine is suitable for the evaluation of the program, to spot where the error is located, analyzing one line at a time; the command `PIEMODE` disables the debugger and activates the evaluator, called PIE (Pattern Interactive Evaluator), suitable for the analysis of single Snobol4 patterns, even unrelated to the program in memory. The PIE/debug commands are explained in the relative section.

When a Snobol4 command is typed in a debug or a PIE mode, the cursor is reset at every new command, unless the command itself is exactly equal to the previous, to let the user repeat the evaluation on the same base string; if you want to manually reset the cursor, you can use `RESET` (see ahead).

All debug commands are case-insensitive; Snobol4 commands follow the language rules, but labels are forbidden.

8.1.1. The line-by-line debug

The line-by-line debug (or simply line debug) is introduced by the `-d/--debug` option at start, or by the command `&DEBUGMODE` in a PIE session.

During the debugging, for every line that is about to execute, before its possible output, its content is printed in reversed colours, followed by the request for the user intervention, at the prompt:

```
>
```

In a program, debugging can also be enabled and disabled at will using the command `DEBUG(S)`, where debugging is enabled if the string `S` is any non-null and non-zero string, otherwise it is disabled.

In the line debug mode, all commands are available, but the analysis of Snobol4 statements is not available; if you type, for instance:

```
$ a = 34
```

this will be interpreted as a regular `ALLVARS`; if you type instead:

```
$ j = 34
```

you will get:

```
Unrecognized debug command. Type help for the command list.
```

because there is no debug command beginning with j.

If the typing is ambiguous, the command with the lower alphabetical ranking is executed; for instance, if you type:

```
> c
```

you will get the execution of the command `CHANNELS`. To execute the desired command (e.g. `CURSOR`) type as many letters as to make the command not ambiguous, e.g.:

```
> cu
```

and `CURSOR` will be set.

8.1.2. The PIE session

The PIE session is introduced by the `-i/--interactive` option at start, or by the command `PIEMODE` in debug mode.

PIE expects the user to type something at the prompt

```
$
```

The execution of a program is continuous, as executed from the terminal, but all variables are retained in memory and any Snobol4 command may be typed. In case of a pattern evaluation, "Yes." or "No." is printed, when the evaluation is accomplished, according to the final state (success or failure).

All debug commands are available, but the primary intent is for Snobol4 execution. For instance,

```
$ a = 34
```

and

```
$ j = 34
```

will be interpreted as regular Snobol4 assignment. To type a debug command, type as many letters as to make the command not ambiguous and use no arguments:

```
$ all
```

Here you will see the `ALLVARS` command in action. If the typing is ambiguous, the command with the lower alphabetical ranking is executed:

```
$ c
```

As with the debug mode, the `CHANNELS` command will be executed.

8.1.3. The debug commands

The following commands work both in the debug and PIE modes; not all commands are useful for each mode, so use the ones that best fit your needs.

Watch out! Debug commands have no arguments: they will be asked, if necessary, with the following request. If you type any argument following the command, your request will be interpreted as a Snobol4 instruction (with unpredictable results).

- `ALLRESET`: reset the evaluation registers, in order to perform the next evaluation from a fresh start, reset all program registers, set all input files to start and output files to the end.
- `ALLVARS`: list all variables, their types and contents, including also null and system variables.
- `BASE`: print the base string (that is, the string against which patterns are matched), if any.
- `BREAKPOINT` (you will be asked for a line number `<n>`): set a breakpoint of the pattern evaluation debugging for the line `<n>`, where supposedly a pattern evaluation (and not an assignment or a direct command) is contained; up to 256 breakpoints can be used for each execution in the current implementation. When the pattern evaluation is completed, the normal debugging continues, and the pattern evaluation debug will happen again when another breakpoint is met

  The following rules govern the `BREAKPOINT` command:
  - If `<n>` is null, all the current breakpoints are listed.
  - If `<n>` is the symbol +, the breakpoint is set to the next line.
  - If `<n>` is the symbol −, the breakpoint is set to the previous line
  - If `<n>` is the symbol |, the breakpoint is set to the current line.

- CHANNELS: list the current available I/O channels. The scheme is the following:

```
The number of opened channels is <n>
List of opened channels:
Channel No. 0: output, destination 'screen'
Channel No. 1: input, source 'keyboard', echoed
Channel No. 2: input/output, source/destination <chan>
...
Channel No. <n-1>: input/output, source/destination <chan>

Current input channel is <n>, source is <source> (input exhausted)
Current output channel is <n> to <dest>
```

  The string (input exhausted) appears if the last input file is at the End-Of-File (i.e. no more input is possible). The strings echoed/no echo appear according the input feature.
- CLIST: list the entire program in a beautified format. The current line appears in green.
- CONTINUE: continue the program execution and disable the debug mode. The output proceeds emitting the regular output in regular characters.
- COUNT: print the number of system and user variables declared so far.
- CURSOR: print the cursor value and the string pointed by it so far.
- DATASTRUCT: print information about all the user data. For each function, the scheme is the following:

```
Name = <s>
Arguments Proto = <proto> [<n>].
```

  Since the information is all contained into the String argument, DATA() structures can be declared from within PIE.
- DEFINED: print information about all the user procedures. For each function, the scheme is the following:

```
Name = <s>
Arguments Proto = <proto> [<n>]
Other Proto = <proto> [<n>]
Starting label = <label>
Starting line number = <n>
NRETURN was used last time? <y/n>
```

  It must be noted that at present PIE cannot execute user functions from console, because soft is interpreted, not compiled.
- DIR: (you will be asked for a directory path <s>): list the files in the specified directory. If the input string is the null string, the files in the current directory are listed (equivalent to ./).
- DISABLE (you will be asked for a line number <n>): remove the breakpoint set upon line <n>, according to the following rules:
  - If <n> is null, nothing changes.
  - If <n> is the symbol #, all breakpoints are disabled.
  - If <n> is the symbol %, the last breakpoint set is disabled.
  - If <n> is the symbol +, the breakpoint of the next line is removed.
  - If <n> is the symbol −, the breakpoint of the previous line is removed.
  - If <n> is the symbol |, the breakpoint of the current line is removed.
- DEBUGMODE: disable the PIE mode and return to the line-by-line debug.
- ERASE: remove all instances of the variables, deleting their contents and setting their initial state, with only the system variables available.
- FILE: print some data about the file currently under evaluation. In case of a loaded file, the scheme is the following:

```
File name: <name>
Directory: <dir>
Dimension: <n> bytes (<m> kB)
Lines    : <n>
```

In case of a pure PIE session (without file), only the session name is printed.

- HELP (you will be asked for a string <s>): print the help page about the debug command <s>; if <s> is null, a list of the available commands is printed; if <s> is specified, it must be a name of one debug command: a small help about the command is printed. If <s> is an asterisk, the help of all available commands is printed.
- INFO: print some info about the debugger.
- LABELS: print a list of all the labels in the program lines and where they are located. The scheme is the following:

```
The number of labels is <m>:
  1. Label <name1> in line <n> has code <n>
  2. Label <name2> in line <n> has code <n>
  ...
  <m>. Label END in line <n> has code 100545
```

Notice that every program must terminate with the label END.

- LIST: list some lines around the current line in a beautified format. Five previous lines and five further lines, with respect to current, are shown (total 11, or less if the previous or further lines are less than required). The current line appears in green. To list the entire program use CLIST or NLIST.
- LOAD (you will be asked for a file name <fname>): load the file <fname>, that becomes the current program under evaluation. All registers are reset, for a fresh new execution. The complete path may be specified into <fname>.
- MEMORY: print the memory amount of the program.
- MONITOR: exit to shell (also SHELL). Type 'exit' to return to soft.
- NEXT: execute current line and step to the next program line (it is a mere clone of the Enter key).
- NLIST: list the entire program in numbered format. The current line appears in green.
- ORIGINAL: list the original program text of the program (empty lines and comments included).
- PARAMETERS: list the current execution parameters, all in one. The scheme is the following:

```
The number of program lines in memory is <n>
The number of Snobol4 commands executed so far is <n>

Base comparing string is '<s>'
Evaluation Request string is <s>
Cursor pointer is <n>
FENCE pointer is <n>
Previous Cursor pointer is <n>
Cursor points to <s>
Remaining base string is '<s>'
Last pattern is '<s>'
Rematch flag:  true/false
Final state:  success/failure
```

A string <s> is written as <null> in case of empty string. The FENCE pointer is line becomes FENCE not active in case *FENCE* is not used in the command line.

- PATTERN: print the pattern evaluation request and the current pattern situation (that is, the pattern gathered so far). If the base string does not contain the single quote, it is printed enclosed in single quotes; otherwise, if the base string does not contain the double quotes, it is printed enclosed in double quotes; otherwise it is printed enclosed in curly brackets.
- PIEMODE: disable the line-by-line mode and enter the PIE mode. If PIEMODE is typed during a program execution, the current execution is terminated before stepping in Pie mode.

- QUIT: stop the program execution both in the line-by-line and the PIE modes. The string "soft debug: End of session. Ok." appears, to distinguish between a wrong stop (caused by some error) and an intentional stop.
- REMARK (you will be asked for a text string <s>): write string <s> onto the job log file, or the chosen channel, depending on options -j or --joblog. The text in the string <s> is in free format.
- RESET: reset the evaluation registers, in order to perform the next evaluation from a fresh start.
- RUN: execute the program in memory, according to the current mode: if in PIE mode, the running is continuous, as from console; if in DEBUG mode, one line at a time is executed. Before execution all registers are reset, all input files are set to start and output files to the end, but all variables are retained.
- SET (you will be asked for a <flag> name and for a <state>): change some system flags state, not coordinated by language procedures, where <flag> may be:
  - ALTER to enable/disable the alteration of the system variables.
  - PATDEBUG to enable/disable the pattern debug.
  - PRINTC to enable/disable the printing of pattern and created variables.
  - STATS to enable/disable the statistics at the end of the program.
  - TIMING to enable/disable the timing at the end of the program.

  If <flag> is null, the current flags states are printed.

  The ON and OFF states respectively enable and disable the feature; the TOGGLE state inverts the current settings.
- SHELL: start a shell session. Type 'exit' to return to soft. The preferred shell may be selected in the variable SHELLEXT in soft.h (by default bash).
- STATE: toggle the printing of a detailed evaluation state after every program line (the state is off by default). The output is in reverse colors, and the scheme is the following:

```
-- line <n> --
base string is '<s>'
pattern request was '<pattern>'
last retrieved string pattern was '<s>'
cursor is <n> and points to '<s>'
last report was a <status>
```

  The meaning of fields is quite clear; I will only add that the last retrieved string pattern is the pattern result gathered so far, and the string pointed by the cursor is a visualization of the next possible pattern; the status may be a *failure* or a *success*, depending on the last execution state. This information is useful to see the Snobol4 process from inside and step by step. Option --final-state prints the same mask (apart from the line number) in normal colors as the last information, after execution.
- TRACE (you will be asked for a variable name <var>): mark variable <var> for tracing. If <var> is null, all the variables under tracing are listed.
- TYPE (you will be asked for a file name <fname>): print the content of the textual file <fname>.
- UNTRACE (you will be asked for a variable name <var>): remove the trace mark from variable <var>. If <var> is equal to the symbol #, the tracing is removed from all variables. If <var> is null, nothing changes.
- VERSION prints current soft version.
- WATCH (you will be asked for a variable name <var>): print detailed information about the argument variable identified by <var>, which may be a natural or a created variable. If it is an array item, it must be given along with the coordinates (e.g. LIST[1,3]). WATCH treats also referenced variables in the form $VAR.
- Any integer number is searched in the variables list codes, and the corresponding variable compact status is reported.
- A number interval in the form *n-m*, with no spaces around the dash, shows a compact report on all variables having codes in the interval *n-m*; intervals are treated as follows:
  - if the indices *n* and *m* are respectively lower than 1 or greater than the current number of declared variables, they are resized accordingly;

- if *n>m*, they are reversed implicitly;
- if the form `n-` is used, the variable list goes from *n* until the last declared variable;
- if the form `-n` is used, the variable list goes from from 1 to *n* included;
- finally, if the dash only `-` is used, all variables are listed (as `ALLVARS`).

The format of the line is

```
[N][MMMM] ==> NAME = CONTENT [TTTT]
```

where `N` is the variable code, `MMMM` is length of the content, `NAME` is the name of the variable, `CONTENT` is its content in any types, and `TTTT` is the data type of the variable.

- The Enter key (in the Debug mode) executes current line and steps to the next program line.
- Any other command is intended as a Snobol4 assignment or pattern evaluation if in PIE mode, or it is an error in debug mode.

A final note about the listing commands `CLIST`, `LIST` and `NLIST` ignore the commented lines, which don't appear in their output. The only way to see the commented lines is with the command `ORIGINAL`.

8.2. The pattern evaluation debug

The pattern evaluation debugging works only on lines with a pattern evaluation, and not assignments or direct command. When activated, the program flow is shown in detail, and this is useful especially for alternations or complicated patterns involving variable terms (like `ARB`). Each program line is solved in details before your eyes, for each pattern evaluation step. Every pattern evaluation is put into a table with the following header (it is advisable to use terminals with 100 or more characters per line):

```
line    scan    match           status  altern. alt scan        next scan       pattern
-------------------------------------------------------------------------------------
```

At every evaluation, success or failure, the line content is printed and revealed; the fields are:

- `line`: the number of the line currently under process, preceded by the letter *L*, to make it different from the numeric values that follow
- `scan`: the current cursor position *before* starting the evaluation
- `match`: the matching string portion of the current token; if the match is the `NULL` string, the slash character **/** is printed
- `status`: it reports `V` for success (Validation) and `NO` for failure. The success introduces a green line, the failure a red one.
- `altern.`: if an alternation is found next, this is the string (only the first 6 characters are printed at most) pointed by the cursor position that will be set in case of failure, for testing another element of the alternation. In case of alternation not found, the slash character **/** is printed
- `alt scan`: if an alternation is found next, this is the cursor value that will be set in case of failure, for testing another element of the alternation. In case of alternation not found, 0 (zero) is printed
- `next scan`: the value that the cursor position will assume *after* the pattern evaluation.
- `pattern`: the pattern string collected so far.
- An asterisk `*` appears uncoloured at the far right end if the failure was due to *FAIL* or to *POS()* or *RPOS()* which may cause a rematch or a failure.
- A grid # appears uncoloured at the far right end if the failure was due to *FENCE*.

At every line, the execution waits for the user intervention (with exception for the `-p/--pattern-debug` option, that does not evaluate any user commands and goes on continuously); available actions are:

- The letter *c* before the Enter key will continue the pattern evaluation for the current line disabling the pattern debug.

- The letter *g* before the Enter key will disable the debug and the pattern debug, and the execution is set to continue until its natural end without any other stops.
- The letter *l* before the Enter key will print the current program line under analysis; under the line, there is an indicator ^ that signals the next part to be evaluated in the matching string.
- The Enter key will cause the execution to step to the next program line.

Any other character or combination of alphanumeric characters or symbols is ignored. For example, for the following line:

```
'1AB+' ANY('AB') FENCE '+'
```

here is the execution-time output:

```
line    scan    match           status  altern. alt scan        next scan       pattern
-------------------------------------------------------------------------------------
L2      0       /               NO      /       0               1               /
L2      1       A               V       /       0               2               A
L2      2       /               V       /       0               2               A
L2      2       +               NO      /       0               2               A       #
```

Here's what is looks like in colours from my monitor:

Another example, involving alternations is the following:

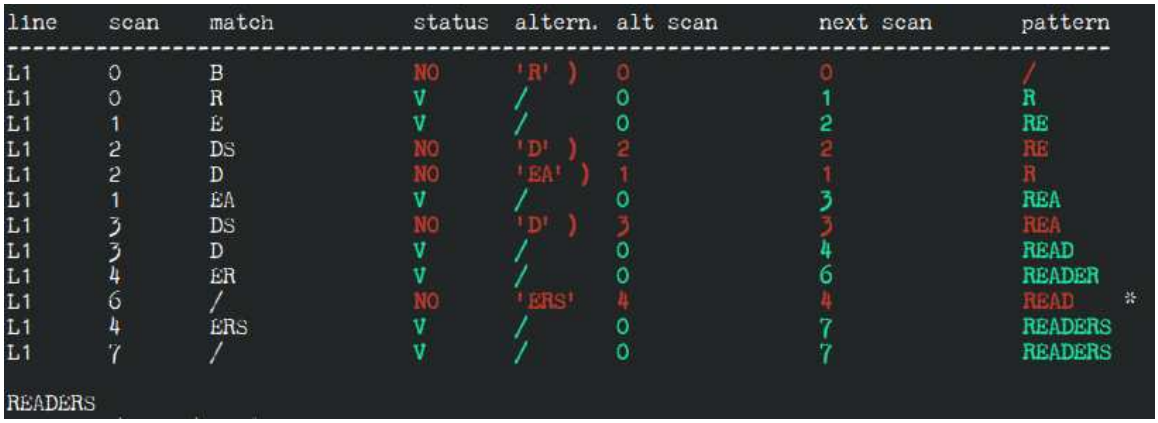
```
      'READERS' ( ( 'B' | 'R' ) ( 'E' | 'EA' ) ( 'DS' | 'D' ) ( 'ER' | 'ERS' ) )
+ . OUTPUT RPOS(0)
```

Here is the execution-time output:

```
line    scan    match           status  altern. alt scan        next scan       pattern
-------------------------------------------------------------------------------------
L1      0       B               NO      'R' )   0               0               /
L1      0       R               V       /       0               1               R
L1      1       E               V       /       0               2               RE
L1      2       DS              NO      'D' )   2               2               RE
L1      2       D               NO      'EA' )  1               1               R
L1      1       EA              V       /       0               3               REA
L1      3       DS              NO      'D' )   3               3               REA
L1      3       D               V       /       0               4               READ
L1      4       ER              V       /       0               6               READER
L1      6       /               NO      'ERS'   4               4               READ    *
L1      4       ERS             V       /       0               7               READERS
L1      7       /               V       /       0               7               READERS

READERS
```

Again, here's what is looks like in colours from my monitor:



```
line    scan    match           status  altern.  alt scan       next scan      pattern
-------------------------------------------------------------------------------------
L1      0       B               NO      'R' )    0              0              /
L1      0       R               V       /        0              1              R
L1      1       E               V       /        0              2              RE
L1      2       DS              NO      'D' )    2              2              RE
L1      2       D               NO      'EA' )   1              1              R
L1      1       EA              V       /        0              3              REA
L1      3       DS              NO      'D' )    3              3              REA
L1      3       D               V       /        0              4              READ
L1      4       ER              V       /        0              6              READER
L1      6       /               NO      'ERS'    4              4              READ    *
L1      4       ERS             V       /        0              7              READERS
L1      7       /               V       /        0              7              READERS

READERS
```

As you can see from the previous output examples (run with option `-p`), this feature decomposes the pattern evaluation into its internal details, letting you see the complete path that led to the matches, rematches, unmatches occurred in the evaluation, with particular emphasis on the alternations, which modify the search, until the final result (success or failure).

Any regular program output is preceded (and followed, if not last) by an empty line, so that it can be distinguished from the debug output.

As already mentioned, the `-p/--pattern-debug` option activates the pattern debug on all lines where there is a pattern evaluation (not an assignment or a direct commands), and no user intervention is required, because the program flow is continuous.

The pattern evaluation debugger can be activated on a specific line in a normal debug session by means of the command `BREAK`, and disabled with `DISABLE`.

Note: You may notice that sometimes the strings returned by the pattern column are incredibly long. This depends on the fact that the pattern evaluation is performed in fullscan mode. See for instance the program `tab.sno` included in the package; run it with option `-p`.

8.3. Variables tracing

The procedure *TRACE(arg_nam)* accepts as argument the name of a variable as a Name item or in String format. If the variable was not instantiated before the *TRACE()* statement, it will be (as a natural variable of type String) for later assignments or usage. This is an enhancement with respect to other Snobol4 environments, because they usually either ignore the tracing because the variable was not found in the variables list, or they return an error message for the same reason.

TRACE() can keep trace of up to 256 variables. The tracing output appears when:

- The variable is directly assigned, as in `VAR = <something>`
- The variable is assigned by an indirect process, like in `. VAR`, `$ VAR` or `@VAR`;
- The variable is changed by the pattern substitution, e.g. `VAR LEN(1) = NULL`

The command *TRACE()* fails if:

- The argument is not a Name or a String value;
- The variable has type Array or Data, that cannot be traced as-is;
- There is a request to trace one more variable than the maximum value.

The command *TRACE()* does not print anything if the keyword *&TRACE* is zero, as it is at start. To enable tracing, *&TRACE* must be set to a positive integer; at each traced line, *&TRACE* is decremented by 1, and when it reaches zero, no more traced lines are printed. So, being *&TRACE* a 32-bit integer, up to 2147483647 tracing calls can be output.

The output string pattern of *TRACE()* is similar to the one used in the Bell version, and appears as following:

```
STATEMENT <n>: <varname> = 'varcontent', TIME = <t>
```

where <n> is the line number where the statement is located, <varname> is the traced variable name, <varcontent> is the new assigned content of the variable, not enclosed in any quotes, <t> is the timing in milliseconds counted from the start of the execution.

To remove one association from tracing, the command *UNTRACE(arg_nam)* must be used, with the same rules of *TRACE()*, for the variable name. *UNTRACE()* has alias *STOPTR(arg_nam)* with identical usage.

The command *UNTRACE()* fails if:

- The argument is not is not a Name or a String value;
- The indicated variable is not in the tracing list.

If the form `UNTRACE()` without arguments is used, all associations with the traced variables are removed.

Notes:

- Tracing slows down the execution process, because a search for a variable name is done at every invocation, and clutters up the program output. So use this feature only when necessary.
- Tracing is available also during debugging, and its output is not in reverse colours.
- The debug commands *TRACE* and *UNTRACE*, are available also, in a slightly different syntax, in the debugger, as commands `TRACE` and `UNTRACE`, letting the user enabling the tracing from within the debug, without the need of altering the program text. Option `--trace=<varname>` also enables the tracing on-the-fly from the command input. It depends on *&TRACE* anyway. See also the debug chapter.

8.3.1. The original Bell TRACE()

The original tracing procedure in the Bell version had two arguments, the second being the identifying context where the variable must be traced; the original Bell included:

`CALL` for tracing the level where the traced-user-procedure is called
`LABEL` for tracing go-tos to the traced-label's name
`VALUE` for tracing the changing of the traced-variable's value
`RETURN` for tracing the traced-user-procedure return value
`FUNCTION` for tracing the arguments passed to the traced-user-procedure
`KEYWORD` for tracing the changing of the keyword's value

soft, instead, has no second argument, and operates in practice always in `VALUE` mode. The function tracing (with `FUNCTION` and `RETURN` contexts) is operated by *&FTRACE* (see the next chapter).

8.4. User-functions tracing

The keyword *&FTRACE* is used to coordinate the user-functions tracing. At start it contains zero, and the tracing is disabled. Whenever it is set to a positive Integer, the tracing is enabled, and *&FTRACE* is decremented by one at each printed line. When it reaches zero, the tracing is disabled again. So, being *&FTRACE* a 32-bit integer, up to 2147483647 tracing calls can be output.

The output string pattern is similar to the one used in the Bell version, and appears as following:

```
STATEMENT <n>: LEVEL <l> CALL OF <name>(<arglist>), TIME = <t>
```

where <n> is the line number where the statement is located, <l> is the calling level (where the level zero is the main level, incremented at each user-procedure call), <name> is the name of the user-procedure, <arglist> is the list of arguments, separated by commas, with their current passed value, <t> is the timing in milliseconds counted from the start of the execution. This line is printed after the gathering of arguments (that may involve other call levels), so that you see the lines in couples open-close, rather that in mixed order, like was in the Bell version, as shown on pages 156-157 of the GB.

When the user function leaves, a closing line is printed:

```
STATEMENT <n>: LEVEL <l> RETURN OF <name> = <s>, TIME = <t>
```

where <n> is the line number where the statement is located, <l> is the calling level, <name> is the name of the user-function, <s> is the calculated return value of the user-function, <t> is the timing in milliseconds counted from the start of the execution.

The current Snobol4 and Spitbol interpreters evaluate one instance of *&FTRACE* for the starting line and another for the closing line, so that you consume two counts for each function. soft, in the conviction that a starting line alone is meaningless, counts one instance of *&FTRACE* for each function, including in the count both the starting and the closing line. So that, for

```
&FTRACE = 5
```

you will see ten lines of output for five functions traces, while in the current Snobol4 and Spitbol interpreters you would see 5 lines for two and a half functions traces. This is a difference with the Bell behaviour, but who cares? This is not a Bell emulator!

8.5. The vim syntax file

As the last tool of debugging, the soft package contains a vim syntax file. The coloured text helps in finding errors, and vim (or gvim) is clever in even finding unpaired brackets or unclosed quotes.

Working with vim and syntax colouring is primary for me. And I can write a vim syntax file, so here is an example of how it works in gvim:

```
* COMMENT UNTIL THE END OF LINE TODO
LAB1    A = 3   ;LAB2   B = 4
        OUTPUT = "A = "
. A
        OUTPUT = "B = " B
# COMMENT UNTIL THE END OF LINE FIXME
        OUTPUT = "CIAO"
END     COMMENT
COMMENT TODO
COMMENT FIXME
```

How a vim syntax file for soft *work*

vim can be downloaded here:

http://www.vim.org "vim Site"

But I know that the other half (maybe the predominant half) uses emacs. I ask any of you capable of writing an emacs syntax file to help me and send one to me. I will include it in the package.

9. ERROR-CATCHING

The soft interpreter loads the whole program, produces a meta-file and then interprets each statement according to the Snobol4 syntax, parsing lines in the given order or by the order set by the various go-tos. The type of error determines the process:

- **Unconditionally fatal**: it's a type of unrecoverable error that stops the program execution and emits an error message (suitable to understand the kind of error) printed to the standard error channel.
- **Conditionally fatal**: it's a type of recoverable error that makes the clause fail; it's responsibility of the programmer to catch it, through a proper go-to, and direct the execution to solve the problem, or completely ignore it.

In any case, the line number where the last error found is stored in *&ERRLINE*, the code of the last error found is stored in *&ERRTYPE*, and the String value or the error message is returned by *ERRTEXT()*, which also resets *&ERRTYPE* and *&ERRLINE* to zero.

The error messages list was started incorporating all the error messages of the original Berkeley environment, the ones listed on the GGM. But soon I discovered that either they described errors that were impossible to catch in my design, or they missed some error messages that could happen in my design.

So, I had to expand the error messages list, and also remove those that were unnecessary. I maintained, for those that were introduced anew, the same *flavour* of the English language used by the extensors of the original Berkeley error messages list, but since I'm not an English mother-tongue, maybe I missed some subtlety.

The format of the error message of an unrecoverable error depends on the fact that it may happen out of or into a statement; if the error happens out of a Snobol4 statement (that is it happens before the start or after the program termination), the scheme is the following: the error message is printed preceded by a question mark:

```
? SNOBOL4 FILE NAME NOT GIVEN.
```

If the error happens into a Snobol4 statement (that is during the program execution), the error scheme is more specialized:

```
ON PROGRAM prog.sno:
? ERROR 15 IN STATEMENT 4 AT LEVEL 0:
DIVISION BY ZERO WAS ATTEMPTED.
        OUTPUT = 4 / B
                   ^
```

As you can see, the file name, the code number, the statement number, the programmer-defined procedure call level, the error message, the incriminated line and a (hopefully) precise position mark of the error are all reported, and the error is thus easily detectable. In case you want to find the mistaken line in the listing, use options `-n`/`--numbered-list`:

```
$ soft -n prog

[001]      N = 0
[002] REDO N = N + 1
[003]      OUTPUT = LE(N,95) N ": " ERRTEXT(N) :S(REDO)
[004]      OUTPUT = 4 / B
[005] END
```

You now see that line 4 uses an uninitialized variable that makes the equation fail.

The list of errors, for the current version, can be generated using option `--errors-list`. Please not that:

1 errors 46, 78, 79 and 107 require a parameter, and if you need to print them, add a second String parameter to *ERRTEXT()*, that will be added in substitution of the `%s` or `%d` tokens (error 46 requires an Integer, the others need a String);

2 errors 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 and 110 are unused and return the empty String;

3 error 0 is not a valid error code.

Here followed the errors list:

```
 1 THE LEFT OPERAND FOR A PATTERN MATCH MUST BE A STRING.
 2 THE RIGHT OPERAND FOR A PATTERN MATCH MUST BE A PATTERN.
 3 PATTERN MATCH WITH REPLACEMENT REQUIRES STRING-VALUED RIGHT HAND SIDE.
 4 TRANSFER TO AN UNDEFINED LABEL.
 5 A FAILURE OCCURRED IN THE EVALUATION OF THE GO-TO PART.
 6 TYPE ERROR IN GO-TO PART.
 7 TOO MANY PRAGMA OPTIONS.
 8 COULD NOT CONNECT TO THE I/O PROCEDURE.
 9 THE VALUE OF A VARIABLE IN A DEFERRED-EVALUATION PATTERN (UNARY *) MUST BE A PATTERN OR STRING.
11 INDIRECT REFERENCE TO THE NULL STRING.
12 ILLEGAL OPERAND FOR CONDITIONAL ASSIGNMENT.
13 NON-INTEGER STRING USED IN NUMERIC CONTEXT.
14 TYPE ERROR IN NUMERIC CONTEXT.
15 DIVISION BY ZERO WAS ATTEMPTED.
16 ARGUMENT FOR LEN, POS, RPOS, TAB, OR RTAB MUST BE IN THE INTERVAL [0,2048].
17 REAL ARITHMETIC OVERFLOW.
18 ILLEGAL USAGE OF OPERATOR @.
19 SYNTAX ERROR IN SYSTEM PROCEDURE.
21 SYNTAX ERROR IN STRING TO BE COMPILED.
22 INCORRECT SYNTAX FOR STRING TO BE CONVERTED TO REAL.
23 IMPROPER ARGUMENT FOR PSEUDO-FIELD FUNCTION (FIRST, REST, LEFT, RIGHT, PARAM, FAMILY,
   OR SELECTOR).
24 CALL OF AN UNDEFINED PROCEDURE.
25 SYNTAX ERROR IN PROCEDURE PROTOTYPE.
26 RETURN FROM LEVEL ZERO.
27 AN -NRETURN- WAS EXPECTED FROM THE PROCEDURE CALLED.
28 A PROCEDURE RETURNING BY -NRETURN- MUST SUPPLY A NAME AS ITS VALUE.
29 VARIABLE TO THE LEFT OF A [ DOES NOT CONTAIN AN ARRAY.
31 TOO FEW SUBSCRIPTS IN AN ARRAY REFERENCE.
32 ILLEGAL CHARACTER IN ARRAY PROTOTYPE.
33 SYNTAX ERROR IN ARRAY PROTOTYPE.
34 LOWER BOUND GREATER THAN UPPER BOUND IN ARRAY PROTOTYPE.
35 AN ARRAY BOUND WAS TOO LARGE.
36 AN ARRAY DIMENSION WAS TOO LARGE.
37 TOO MANY ELEMENTS FOR AN ARRAY.
38 SYNTAX ERROR IN SELECTOR FOR ITEM().
39 SYNTAX ERROR IN DATA PROTOTYPE.
41 DATA CONSTRUCTOR CANNOT SUPPLY A NAME.
42 THE PARAMETER FOR A FIELD FUNCTION WAS NOT A DATA REFERENCE.
43 NO SUCH FIELD IN THE REFERENCED DATA STRUCTURE.
44 FILE SPECIFIED TO I/O PROCEDURE MUST BE CURRENTLY ATTACHED.
45 ILLEGAL FILENAME GIVEN TO I/O ASSOCIATION PROCEDURE.
46 WARNING: ATTEMPT TO READ PAST END-OF-INFORMATION. LEVEL IS %d.
47 NULL STRING.
48 ONLY STRINGS MAY BE OUTPUT.
49 DUPLICATED NAMES IN DATA PROTOTYPE.
51 THE STATEMENT LIMIT HAS BEEN EXCEEDED.
52 TOO MANY SUBSCRIPTS IN AN ARRAY REFERENCE.
53 ILLEGAL LABEL NAME.
54 THE SPECIFIED VARIABLE IS NOT ATTACHED TO AN INPUT CHANNEL.
55 THE SPECIFIED VARIABLE IS NOT ATTACHED TO AN OUTPUT CHANNEL.
56 UNBALANCED ROUND OR SQUARE BRACKETS.
57 ILLEGAL ARRAY ITEM REQUESTED.
58 THE ENTRY POINT OF USER PROCEDURE WAS NOT FOUND.
```

```
 59 LABEL IS NOT UNIQUE.
 61 THE MAXIMUM NUMBER OF TRACED VARIABLES HAS BEEN EXCEEDED.
 62 ILLEGAL VARIABLE NAME FOR TRACING.
 63 VARIABLE NOT FOUND IN THE TRACING LIST.
 64 ILLEGAL VARIABLE NAME.
 65 MISSING ARGUMENT.
 66 VARIABLE NOT FOUND.
 67 ILLEGAL ASSIGNMENT ON PROTECTED SYSTEM VARIABLE.
 68 SNOBOL4 FILE NOT FOUND.
 69 FILE NOT FOUND OR IMPROPER NAME GIVEN.
 71 SNOBOL4 FILE NAME NOT GIVEN.
 72 ILLEGAL ERROR CODE.
 73 MEMORY ERROR. TERMINATING.
 74 TOO MANY VARIABLES. TERMINATING.
 75 TOO MANY INCLUDED FILES. TERMINATING.
 76 OUTPUT ERROR: COULDN'T CREATE THE OUTPUT FILE.
 77 INPUT ERROR: COULDN'T CREATE THE INPUT FILE.
 78 WARNING: UNRECOGNIZED OPTION -%s. IGNORED.
 79 WARNING: UNRECOGNIZED OPTION %s. IGNORED.
 81 TOO MANY FILES. TERMINATING.
 82 TOO MANY LABELS. TERMINATING.
 83 TOO MANY ALTERNATION ITEMS. TERMINATING.
 84 TOO MANY DATA ITEMS. TERMINATING.
 85 TOO MANY DEFINE ITEMS. TERMINATING.
 86 AMBIGUOUS GO-TO ADDRESS.
 87 NULL STRING IN ARBNO() ARGUMENT.
 88 UNEXPECTED FAILURE IN -NOFAIL MODE.
 89 TOO MANY PATTERNS IN THE EVALUATION. TERMINATING.
 91 ILLEGAL NAME ASSIGNMENT.
 92 CONCATENATION OPERAND MUST NOT BE ARRAY OR DATA.
 93 ILLEGAL ARGUMENT FOR PARAM().
 94 NULL ARGUMENT FOR PATTERN STRING PROCEDURE0EAK, SPAN, ANY OR NOTANY).
 95 ILLEGAL KEYWORD USED.
 96 I/O ERROR: FREEZE() FILE INACCESSIBLE.
 97 WARNING: FREEZE() NOT AVAILABLE IN PIE.
 98 DATA NOT ALIGNED IN FREEZE() FILE.
 99 CANNOT OPERATE ON STANDARD INPUT/OUTPUT CHANNELS.
101 ILLEGAL POSITION REQUESTED FOR SET() OR SEEK().
102 ATTEMPT TO RENAME A FILE ALREADY OPENED.
103 ILLEGAL CHARACTER.
104 ILLEGAL TYPE FOR CONVERT().
105 FAILURE IN EVALUATING CONVERT().
106 CORRUPTED SNOBOL FILE. TERMINATING.
107 THE FILE %s IS PROBABLY NOT A TEXTUAL FILE.\nFORCE OPENING [y/n]?
108 SNOBOL FILE NOT OPEN.
109 TOO MANY PARENTHESES. TERMINATING.
111 DEFINE() CANNOT RETURN CODE ITEMS IN THIS VERSION.
112 COPY() CANNOT COPY CODE ITEMS IN THIS VERSION.
```

10. IN THE END...

This program was designed, written and implemented just for fun. I'm not a professional programmer. But I like programming a lot. So here it is.

This manual was written using gvim and it was type-processed using groff. If you find errors or if you don't like something, it's entirely my fault.

If you find any bugs, which are all the result of my inexperience and are due to me and me alone, write to <ing dot antonio dot maschio at gmail dot com>, do not hesitate. I will try to fix the bugs as soon as possible.

Two things are still to be said:

The first is that I hope you like soft.

The second is: it's GPL, enjoy! ♥

Appendix 1: **CONCEPTS INDEX**

The concept index reports all the relevant arguments in the manual; if you think something is missing, please report it.

Appendix 2: **LANGUAGE ELEMENTS INDEX**

Appendix 3: **THE DIFFERENCES BETWEEN BERKELEY AND BELL (AND soft)**

As reported quite anywhere in this manual, the two versions Bell and Berkeley share many Snobol4 characteristics (the main ones) but also are different enough as to make programs not always portable between the two system.

So, for your knowledge, the following table summarizes the differences between the two Snobol4 Bell and Berkeley, and also reports the behaviour of soft in relation to the various topics. All arguments from Appendix M of the GGM, plus some other, are here reported. If you should find that some datum is missing, please write me, and I will add it to the table.

Topic	**Berkeley**	**Bell**	**soft**
Extra parameters in system procedures	ignored	breaks or fails	uses the Bell protocol
Extra parameters in user procedures	ignored	ignored	breaks or fails
Collating sequence in character set:	6-bit CDC display code (CDC 6000) with 63 characters among `ABCDEFGHIJKLM NOPQRSTUVWXYZ 0123456789 !"$%&/()='? [+*]@,;.:-|<>-` plus the blank space, but without lower letters	8-bit ASCII (IBM System 360) (with lower letters)	8-bit ASCII (PC with Unix/Linux) (with lower letters), case insensitive for all procedures and variables names.
INPUT() and *OUTPUT()* arguments	uses its own I/O protocol	uses the FORTRAN IV I/O protocol	simulates the Berkeley protocol
ARRAY()'s second argument	not available	available	uses the Bell protocol
test `IDENT('1',1)`	succeeds	fails	uses the Berkeley protocol
Redefinition of labels in *CODE()* segments	not available	available	uses the Berkeley protocol
CONVERT()	has one argument and a limited usage	has two arguments and a wider usage	uses both protocols (adapting the Bell one to the Berkeley types)
Interrogation operator	implemented as *IF()*	implemented as *?*	uses the Berkeley protocol
Unary operator * (asterisk)	only for simple natural variables	for all variables and Expressions	uses the Berkeley protocol
Name operator . (dot)	returns always a Name type	returns a String type if applied to natural variables	uses the Berkeley protocol
test `IDENT('VAR',.VAR)`	fails	succeeds	uses the Berkeley protocol
Multiplication and division precedence	they have the same precedence	multiplication has a higher precedence than the division	uses the Berkeley protocol
Keywords	not available	available	uses the Bell protocol (with differences)
Empty strings	interpreted as zero in numeric contexts, empty Strings in string contexts	always interpreted as empty Strings	uses the Berkeley protocol
Unary operators '+' and '-'	if used isolated, they are interpreted as the number zero	they must be bound to the following digits	uses the Bell protocol (advanced built-in arithmetic engine)

Special label names *RETURN*, *FRETURN* and *NRETURN*	they can be used as labels, losing their special meaning	they cannot be used as labels	uses the Bell protocol
Special label name *END*	it can be used as label, losing its special meaning	it cannot be used as a label	uses the Berkeley protocol, but retains its special meaning in case of option `--r/--resource`; in any case, when *END* is found in a go-to, the program terminates
Use of label after *END* setting the start execution	not available	available	uses the Bell protocol
Printing objects with *OUTPUT*	an error is printed if the object is neither a number nor a String	a special symbol depicting the prototype is printed	uses the Bell protocol
String length	131.070 characters at most	virtually unlimited	2047 characters (plus the terminator) at most
OUTPUT limits	no more than 132 characters are output on a single process	132 characters for as many lines as necessary are output on a single process	uses the Berkeley protocol, but widens the output limit up to 2047 characters (plus the terminator), printed continuously without breaks
Assignment operator (=)	it does not need to be bounded by blanks	it requires to be bounded by blanks	uses the Berkeley protocol
Mathematical operators	they must be bounded by blanks	they must be bounded by blanks	uses the Berkeley protocol, though not strictly necessary for mathematical expressions
The colon of go-to	it does not need not to be preceded by a blank	it requires to be preceded by a blank	uses the Berkeley protocol
Double quote literal in strings	the double quote literal delimiter inside the same quotes represent one quote instance	they are interpreted literally as two quotes	uses the Berkeley protocol
Continuation lines breaks	statements continued over line boundaries may be broken anywhere, not necessarily where a blank appears	statements may be broken only where a blank appears	uses the Berkeley protocol
Real literals	they may begin with a signed dot or with a signed digit	they need to begin with a signed digit	uses the Berkeley protocol
Terminating *END*	not required	required	required only when option `-r/--resource` is used
Alternating characters set	available	not available	uses the Bell protocol
Labels representation	a label must be an identifier; that is, it must begin with a letter and consists of nothing but letters, numbers, and periods	a label may consist of a letter or a digit followed by any number of other characters from the entire character set, except the blank	uses the Berkeley protocol

Program directives for listing	statement numbers always appear to the left of the statements and there is no way to specify they must be set to the right.	statement numbers normally appear to the right of the statements, but it is possible to specify that they appear to either the left or the right (using `-LIST LEFT` or `-LIST RIGHT` in the listing)	uses the Bell protocol
Mixed mode arithmetic or comparison involving Real and Integer numbers	not available	available	uses the Bell protocol
Quick scan/full scan	full scan only	both	uses the Berkeley protocol
`-SPACE` directive	available	not available	uses the Berkeley protocol
Special operators @, **	not available	available	uses the Bell protocol
Special operator ¬ (negation)	not available	available	uses the Berkeley protocol
Table, Expression and External data types	not available	available	uses the Berkeley protocol
Procedure *PUNCH*	not available, settable through *OUTPUT()*	available	uses the Berkeley protocol (through intermediate temporary files)
Pattern variable *SUCCEED*	not available	available	uses the Berkeley protocol
Procedures *ARG()*, *BACKSPACE()*, *CLEAR()*, *COPY()*, *DUMP()*, *DUPL()*, *EVAL()*, *FIELD()*, *INTEGER()*, *LOCAL()*, *REMDR()*, *REPLACE()*, *STOPTR()*, *TRACE()*	not available	available	uses the Bell protocol (with some differences)
Procedures *COLLECT()*, *LOAD()*, *OPSYN()*, *TABLE()*, *UNLOAD()*, *VALUE()*	not available	available	uses the Berkeley protocol
Procedures *ITEM()* and *PROTOTYPE()*	available in extended format	available in basic format	uses the Berkeley protocol
Procedures *CLOCK()*, *TYPE()*, *PARAM()*, *FIRST()*, *REST()*, *LEFT()*, *RIGHT()*, *FAMILY()*, *SELECTOR()*, *NEXTVAR()*	available	not available	uses the Berkeley protocol (with enrichments)

Table of Contents